FPT UNIVERSITY

FPT UNIVERSITY
SoftwareDesignDocument

Frevia

GroupBar	
Group Members	Pham Kha Vy - CE182254 Nguyen Tien Thuan - CE181024 Tran Phat Tai - CE181632 Vo Nhut Tin - CE180619 Nguyen Lu Nhat Huy - CE181196
Supervisor	Mrs. Cao Hong Nga
Ext Supervisor	
Capstone Project code	FRE

- Can Tho, October 2026 -

Table of Contents

1 Overall Description **4**

1.1 Assumptions 4

1.2 Design Constraints 5

1.2.1 Platform Compatibility 5

1.2.2 Integration Complexity 5

1.2.3 Performance Requirements 5

1.3 Technology Suggestion 6

1.3.1 NestJS Framework 7

1.3.2 Next.js and React 19 Frontend 7

1.3.3 Prisma ORM and PostgreSQL 7

1.3.4 Supporting Services 7

2 System Architecture Design **8**

2.1 Overall Architecture 8

2.2 System Architecture 9

2.2.1 API Module Structure 9

2.3 Package Diagram 11

3 System Detailed Design **12**

3.1 View Forum Category List 12

3.2 View Forum Category Detail 13

3.3 View Post List 14

3.4 View Post Detail 16

3.5 Create Post 18

3.6 Update Post 19

3.7 Delete Post 21

3.8 Like Post 23

3.9 Unlike Post 25

3.10 View Post Likes 27

3.11 View Post Comments 28

3.12 Create Comment 30

3.13 Update Comment 32

3.14 Delete Comment 33

3.15 Like Comment 35

3.16 Unlike Comment 36

3.17 Report Content 38

3.18 View Conversation List . . . 41
3.19 Start Conversation . . . 42
3.20 View Conversation Detail . . . 44
3.21 Send Message . . . 46
3.22 Mark Message as Read . . . 48
3.23 Delete/Recall Message . . . 50
3.24 Pin/Unpin Conversation . . . 52
3.25 Send File . . . 54
3.26 View Moderation Queue . . . 56
3.27 View Moderation Item Detail . . . 58
3.28 Resolve Moderation Item . . . 60
3.29 View ID Verification Request List . . . 61
3.30 View ID Verification Request Detail . . . 63
3.31 Approve ID Verification . . . 64
3.32 Reject ID Verification . . . 66
3.33 View Deleted Posts/Comments List . . . 68
3.34 View Deleted Posts/Comments Detail . . . 70
3.35 Restore Deleted Posts/Comments . . . 72

4 Class Specification . . . 74

4.1 ForumController Class . . . 74
4.2 ForumService Class . . . 77
4.3 ForumRepository Class . . . 80
4.4 ForumLikeController Class . . . 85
4.5 ForumLikeService Class . . . 85
4.6 ForumLikeRepository Class . . . 86
4.7 ForumCommentController Class . . . 88
4.8 ForumCommentService Class . . . 90
4.9 ForumCommentRepository Class . . . 91
4.10 ForumReportController Class . . . 94
4.11 ForumReportService Class . . . 96
4.12 ForumReportRepository Class . . . 98
Supporting DTO classes for reports . . . 100
4.13 ConversationController Class . . . 101
4.14 ConversationService Class . . . 103
4.15 ConversationRepository Class . . . 107

4.16 ConversationsGateway Class 110
4.17 ForumAdminController Class 112
4.18 ForumAdminService Class 116
4.19 ForumAdminRepository Class 120
4.20 IdentityVerificationsAdminController Class 124
4.21 IdentityVerificationsAdminService Class 126
4.22 IdentityVerificationsAdminRepository Class 128

1 Overall Description

1.1 Assumptions

This system is designed based on the following assumptions:

- The Node.js runtime and the frameworks chosen for the project (NestJS 11 for the back-end and Next.js 16 for the frontend) will continue to be actively supported and maintained, ensuring long-term stability and compatibility with evolving technologies.
- The RESTful API architecture will remain a scalable and efficient choice for separating the Next.js presentation layer from the NestJS business layer.
- PostgreSQL, accessed through Prisma ORM 7, will continue to provide reliable, scalable, and secure relational data storage suitable for handling user identity, jobs, contracts, escrow payments, and forum data.
- Third-party services used by the platform — Stripe (payments and escrow), Cloudinary (file storage), Google (OAuth login), the AI moderation/analysis service, and mail delivery via Nodemailer — will stay available and API-compatible during the project lifetime.
- Modern web browsers (e.g., Chrome, Edge, Firefox, Safari) will continue to support the front-end standards required by Next.js/React 19, including HTML5, CSS3 (TailwindCSS), JavaScript ESM, and responsive rendering with React Server Components.
- The application will be deployed in environments with stable internet access to support real-time messaging (Socket.io) and online payment processing.
- Users (Guests, Clients, Freelancers, Experts, and Admins) are familiar with basic web application navigation: browsing job boards, filling forms, managing profiles, and following payment flows.

This system design is also based on the following risks:

- Any breaking change in the major versions of Next.js, NestJS, or Prisma could require refactoring of the user interface components or of the data access layer.
- Cross-browser inconsistencies or outdated device software may affect rendering or the real-time messaging experience.
- Cybersecurity threats such as credential stuffing, token hijacking, or database breaches could compromise sensitive personal and financial information if not proactively managed (the design therefore includes bcrypt-hashed passwords, JWT access/refresh tokens, session revocation, and role-based access control).

- Outages or policy changes of the integrated third-party services (Stripe, Cloudinary, Google OAuth, AI moderation) could temporarily disrupt payments, file uploads, login, or content moderation.
- Network connectivity issues could disrupt access to job proposals, contract collaboration, or checkout flows that rely on server interaction.

1.2 Design Constraints

This system should comply with the following constraints:

1.2.1 Platform Compatibility

The Frevia application must be compatible with modern web browsers (e.g., Chrome, Edge, Firefox, Safari) for all user roles including Guest, Client, Freelancer, Expert, and Admin. The frontend is implemented with Next.js 16 (App Router, React Server Components) styled with TailwindCSS 4 and shadcn/ui components; the backend is a NestJS 11 REST API. Responsive web design principles must be applied so that a consistent user experience is provided across desktop, tablet, and mobile browsers. Internationalization is achieved with next-intl so the interface can be served in multiple languages from the same code base.

1.2.2 Integration Complexity

The Frevia platform integrates a variety of components: the Next.js web application, the NestJS REST API, the PostgreSQL database (via Prisma 7 with the pg adapter), Stripe Checkout and Stripe Connect for escrow payments and payouts, Cloudinary for image and document storage, Socket.io for real-time chat and notifications, Google OAuth for federated login, e-mail for OTP delivery, and AI services for content moderation, CV parsing, and identity document verification. Communication between the client and the server is carried out through versioned REST endpoints secured with JWT access tokens and refresh tokens stored in dedicated session records; role-based access control restricts each operation to the allowed roles. Design decisions must keep these integrations loosely coupled behind services and repositories so that a provider can be replaced without rewriting business logic.

1.2.3 Performance Requirements

The application must remain responsive under concurrent usage by many freelancers and clients. Job browsing, proposal listing, forum feeds, and conversation histories must load promptly; therefore the design mandates server-side pagination for every list operation, database

indexes on frequently filtered columns, optimized Prisma queries executed inside transactions for counted lists, and streaming React Server Components to reduce time-to-first-byte. Real-time features (messaging, notifications) must propagate events with minimal delay via Socket.io. Payment operations must reconcile eventual states through Stripe webhooks and explicit synchronization endpoints.

1.3 Technology Suggestion

Frevia adopts a RESTful API architecture to support secure, stateless, and scalable communication between the Next.js frontend and the NestJS backend. This approach provides:

- Modular, reusable API endpoints for account management, job posting, proposals, contracting, milestone escrow, dispute resolution, community forum, and administration.
- Easy extension for mobile applications or third-party integrations in the future.
- Secure access using JWT tokens with role-based authorization for every endpoint.
- Uniform use of standard HTTP methods (GET, POST, PATCH, PUT, DELETE), making the system intuitive to maintain.
- A clear separation between the presentation layer and the business logic layer.

All request/response payloads are validated and typed with shared Zod schemas packaged in `@shared/types`, which is consumed by both the frontend and the backend so that a single source of truth prevents contract drift between the two applications.

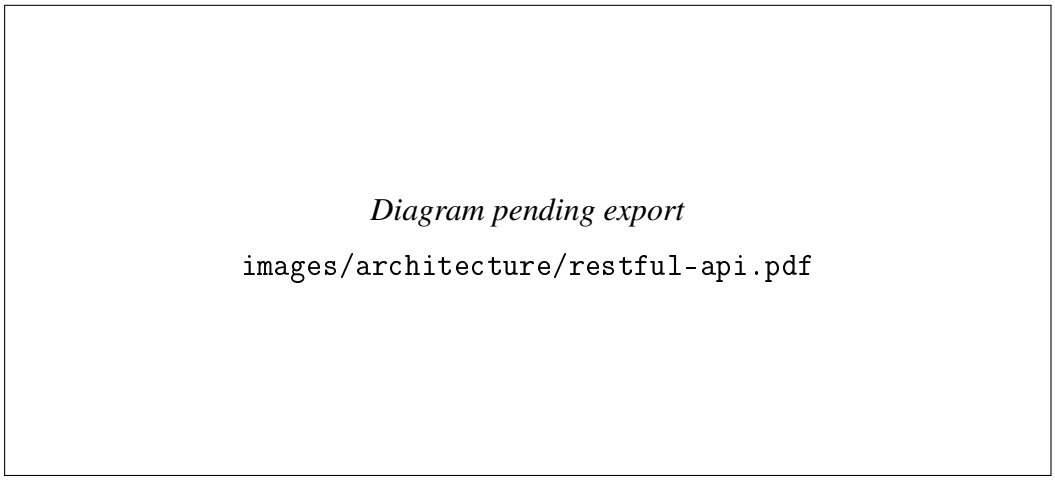


Figure 1: RESTful API communication between the Next.js frontend and the NestJS backend

1.3.1 NestJS Framework

The backend of Frevia is built with NestJS 11, a TypeScript-first, modular web framework running on Node.js. It is selected for its:

- Rich dependency-injection and module ecosystem, matching the Controller–Service–Repository layering.
- First-class decorator-based REST routing with guards and interceptors for authentication, roles, permissions, and response serialization.
- Excellent support for Zod schema validation (nestjs-zod) and typed DTOs.
- Component structure for WebSockets (Socket.io gateways) used by the messaging module.
- Straightforward integration with Prisma ORM and PostgreSQL for persistence.

1.3.2 Next.js and React 19 Frontend

The web client uses Next.js 16 with the App Router. React Server Components render public pages (job lists, job details, forum) directly from the server, while TanStack Query drives interactive islands (forms, likes, real-time chat). react-hook-form together with Zod resolvers validate user input with the same schemas the API uses.

1.3.3 Prisma ORM and PostgreSQL

Persistence relies on PostgreSQL accessed through Prisma 7. The schema defines 62 models covering identity and access, profiles, jobs, proposals, contracts and milestones, payments and disputes, reviews, messaging, the community forum, and platform configuration. Prisma's type-safe client keeps data access code verifiable at compile time.

1.3.4 Supporting Services

Payments and payouts use Stripe Checkout and Stripe Connect with webhook-driven reconciliation; binaries are stored in Cloudinary; emails (OTP, notifications) are sent with Nodemailer; Google login uses OAuth 2.0; environment configuration is loaded through Infisical so no secret is committed to the repository.

2 System Architecture Design

2.1 Overall Architecture

Frevia follows a three-tier, deployment-separated architecture (Figure 2):

- **Presentation tier — Next.js 16 web application:** server components render public pages (job board, job detail, community forum, expert directory) and private dashboards; client components handle interactivity such as forms, likes, and chat. All server access goes through a single typed HTTP client that attaches the JWT access token and refreshes it transparently when it expires.
- **Application tier — NestJS REST API:** a modular monolith organized into 31 route modules. Every module is internally layered as Controller → Service → Repository. Controllers validate and serialize with shared Zod DTOs; services hold business rules and authorization checks; repositories encapsulate all Prisma queries.
- **Data tier — PostgreSQL via Prisma 7:** the relational store for 62 domain models. Binary files are never stored in the database but in Cloudinary; real-time transport for chat and notifications is handled by a Socket.io gateway beside the REST API.

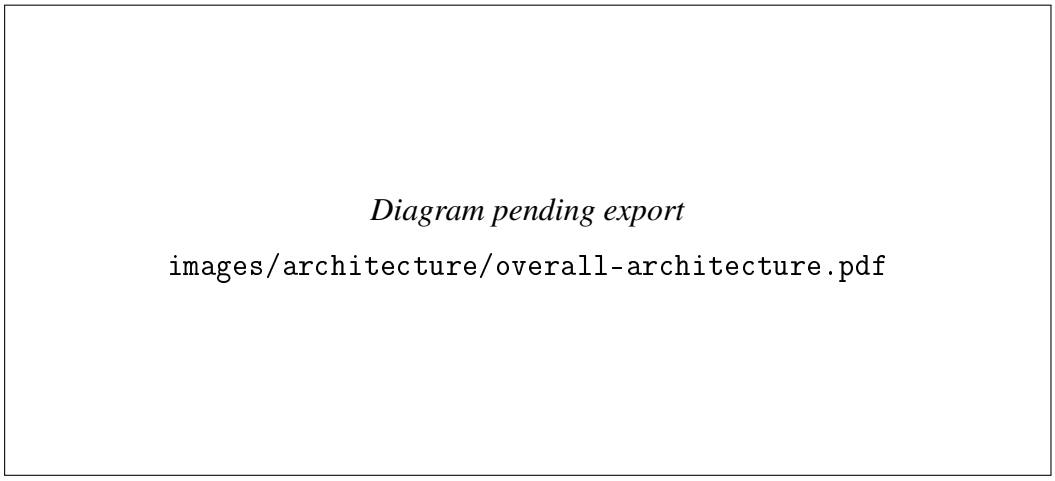


Figure 2: Overall architecture of the Frevia system

The two applications are deployed independently (the web app on Vercel, the API on a Node.js host) and communicate only through the documented REST contract validated by the shared Zod schemas package.

2.2 System Architecture

The internal architecture of the API follows a strict layered pattern, enforced identically in every module (Figure 3):

- **Controller layer** — thin Routing Layer. Exposes REST endpoints, applies guards (authentication, roles, permissions), binds validated request DTOs, and delegates to a service. Controllers never query the database directly.
- **Service layer** — business logic layer. Implements use cases, ownership/authorization checks, state-machine transitions (proposal, contract, dispute), and cross-aggregation logic; it throws typed HTTP exceptions (`NotFoundException 404`, `ForbiddenException 403`, `ConflictException 409`) that a global exception filter converts to a uniform error response.
- **Repository layer** — data access layer. The only component allowed to touch Prisma; complex list queries (pagination + counting) are wrapped in `$transaction` for consistency.
- **Shared kernel** — the `@shared/types` workspace package contains Zod models, constants, permission names, and error codes shared with the web app.

Security flow: every request carries a JWT access token; the auth guard validates it and resolves the active role; role and permission guards authorize module-specific operations; refresh tokens are stored server-side in a `Session` row and can be revoked from the session management feature.

AI-assisted pipeline: user-generated content (forum posts/comments, CVs, identity documents) is passed through AI moderation or AI analysis services; forum content can be auto-approved when clean, or queued in `ModerationQueue` for administrator approval when suspicious.

2.2.1 API Module Structure

Table 1 summarizes the 31 route modules of the API and their responsibilities.

Table 1: API route modules and responsibilities

Module	Responsibility
auth	registration, OTP, login, refresh token, logout, forgot password, Google OAuth, current account, role switching/joining
sessions	device session listing, detail, revocation

Module	Responsibility
account-profile	general account profile, avatar, identity verification upload/AI check, client profile, social links, favorite freelancers, following
users	administrator user management and skill catalog
roles, permissions	RBAC role and permission administration
profiles	professional profile, CV upload/AI analysis, skills
portfolios	portfolio item CRUD on profiles
expert-profile	expert profile management and public expert directory
profile-revisions	profile change request approval workflow
identity-verifications-admin	identity verification review
job-categories	public job category browsing
job-categories-admin	job category administration (soft delete + restore)
skills-admin	skill catalog administration (soft delete + restore)
browse-job	public job listing and job detail by slug
manage-job	client job management, bookmarks, skill search
proposals	proposal lifecycle: draft, submit, withdraw, interview, reject, accept
contract	contract lifecycle: create, update, sign, complete, cancel
milestone	milestone management inside a contract
milestone-file, milestone-submission	milestone attachments and work submission review
shared-file	contract shared workspace files
payments	Stripe Connect onboarding, platform fee, milestone funding, release, refund, dispute fee, transactions, webhook
disputes	dispute lifecycle (user side and admin arbitration side)
reviews	contract reviews and responses
conversations	direct messaging with real-time delivery
notifications	notification feed management
forums	forum categories and posts (public side)
forums-comment, forums-like, forums-reports	comments, likes, content reports
forums-admin	AI moderation queue, pending posts, trash restore, forum statistics
saved-search	saved job search filters
banners, banners-admin	advertisement banner delivery and administration

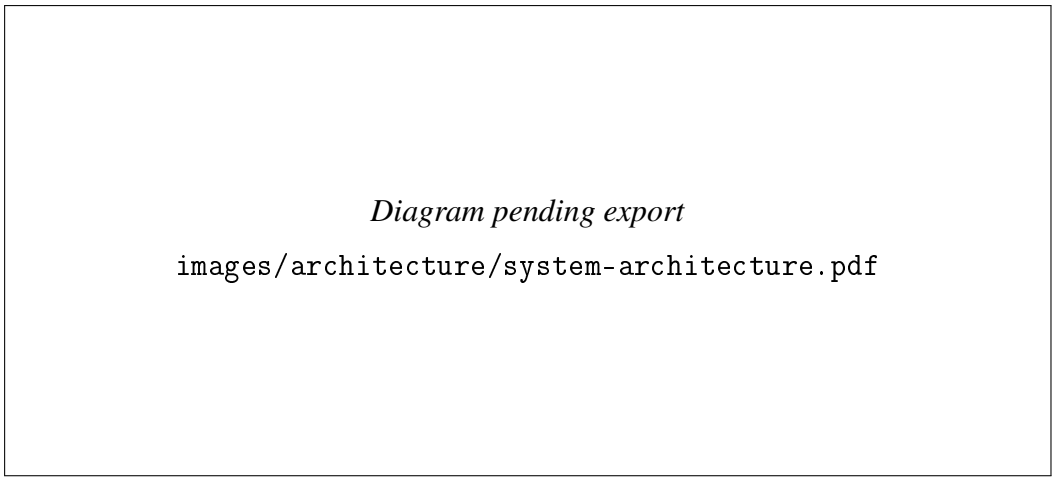


Figure 3: Layered system architecture with security and AI moderation flows

2.3 Package Diagram

The repository is a pnpm monorepo managed by Turborepo with three workspaces (Figure 4):

- `apps/web` — the Next.js application. Sub-packages group screens by actor (guest browsing, client, freelancer, admin), shared UI components (`@repo/ui`), server actions, and the i18n message catalogs used by next-intl.
- `apps/api` — the NestJS application. `src/routes/*` contains the 31 domain modules described above; `src/shared` contains cross-cutting infrastructure: authentication guards and decorators, permission constants, global HTTP exception filter, Zod serializer interceptor, Cloudinary storage service, and Prisma client setup. Each domain module folder is organized as `<module>.controller.ts`, `<module>.service.ts`, `<module>.repository.ts`, DTO classes, and module-specific error factories (e.g., `forums.error.ts`).
- `packages/shared-type` — the single source of truth for API contracts: Zod request/response models (`model/*`), enumeration/constant definitions, and permission name lists, imported by both `apps/web` and `apps/api`.

Dependencies only point downwards: `web` and `api` both depend on `shared-type`; no route module imports another route module — cross-module data needs flow through services at the Prisma schema level.

Diagram pending export

`images/architecture/package-diagram.pdf`

Figure 4: Package diagram of the Frevia monorepo

3 System Detailed Design

3.1 View Forum Category List

a. Class Diagram

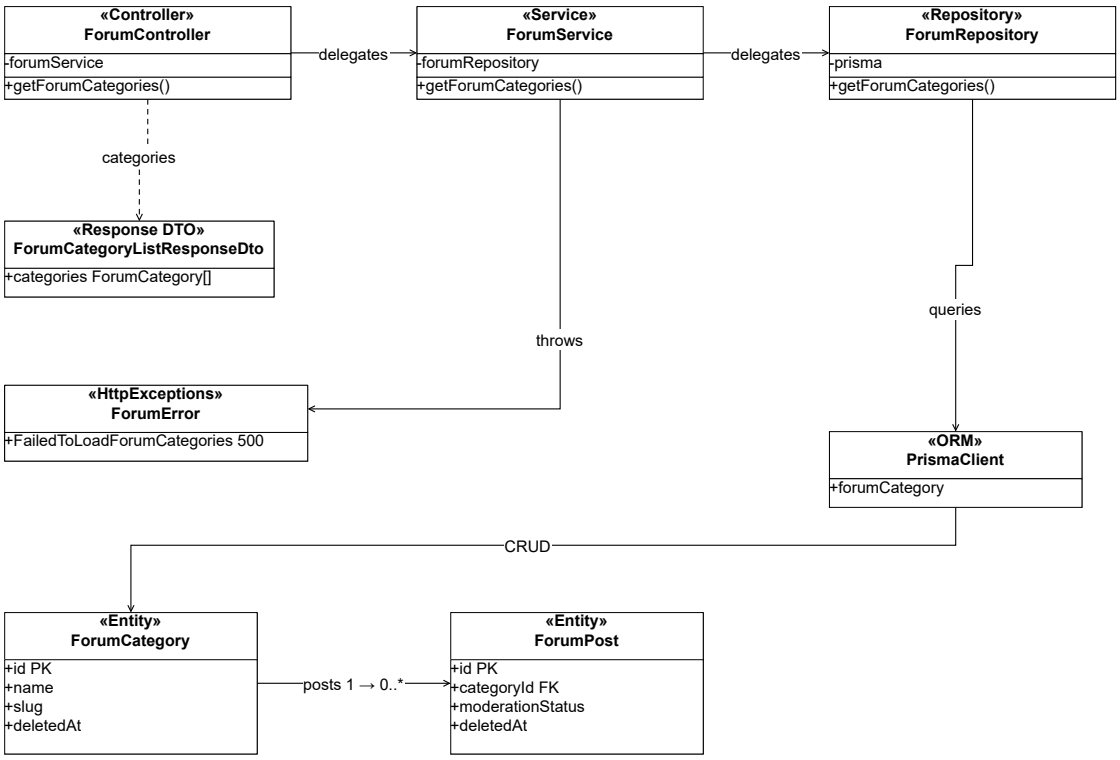


Figure 5: Class diagram of View Forum Category List

b. Sequence Diagram(s)

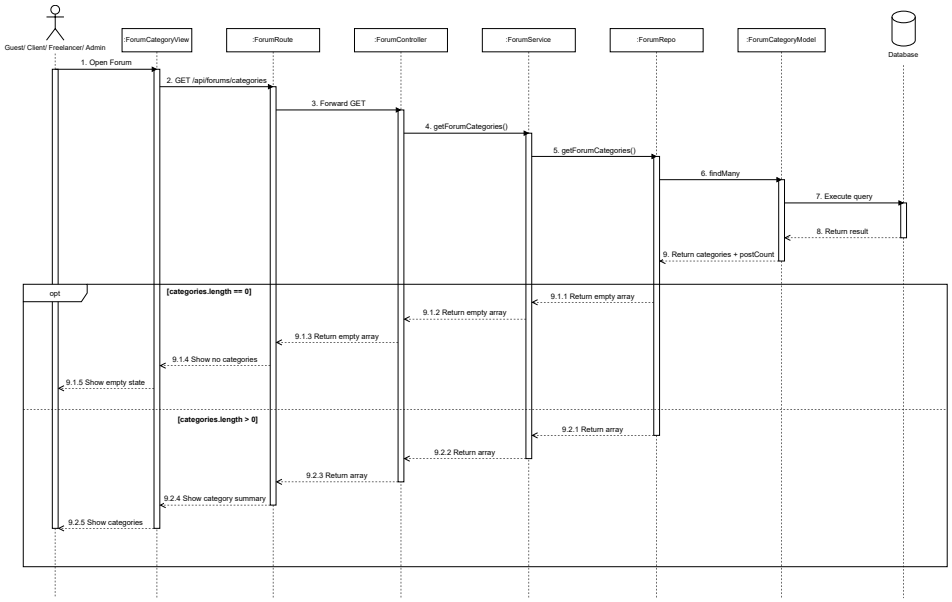


Figure 6: Sequence diagram of View Forum Category List

3.2 View Forum Category Detail

a. Class Diagram

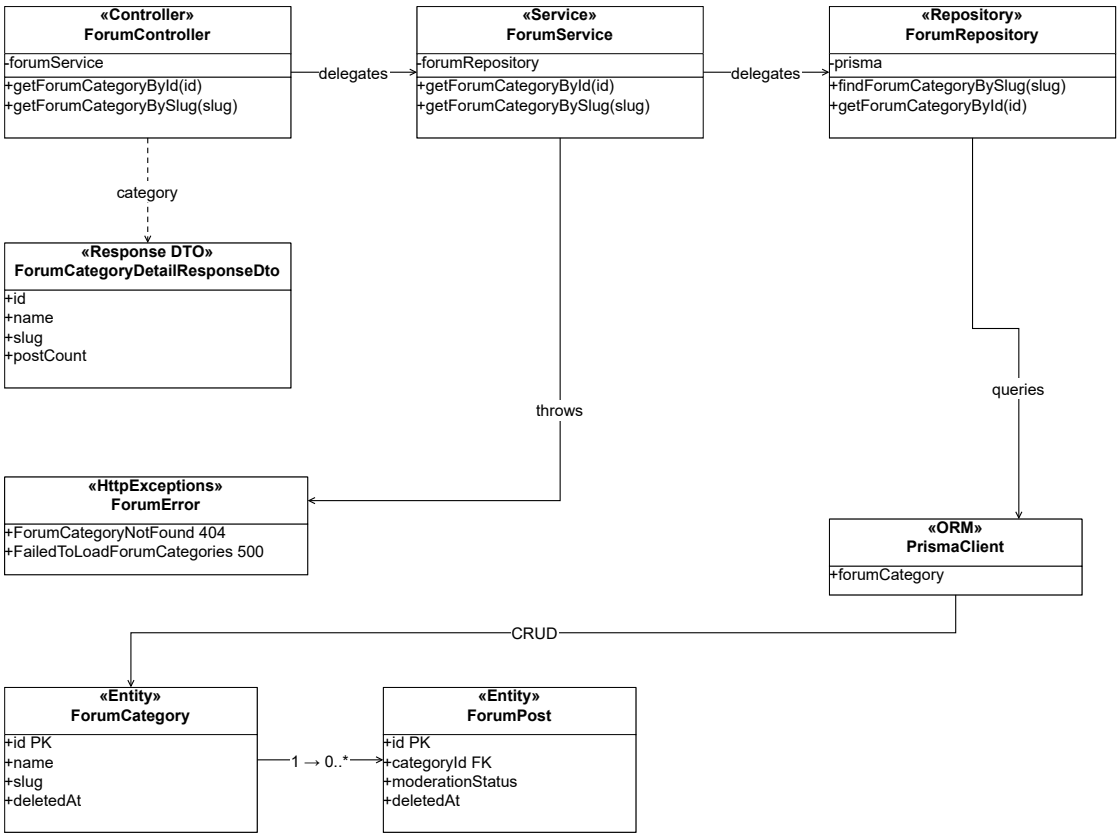


Figure 7: Class diagram of View Forum Category Detail

b. Sequence Diagram(s)

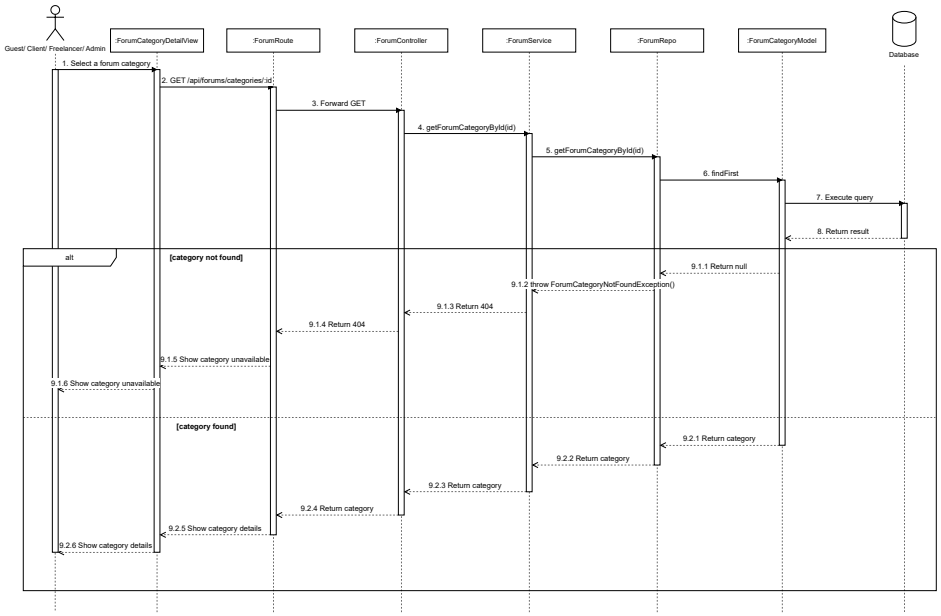


Figure 8: Sequence diagram of View Forum Category Detail

3.3 View Post List

a. Class Diagram

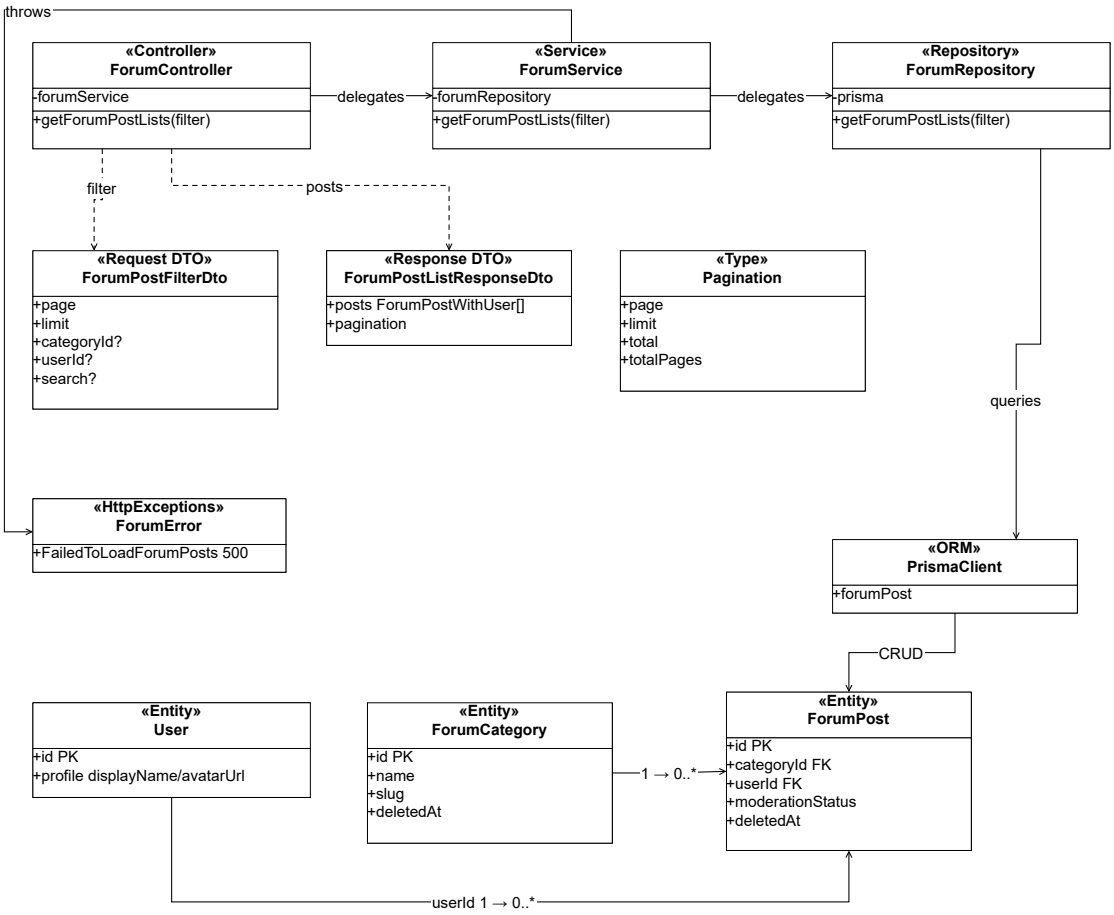


Figure 9: Class diagram of View Post List

b. Sequence Diagram(s)

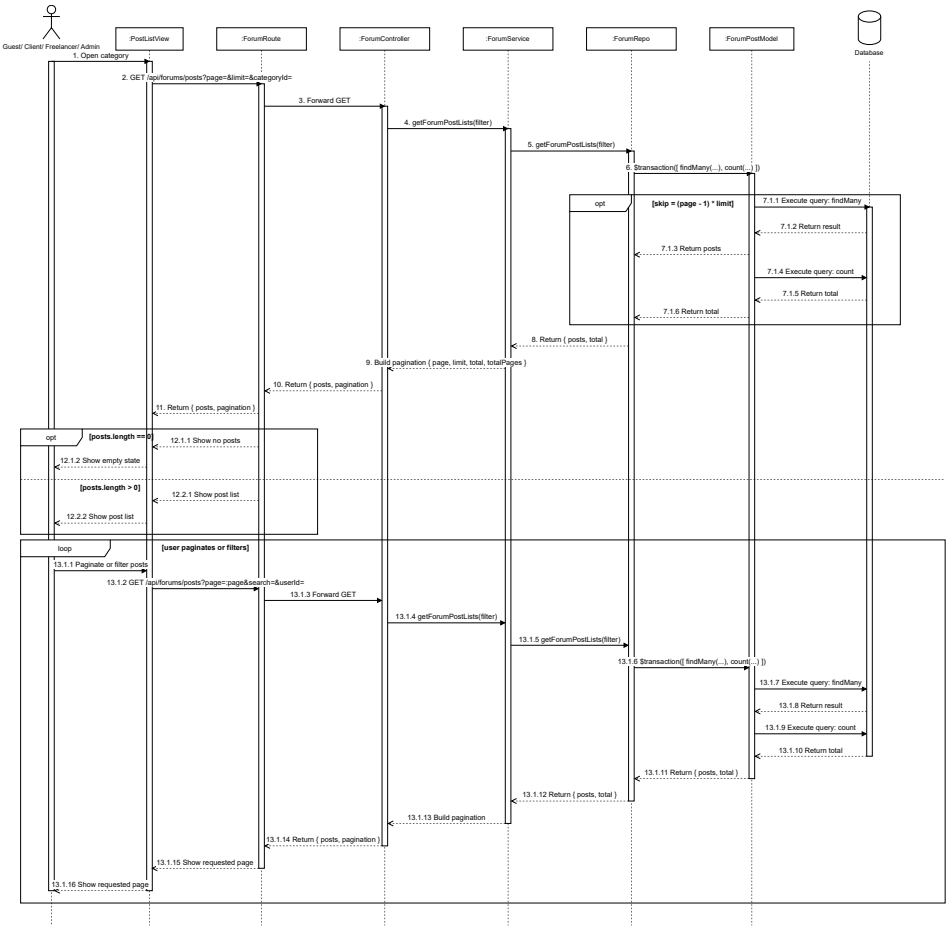


Figure 10: Sequence diagram of View Post List

3.4 View Post Detail

a. Class Diagram

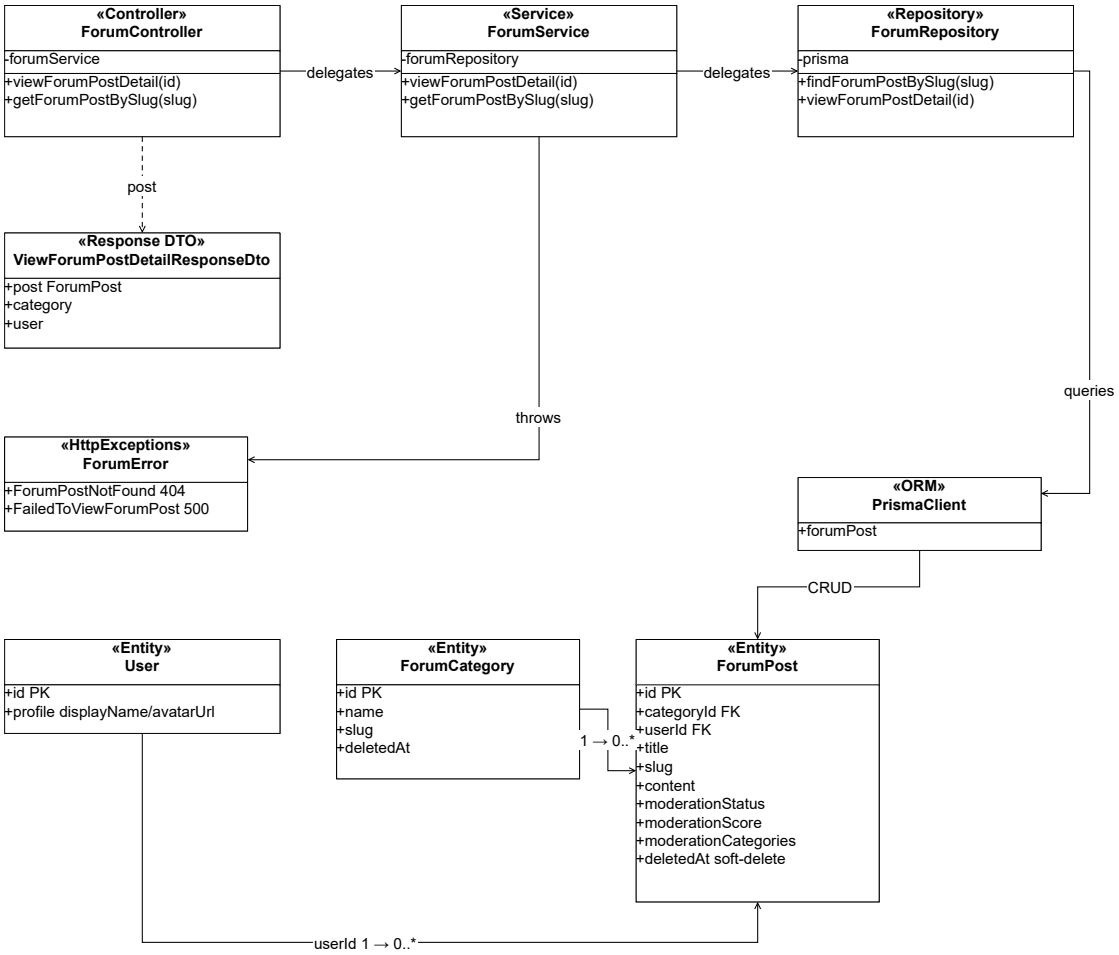


Figure 11: Class diagram of View Post Detail

b. Sequence Diagram(s)

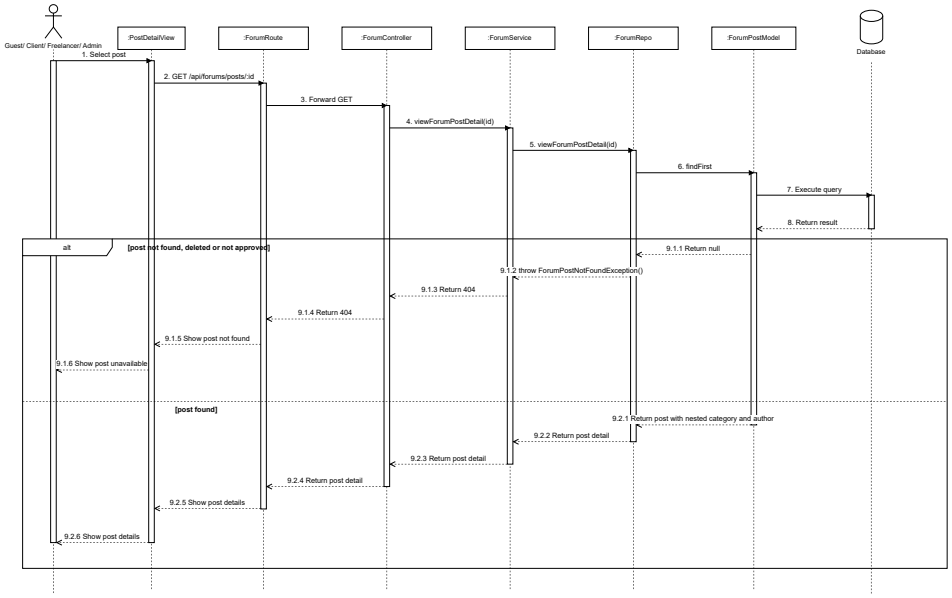


Figure 12: Sequence diagram of View Post Detail

3.5 Create Post

a. Class Diagram

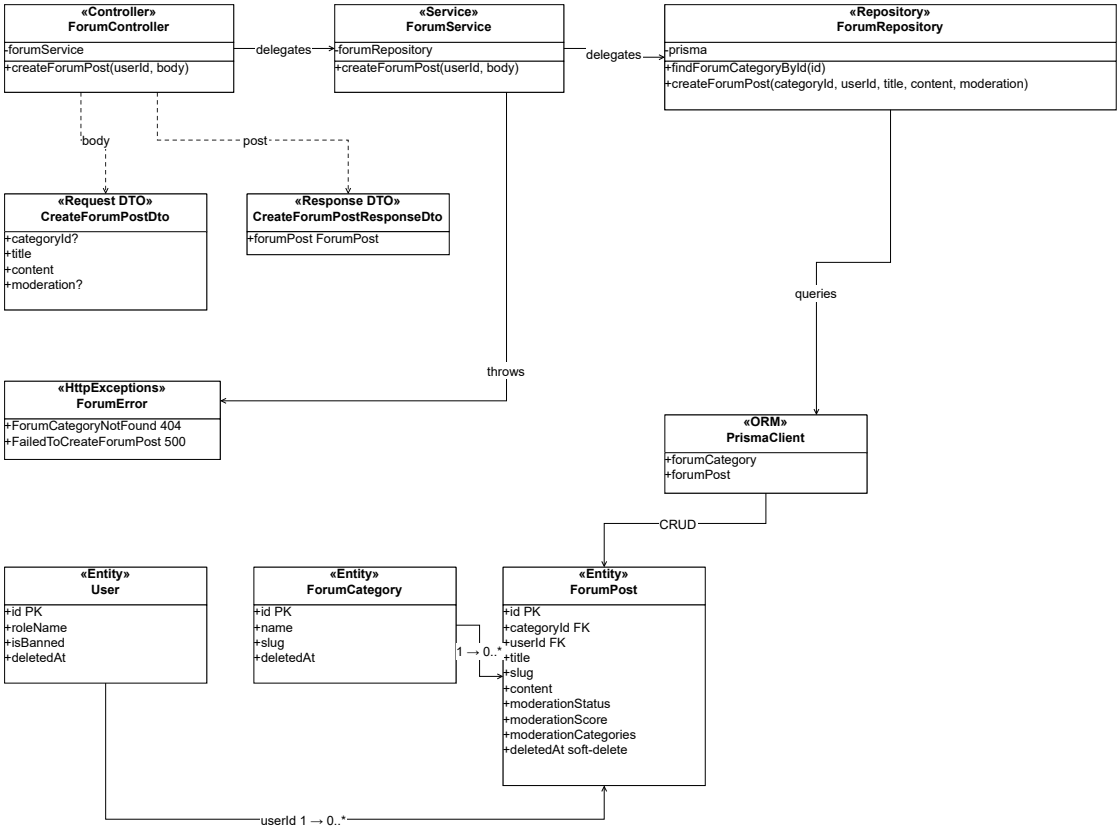


Figure 13: Class diagram of Create Post

b. Sequence Diagram(s)

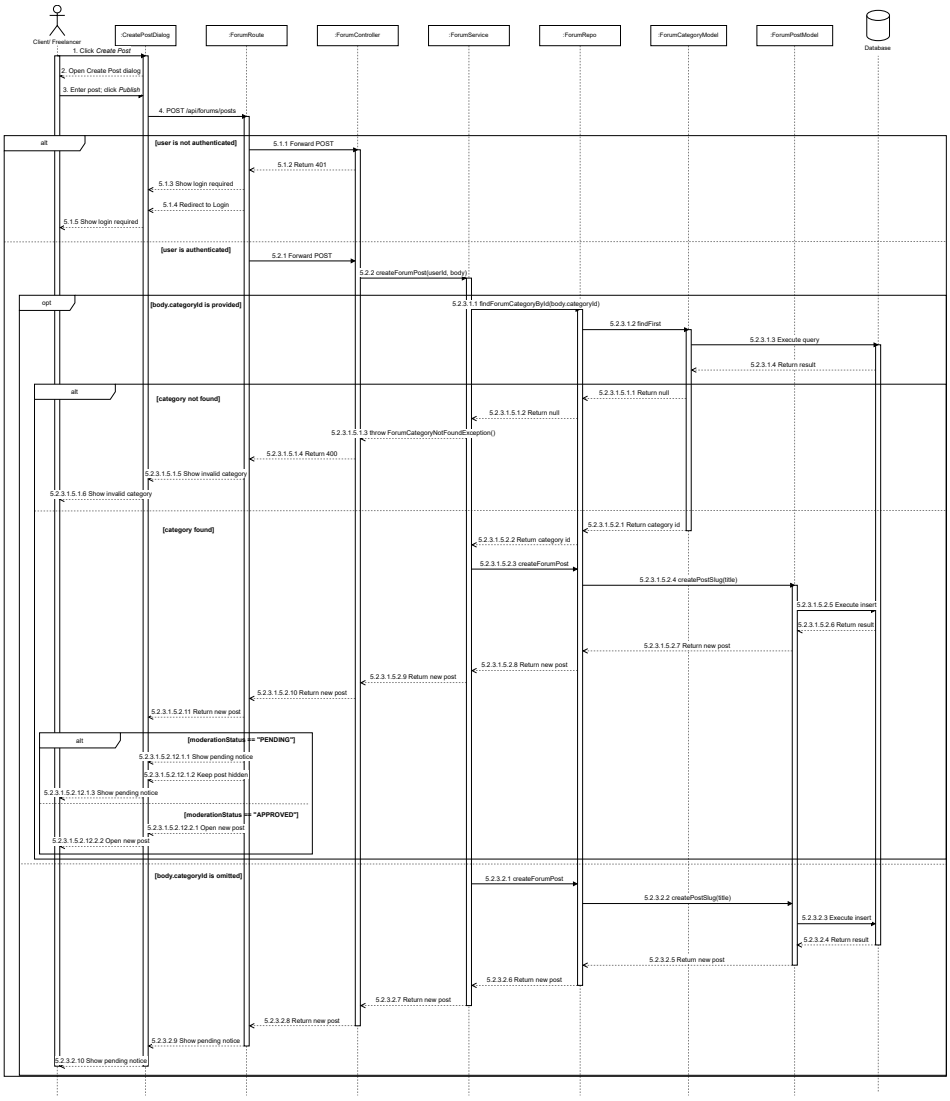


Figure 14: Sequence diagram of Create Post

3.6 Update Post

a. Class Diagram

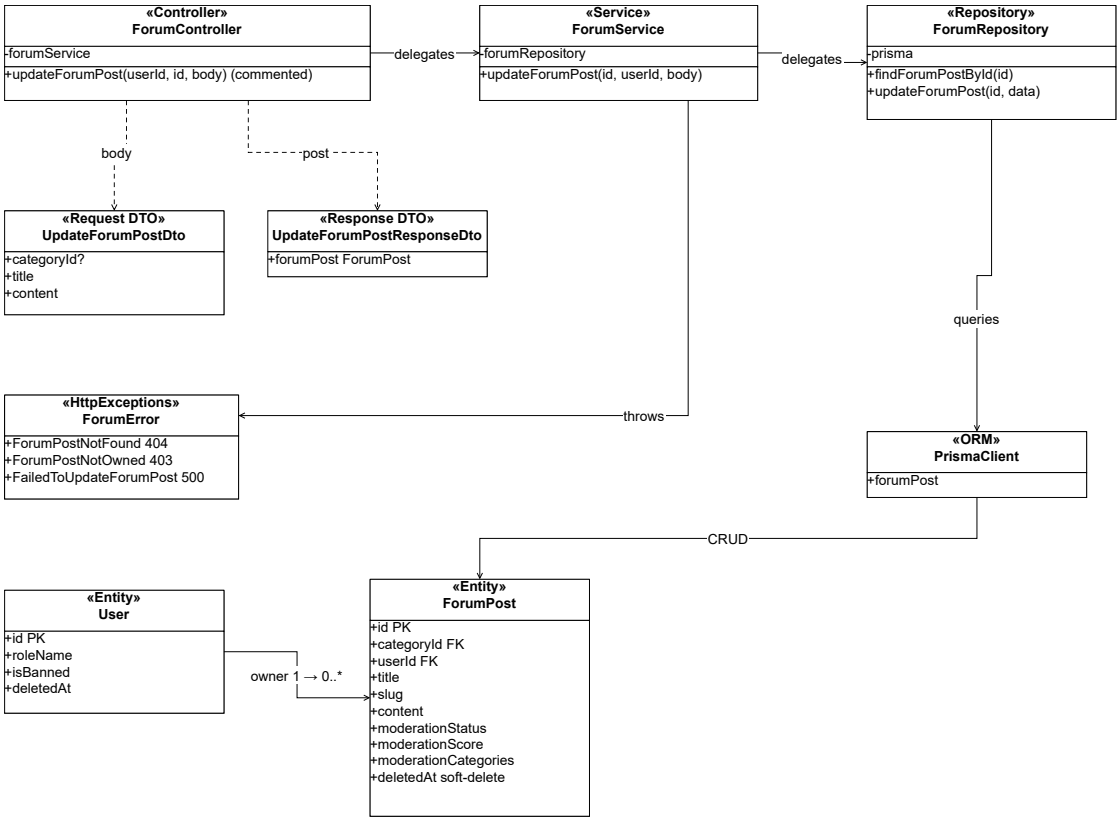


Figure 15: Class diagram of Update Post

b. Sequence Diagram(s)

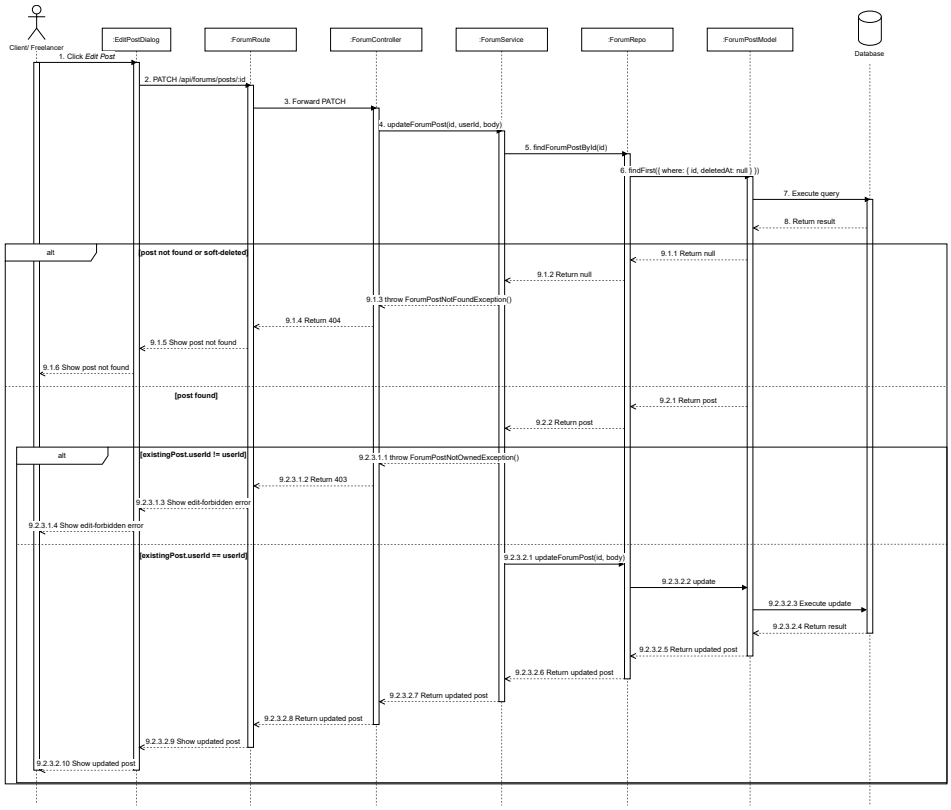


Figure 16: Sequence diagram of Update Post

3.7 Delete Post

a. Class Diagram

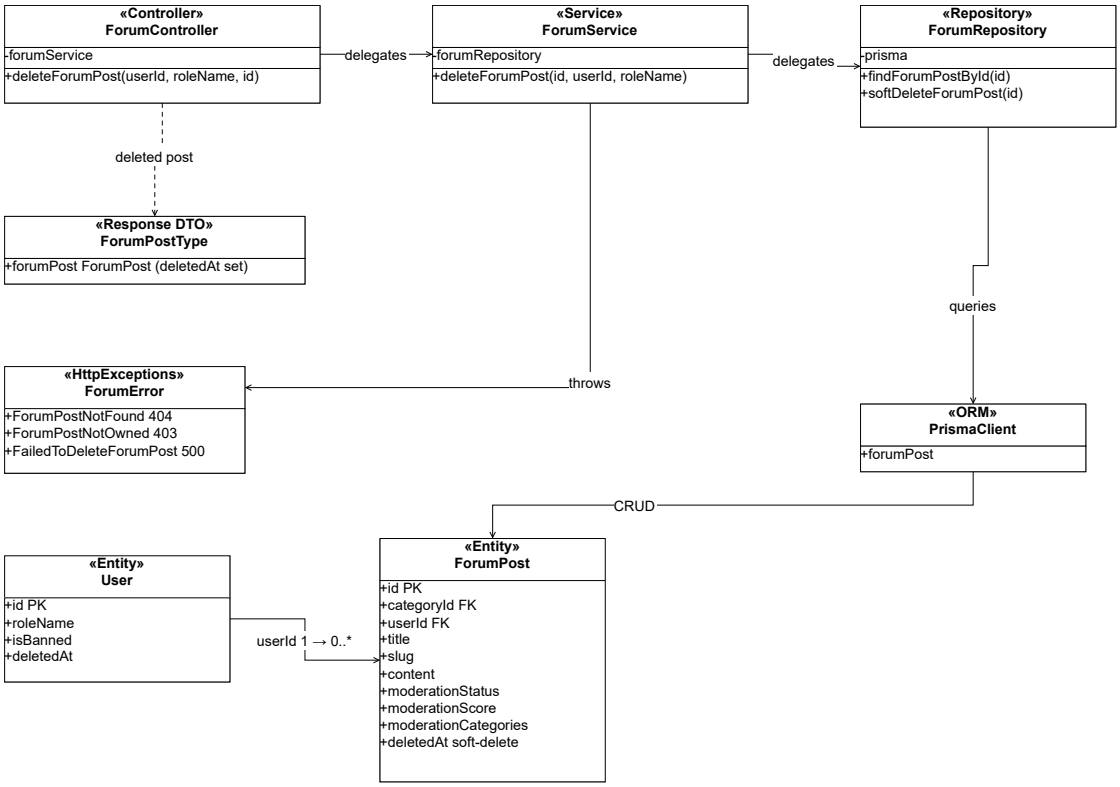


Figure 17: Class diagram of Delete Post

b. Sequence Diagram(s)

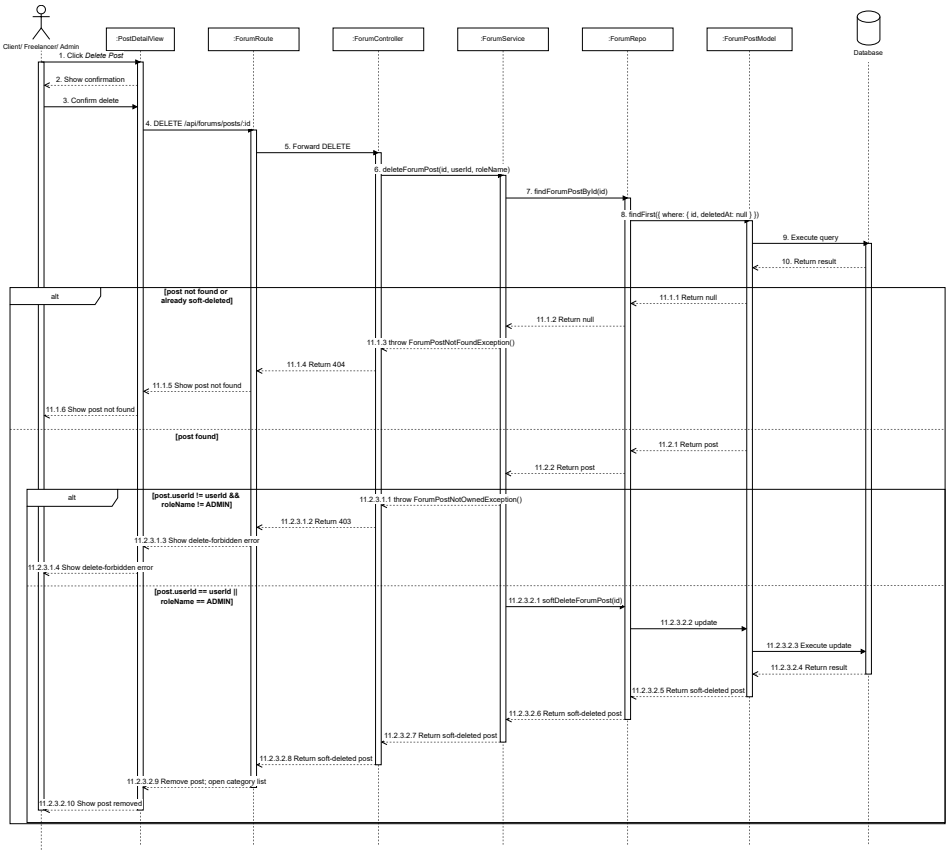


Figure 18: Sequence diagram of Delete Post

3.8 Like Post

a. Class Diagram

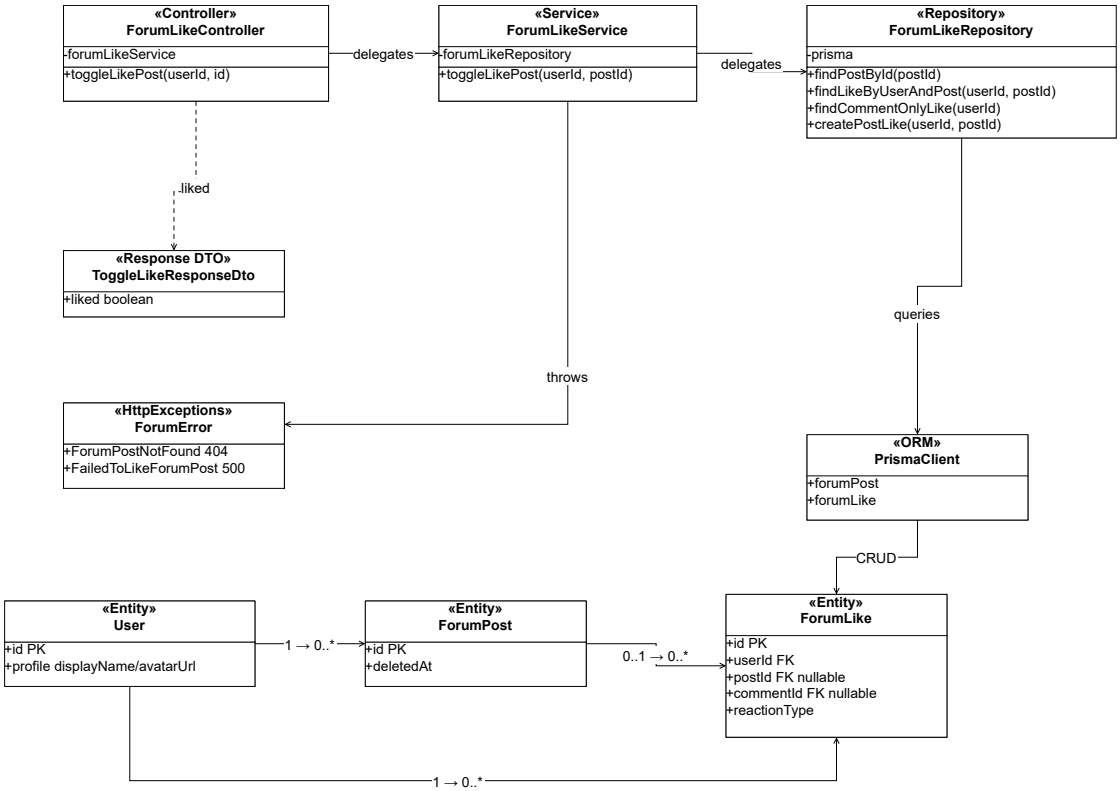


Figure 19: Class diagram of Like Post

b. Sequence Diagram(s)

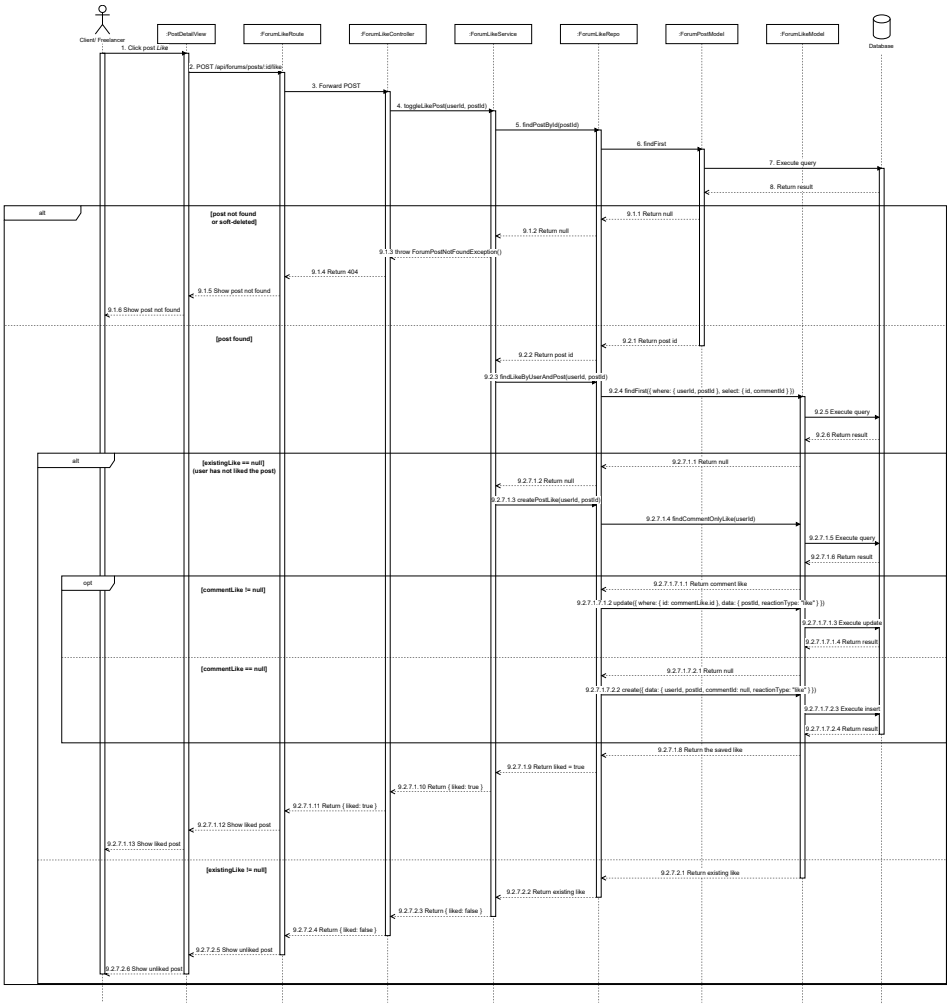


Figure 20: Sequence diagram of Like Post

3.9 Unlike Post

a. Class Diagram

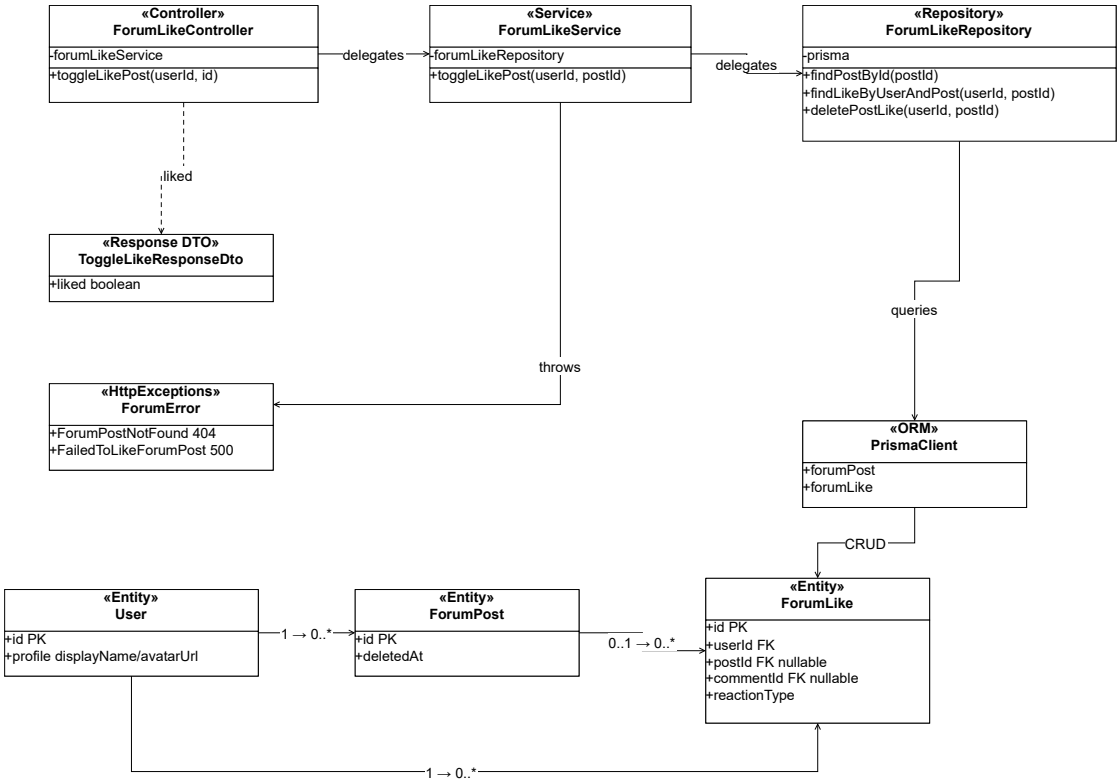


Figure 21: Class diagram of Unlike Post

b. Sequence Diagram(s)

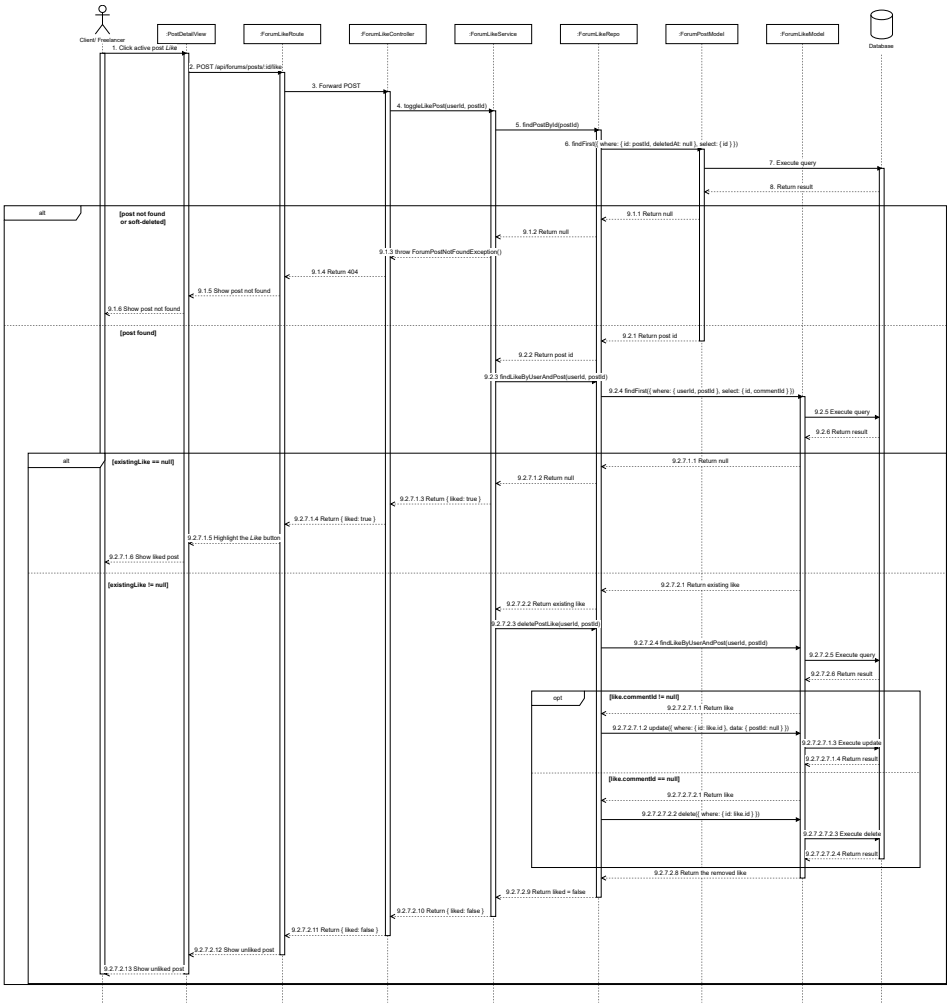


Figure 22: Sequence diagram of Unlike Post

3.10 View Post Likes

a. Class Diagram

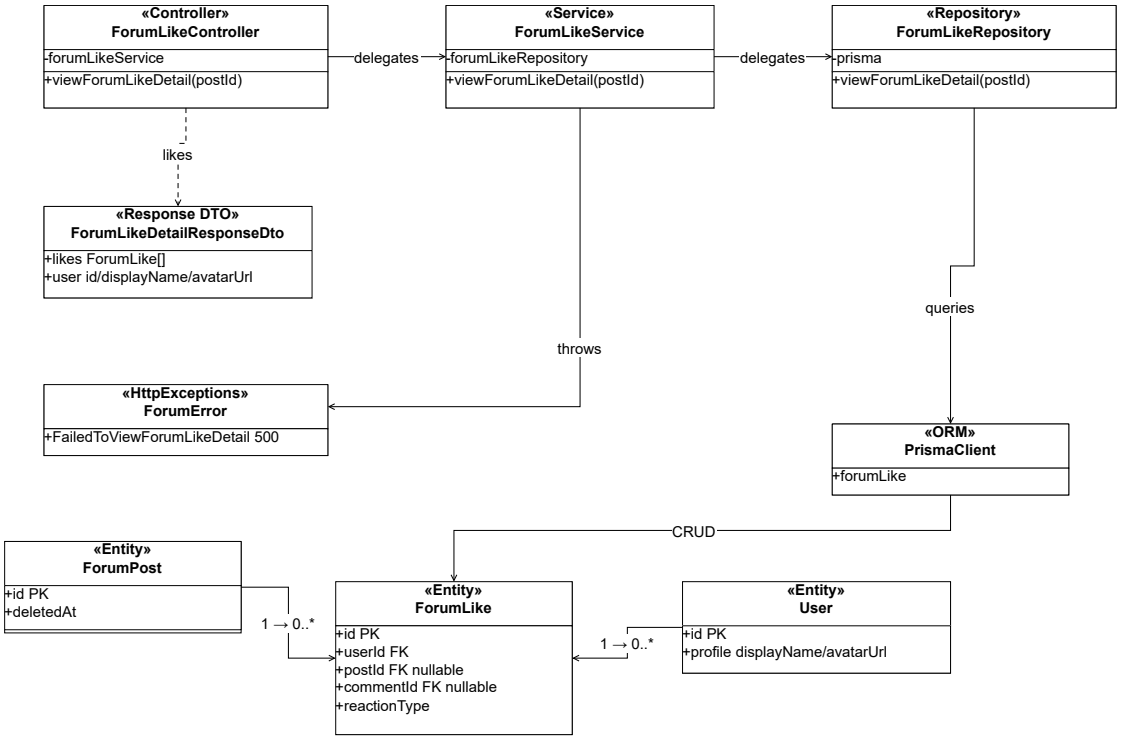


Figure 23: Class diagram of View Post Likes

b. Sequence Diagram(s)

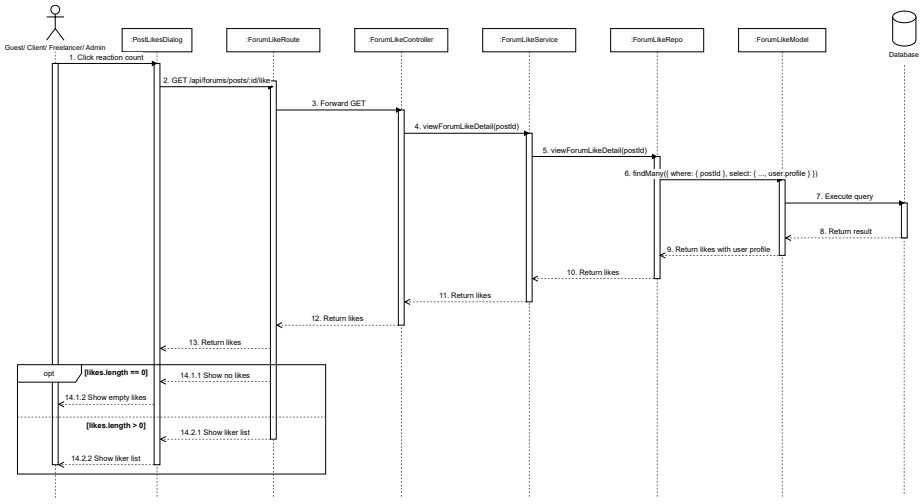


Figure 24: Sequence diagram of View Post Likes

3.11 View Post Comments

a. Class Diagram

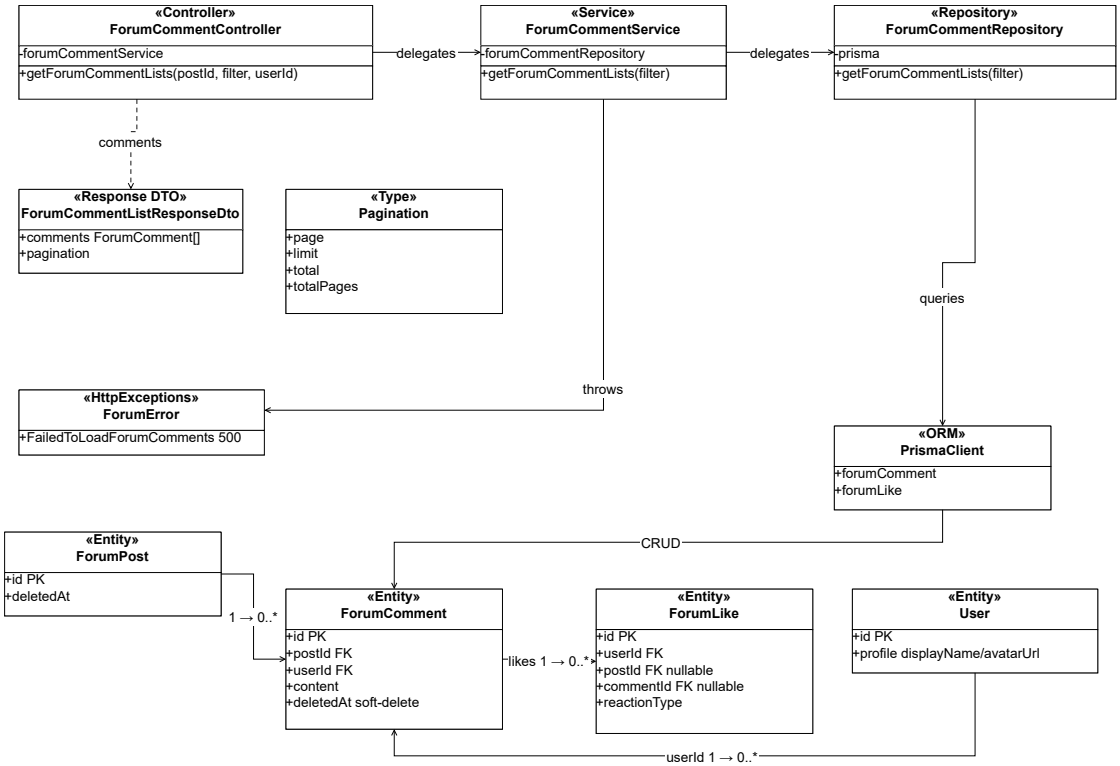


Figure 25: Class diagram of View Post Comments

b. Sequence Diagram(s)

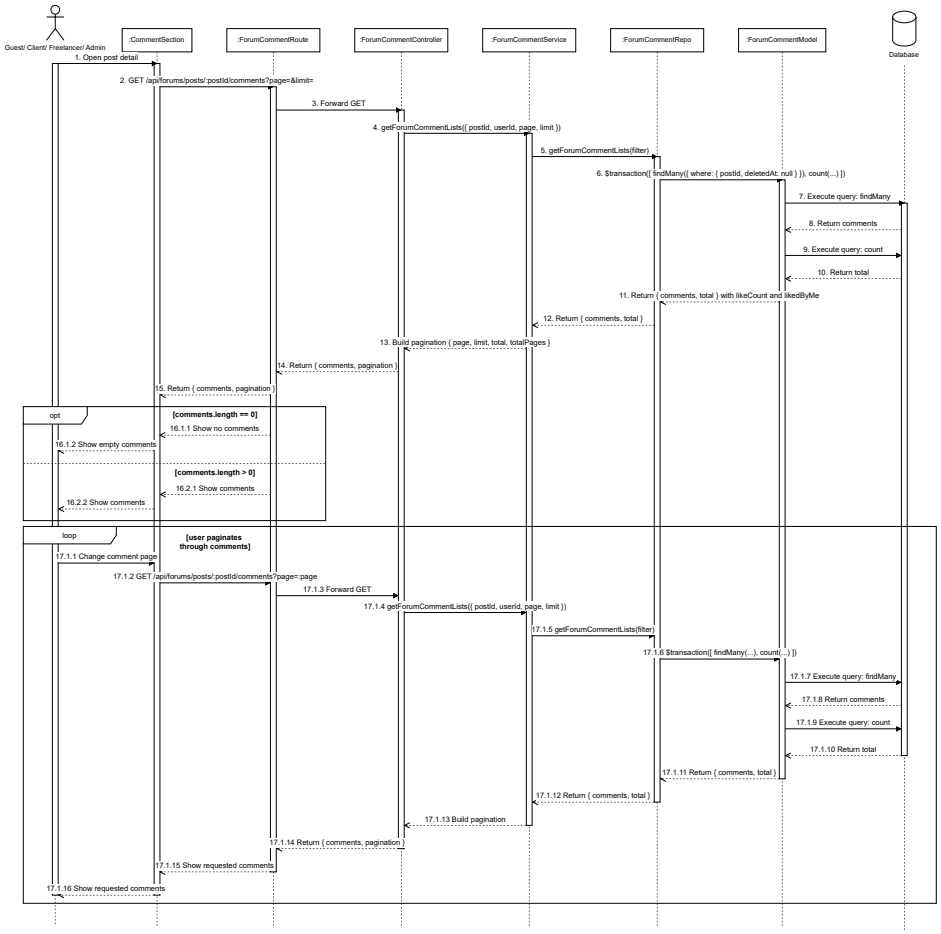


Figure 26: Sequence diagram of View Post Comments

3.12 Create Comment

a. Class Diagram

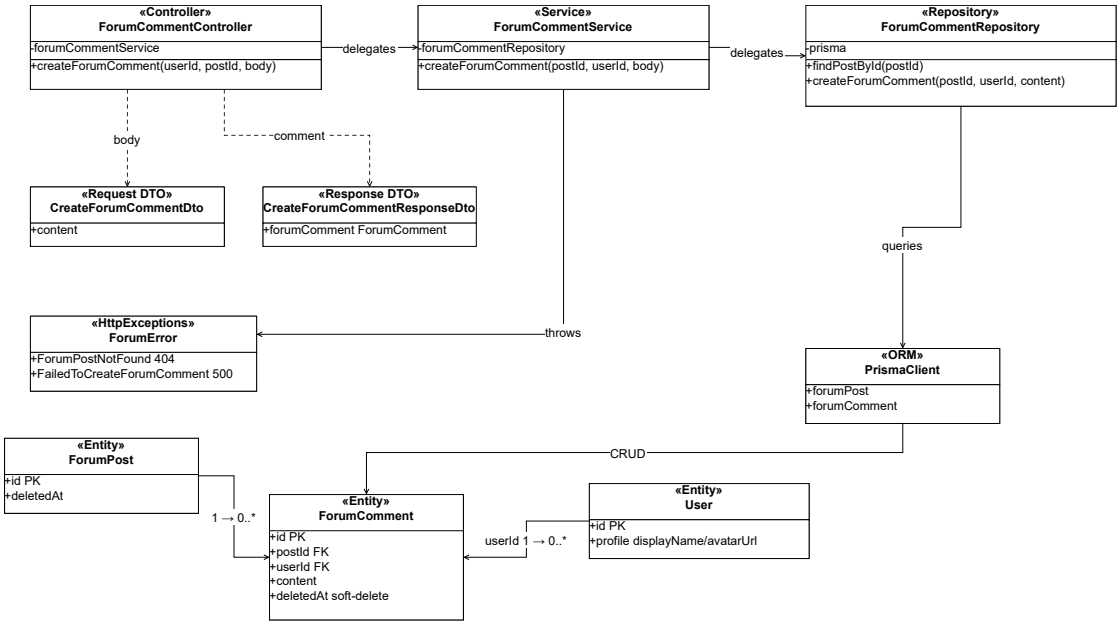


Figure 27: Class diagram of Create Comment

b. Sequence Diagram(s)

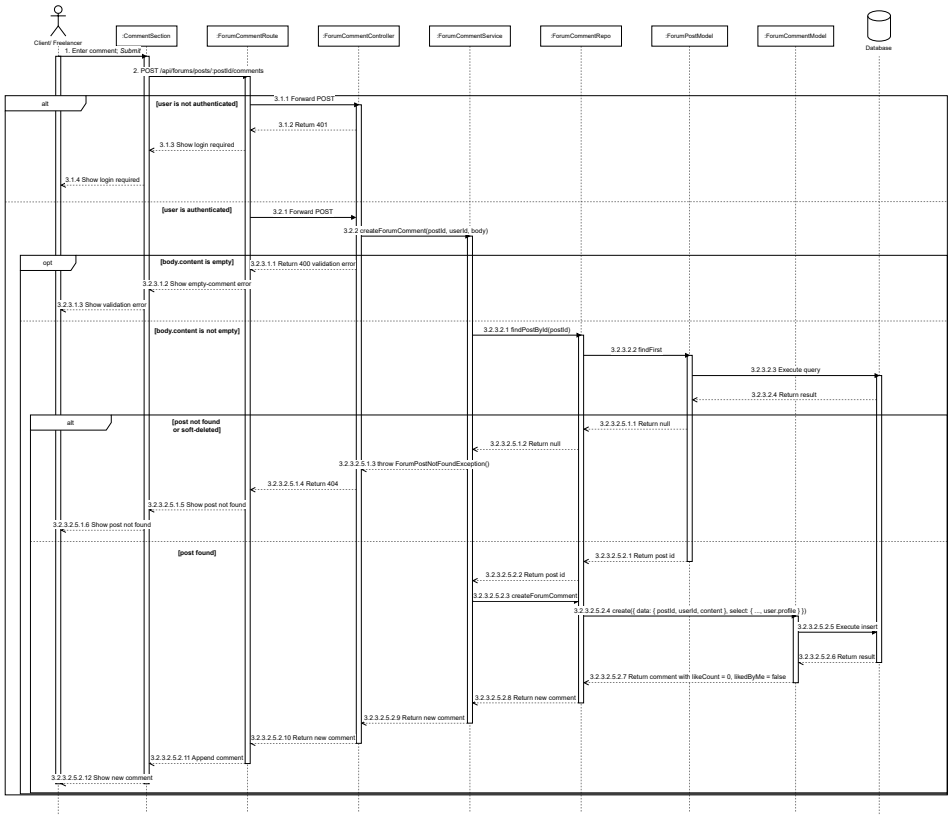


Figure 28: Sequence diagram of Create Comment

3.13 Update Comment

a. Class Diagram

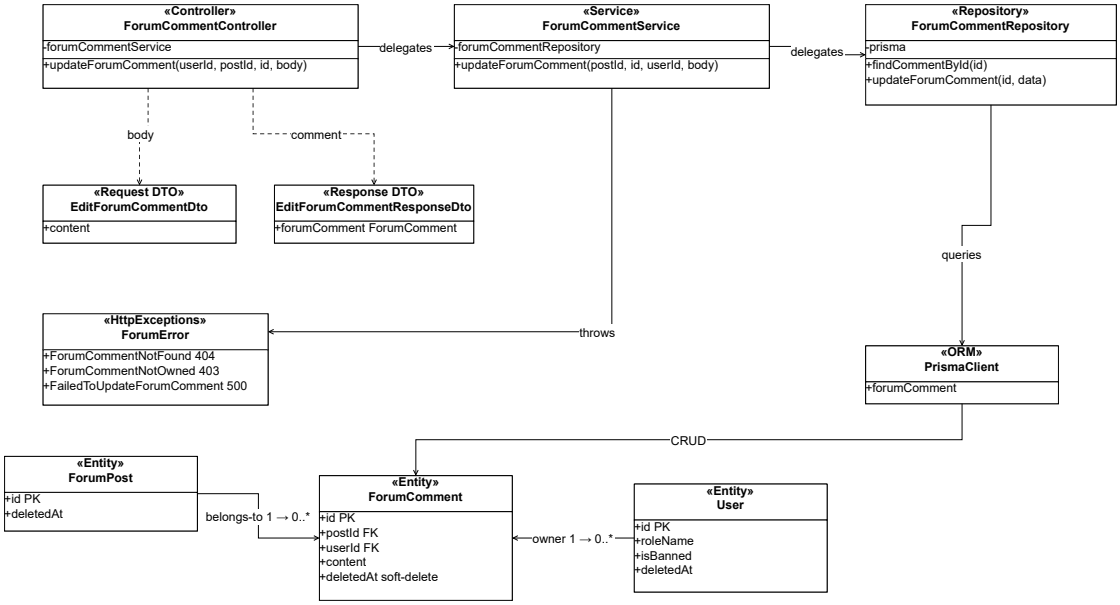


Figure 29: Class diagram of Update Comment

b. Sequence Diagram(s)

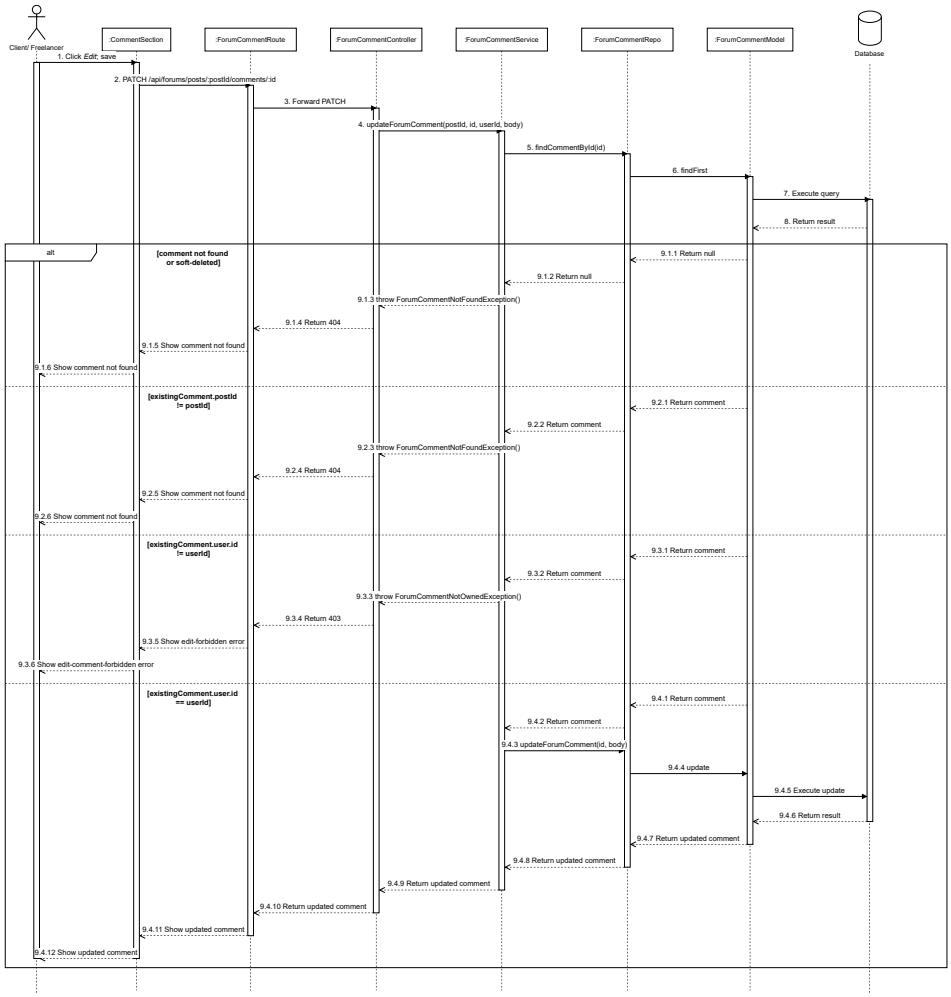


Figure 30: Sequence diagram of Update Comment

3.14 Delete Comment

a. Class Diagram

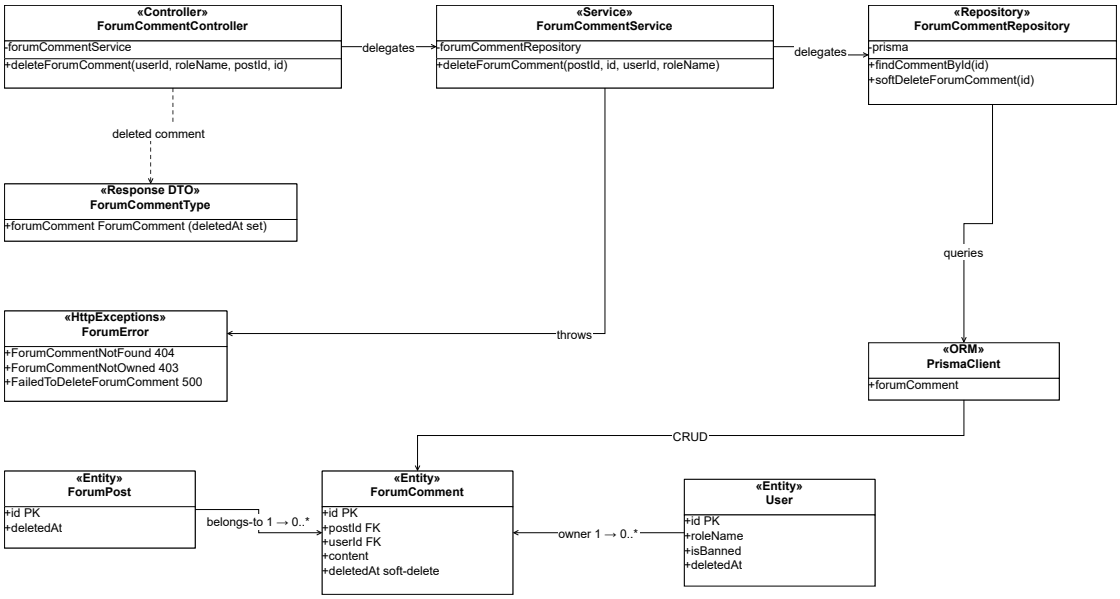


Figure 31: Class diagram of Delete Comment

b. Sequence Diagram(s)

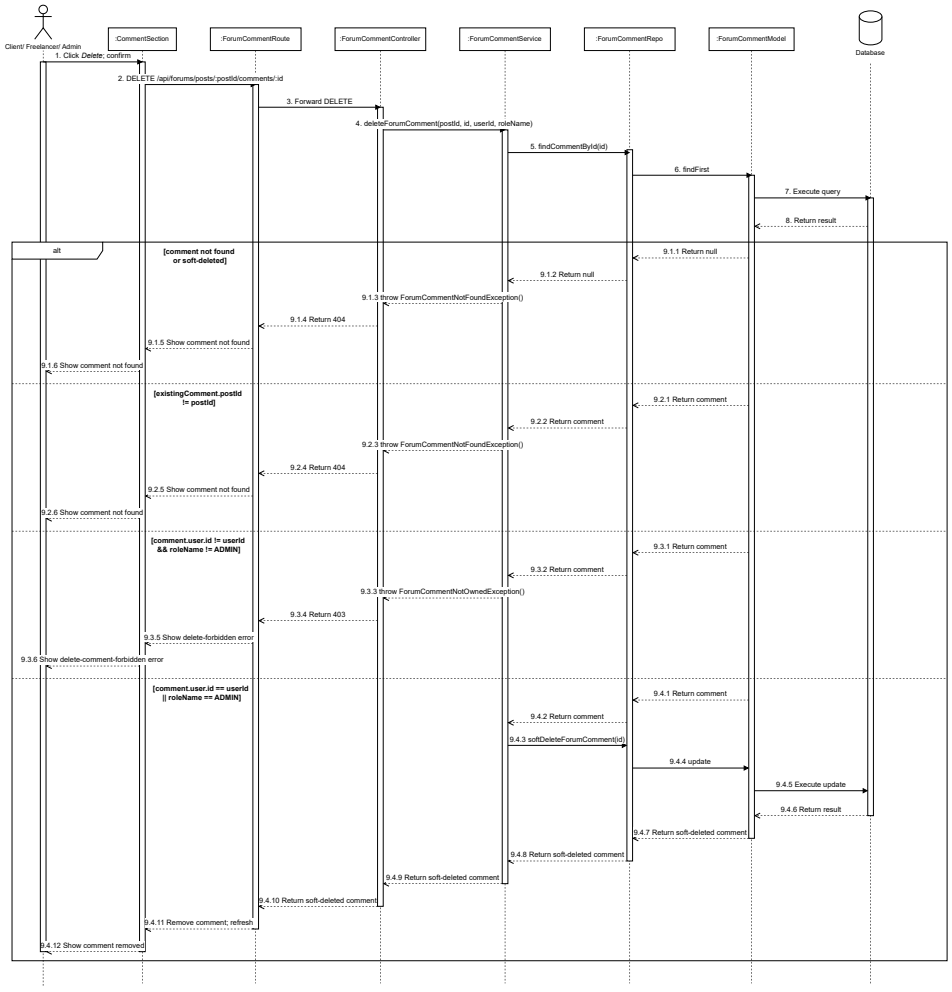


Figure 32: Sequence diagram of Delete Comment

3.15 Like Comment

a. Class Diagram

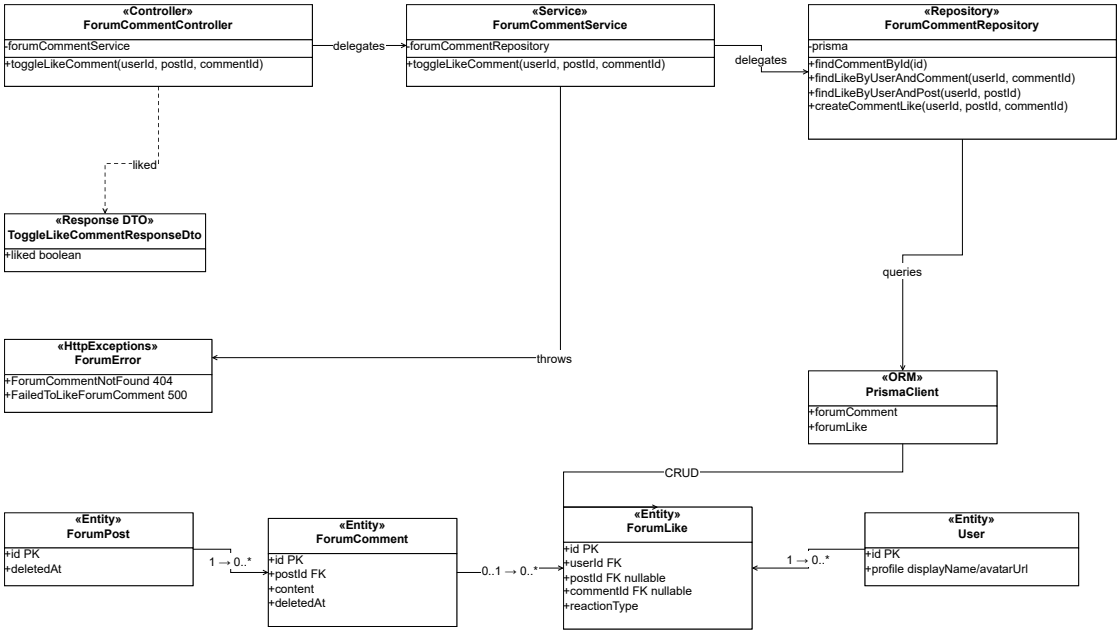


Figure 33: Class diagram of Like Comment

b. Sequence Diagram(s)

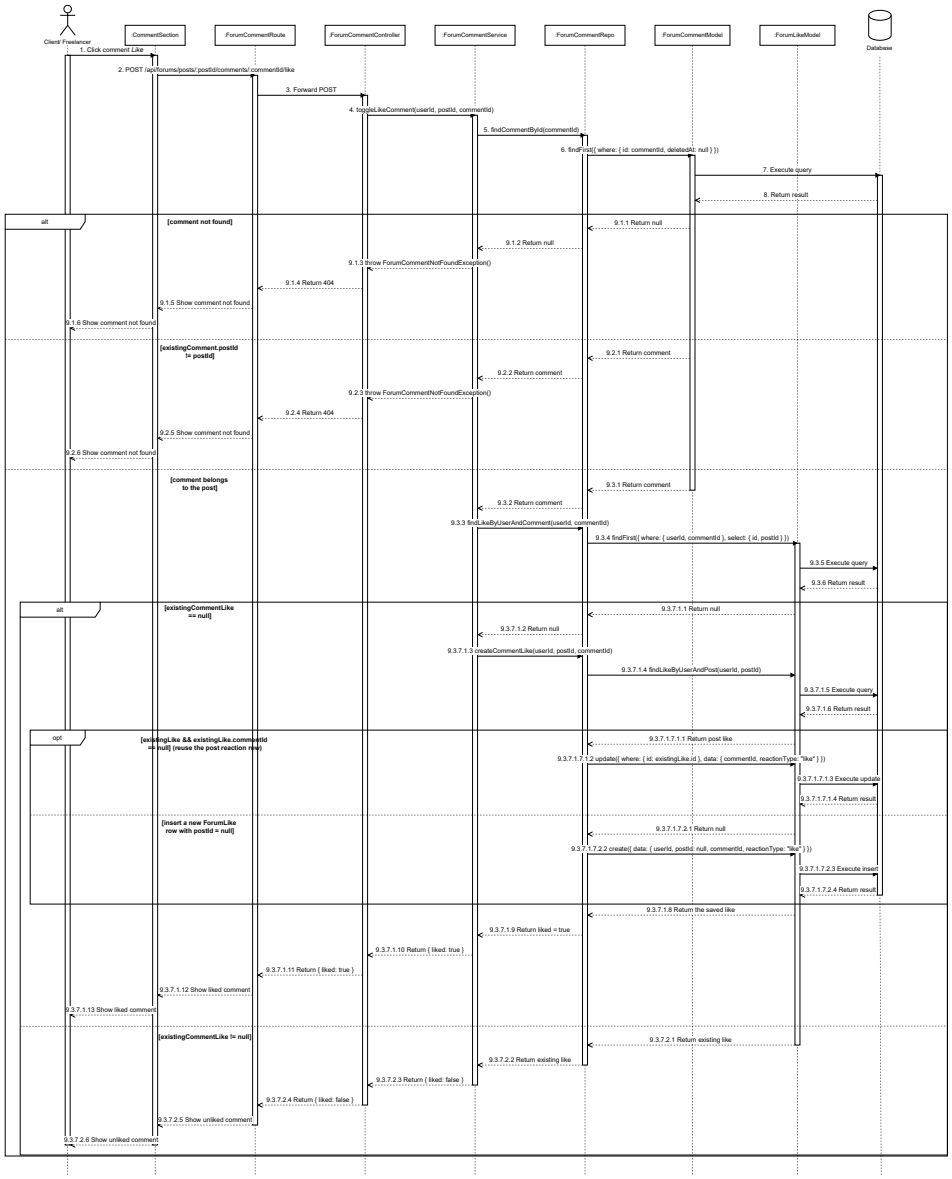


Figure 34: Sequence diagram of Like Comment

3.16 Unlike Comment

a. Class Diagram

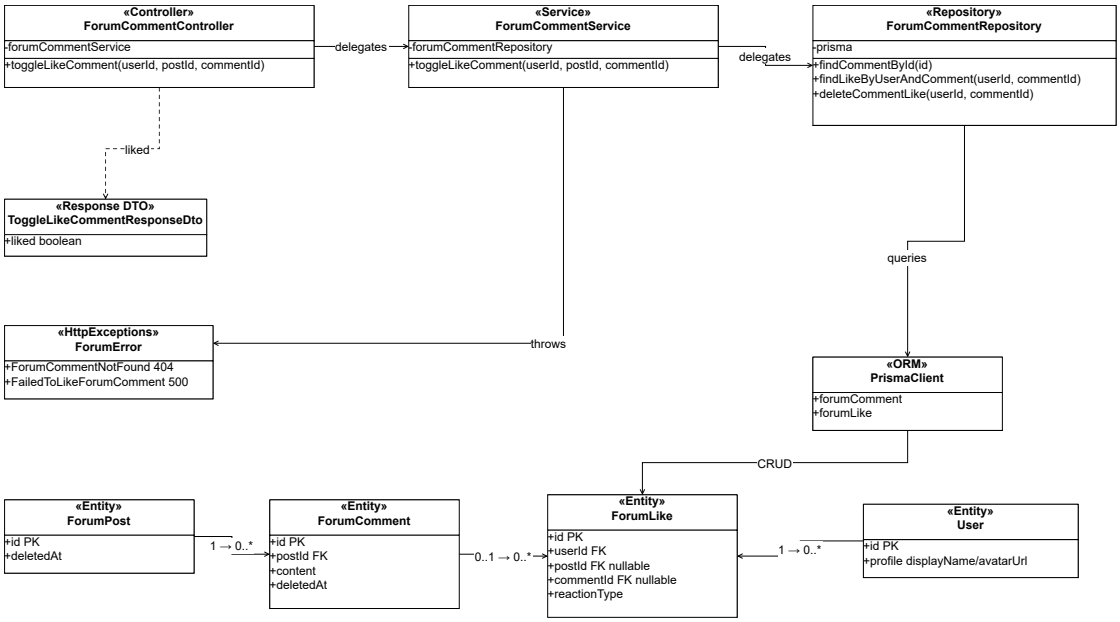


Figure 35: Class diagram of Unlike Comment

b. Sequence Diagram(s)

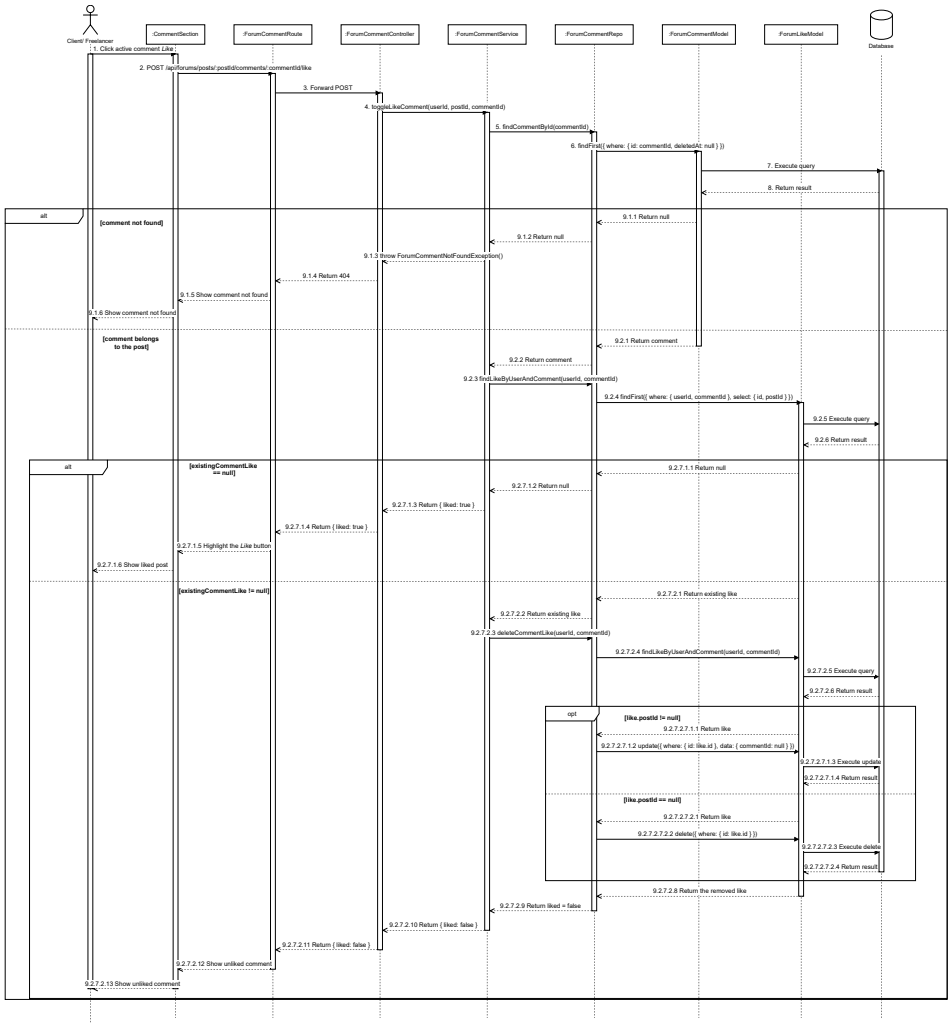


Figure 36: Sequence diagram of Unlike Comment

3.17 Report Content

a. Class Diagram

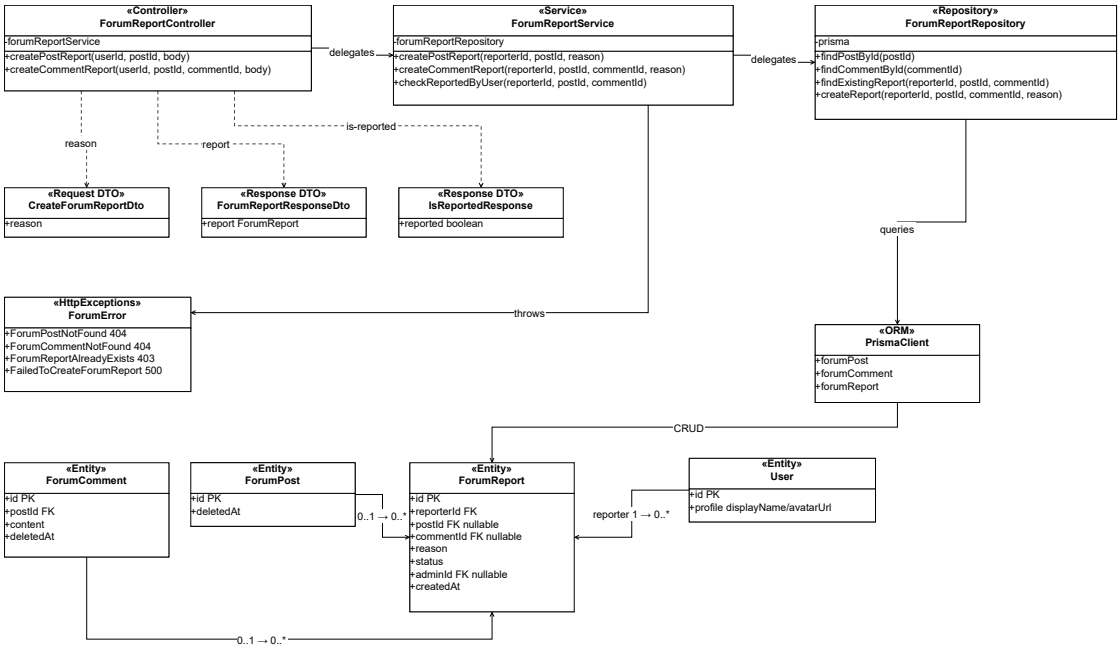


Figure 37: Class diagram of Report Content

b. Sequence Diagram(s)

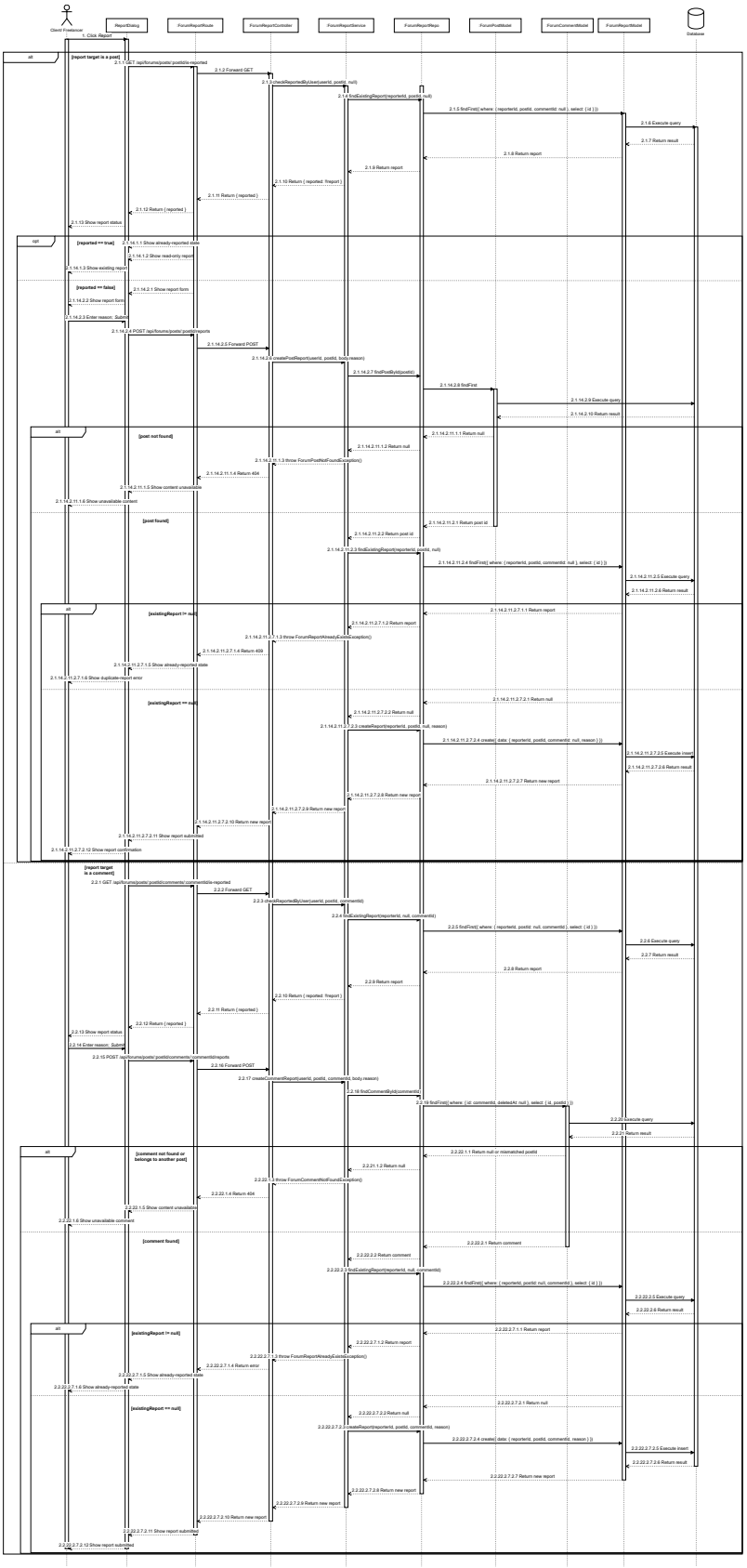


Figure 38: Sequence diagram of Report Content

3.18 View Conversation List

a. Class Diagram

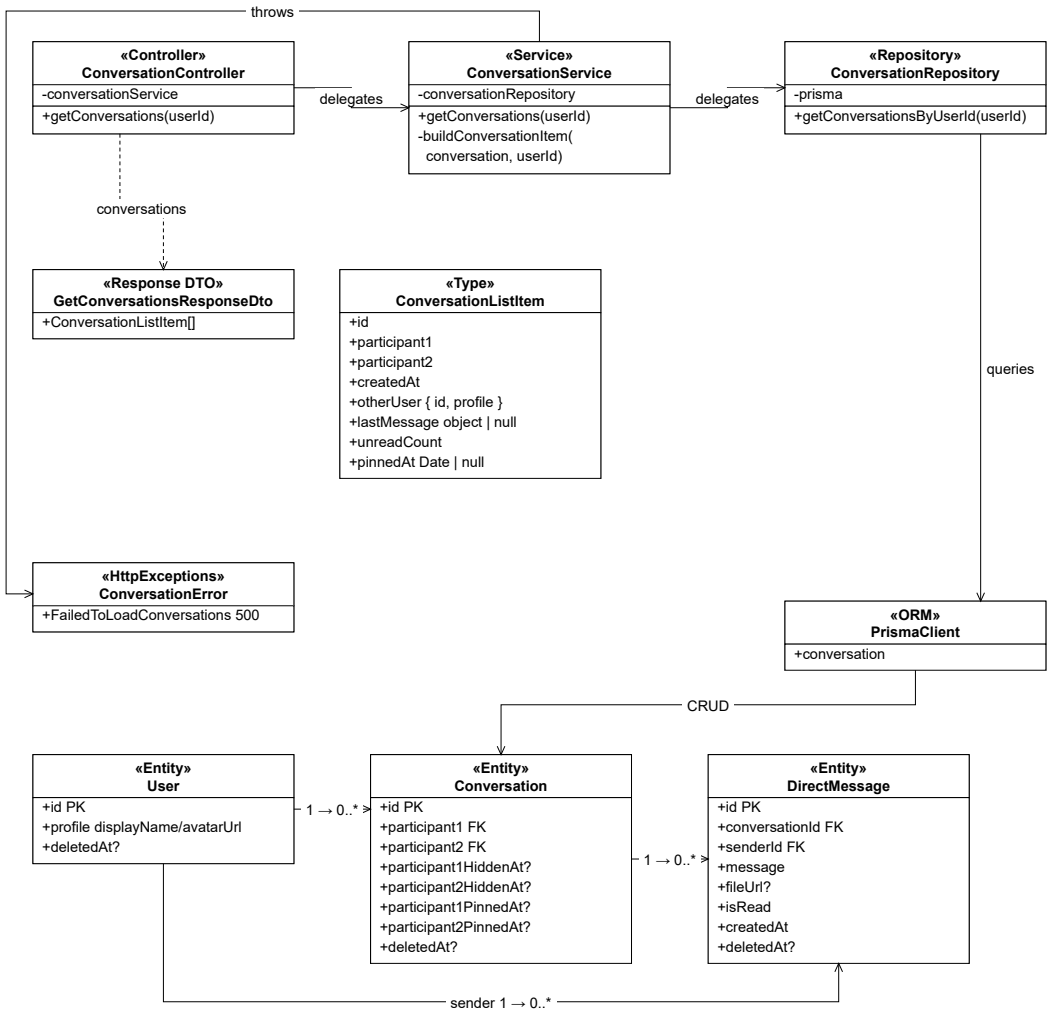


Figure 39: Class diagram of View Conversation List

b. Sequence Diagram(s)

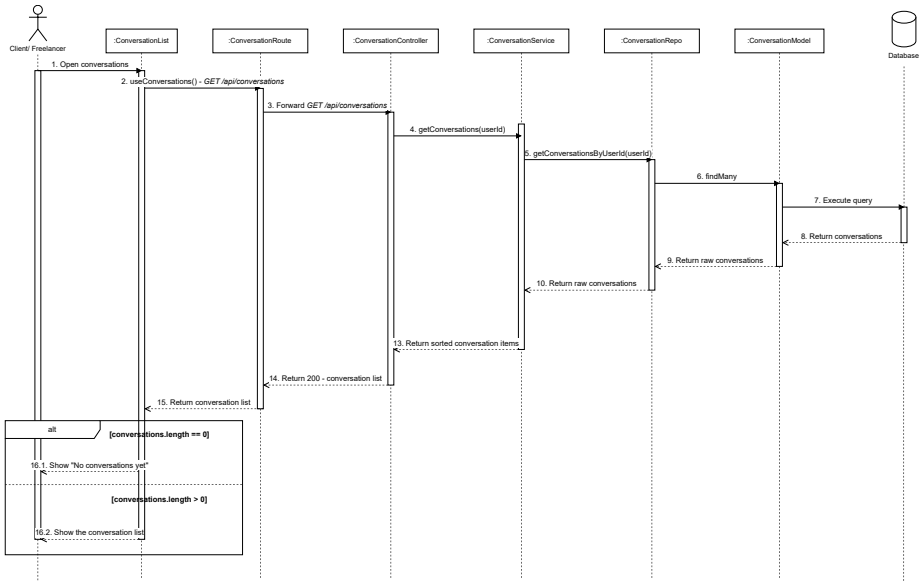


Figure 40: Sequence diagram of View Conversation List

3.19 Start Conversation

a. Class Diagram

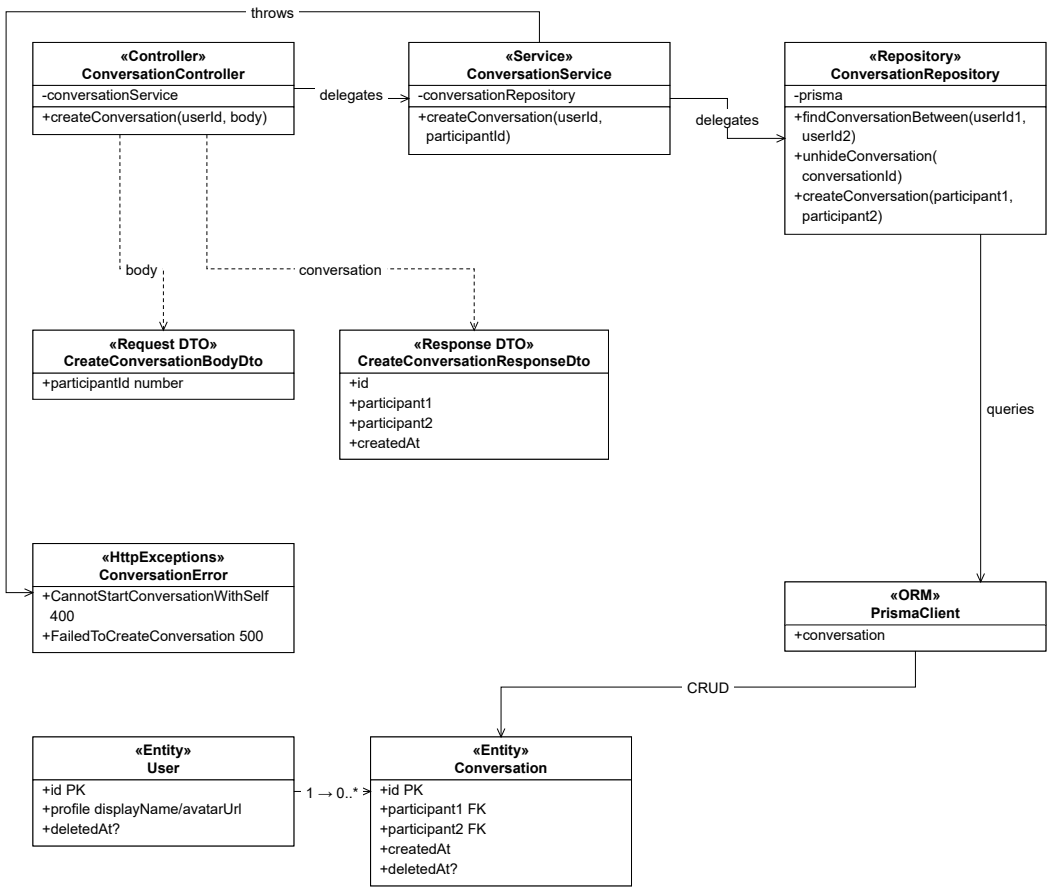


Figure 41: Class diagram of Start Conversation

b. Sequence Diagram(s)

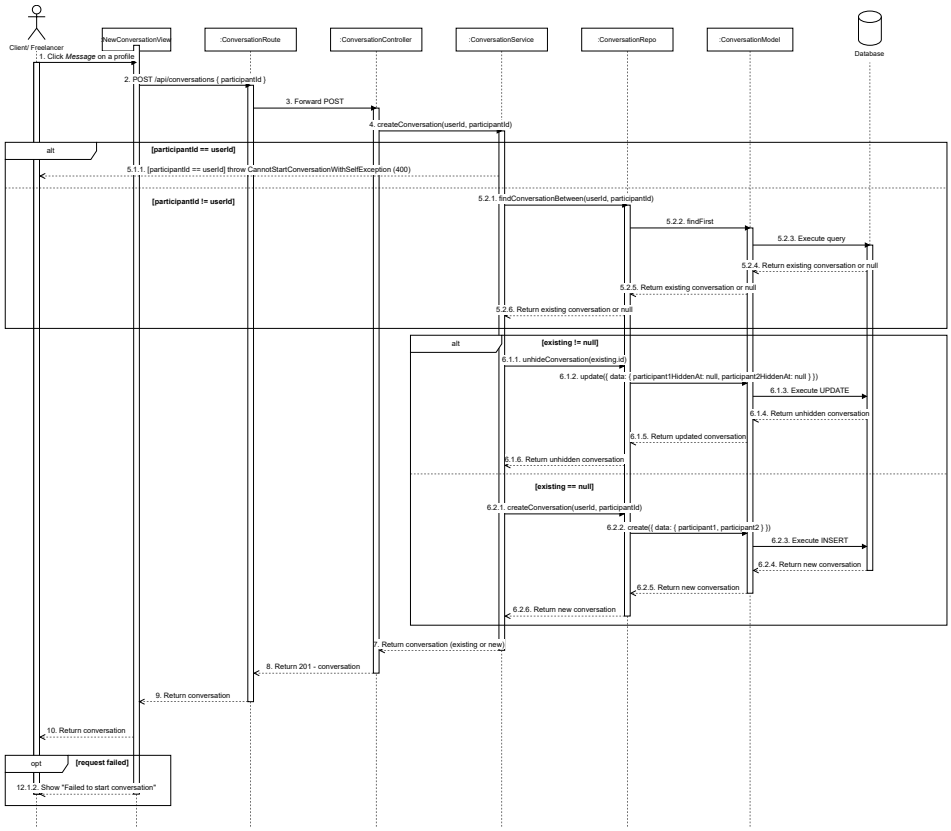


Figure 42: Sequence diagram of Start Conversation

3.20 View Conversation Detail

a. Class Diagram

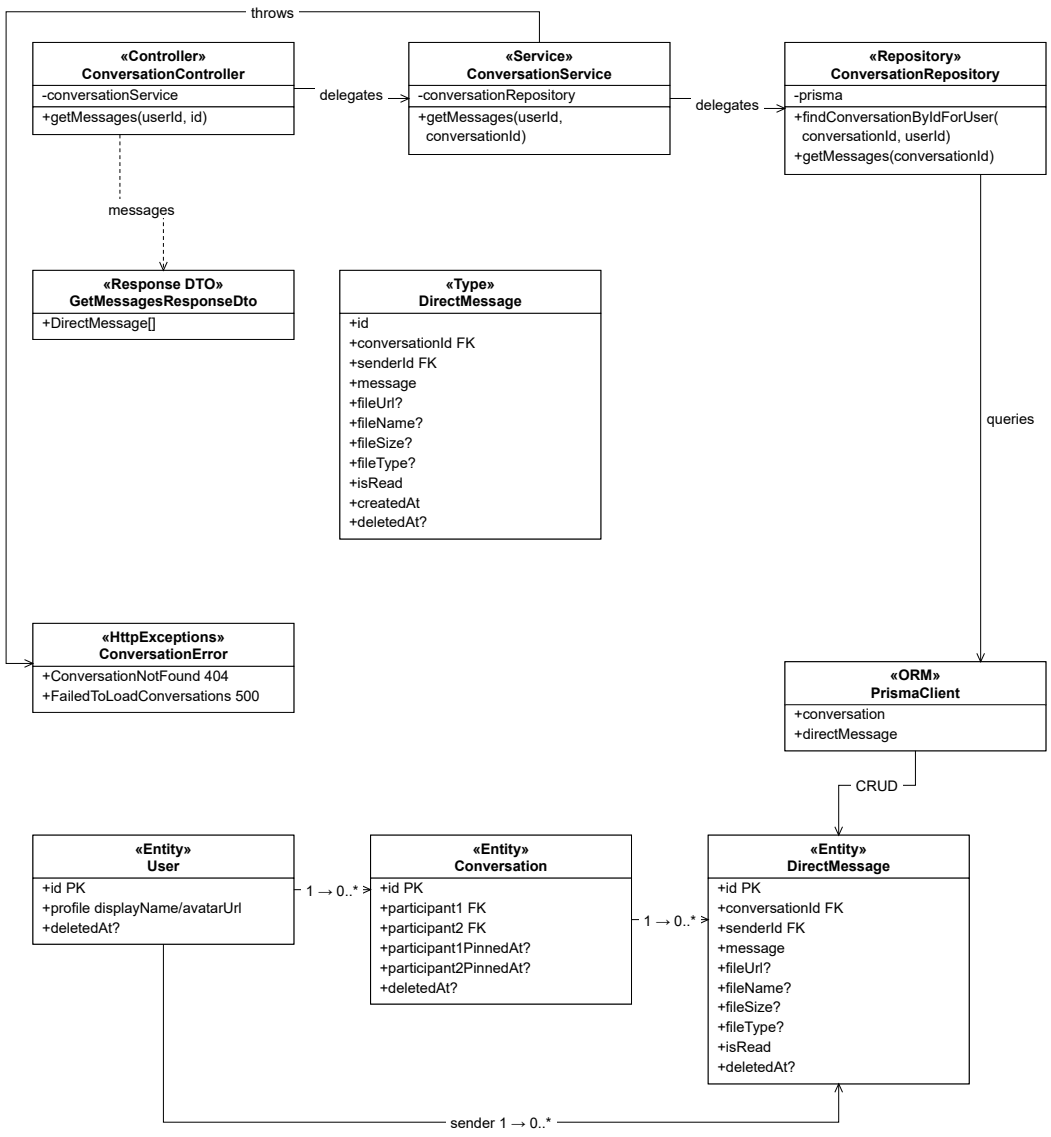


Figure 43: Class diagram of View Conversation Detail

b. Sequence Diagram(s)

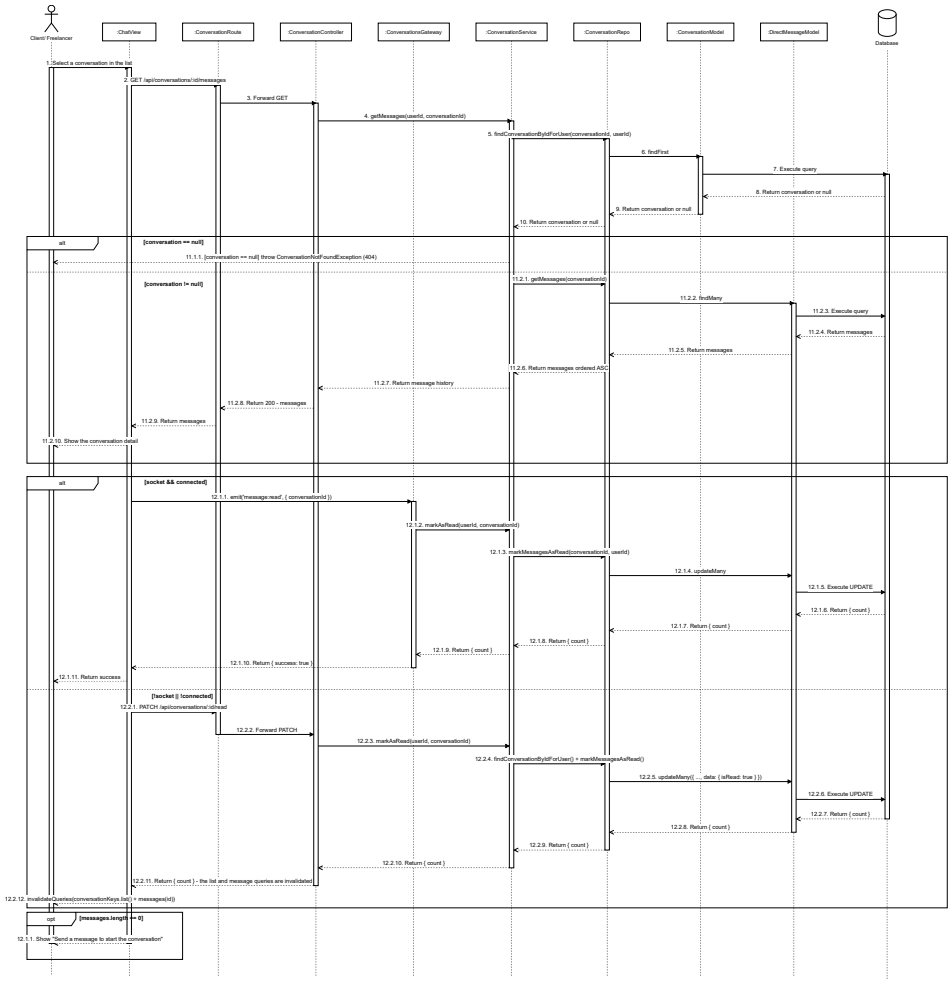


Figure 44: Sequence diagram of View Conversation Detail

3.21 Send Message

a. Class Diagram

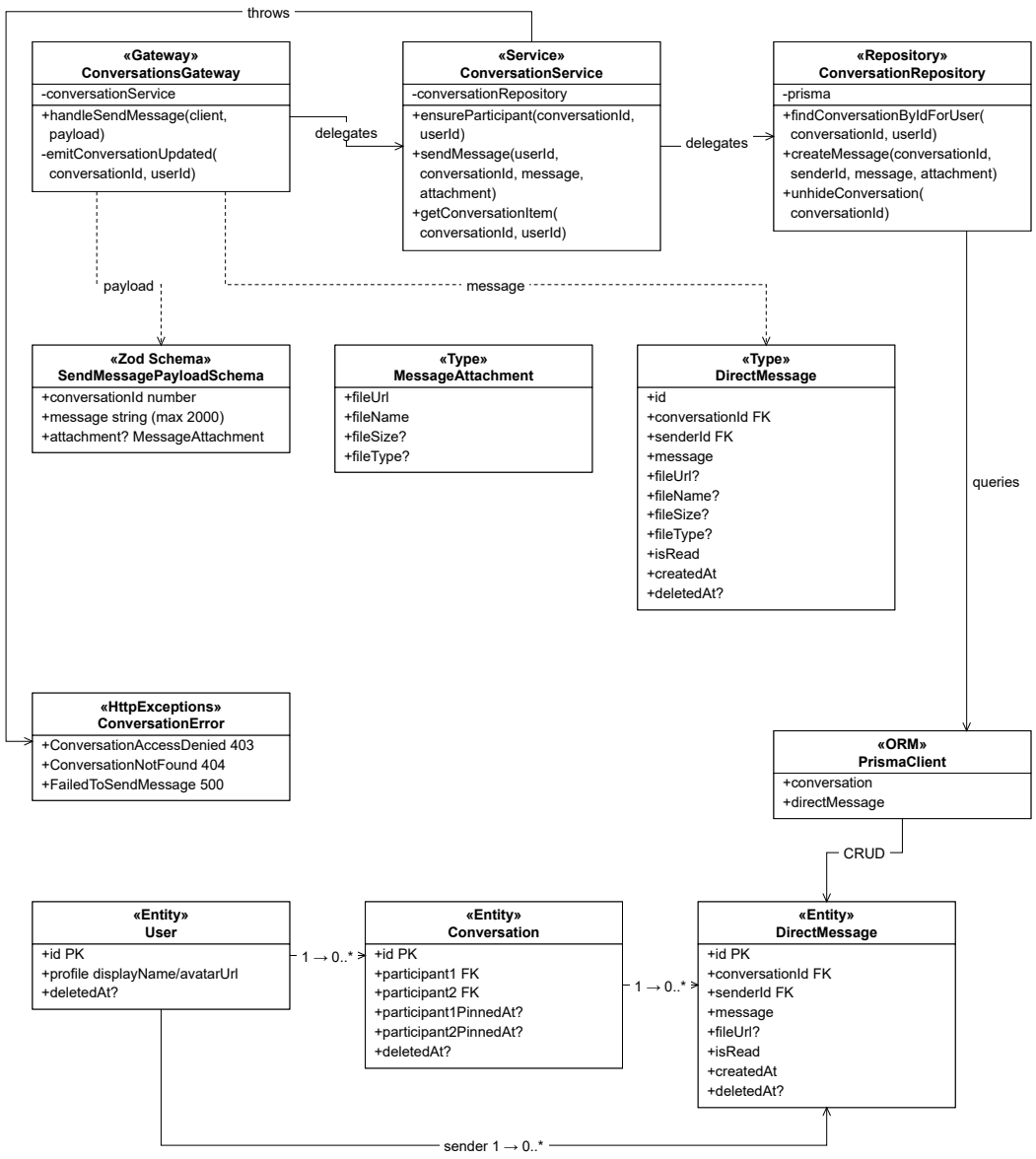


Figure 45: Class diagram of Send Message

b. Sequence Diagram(s)

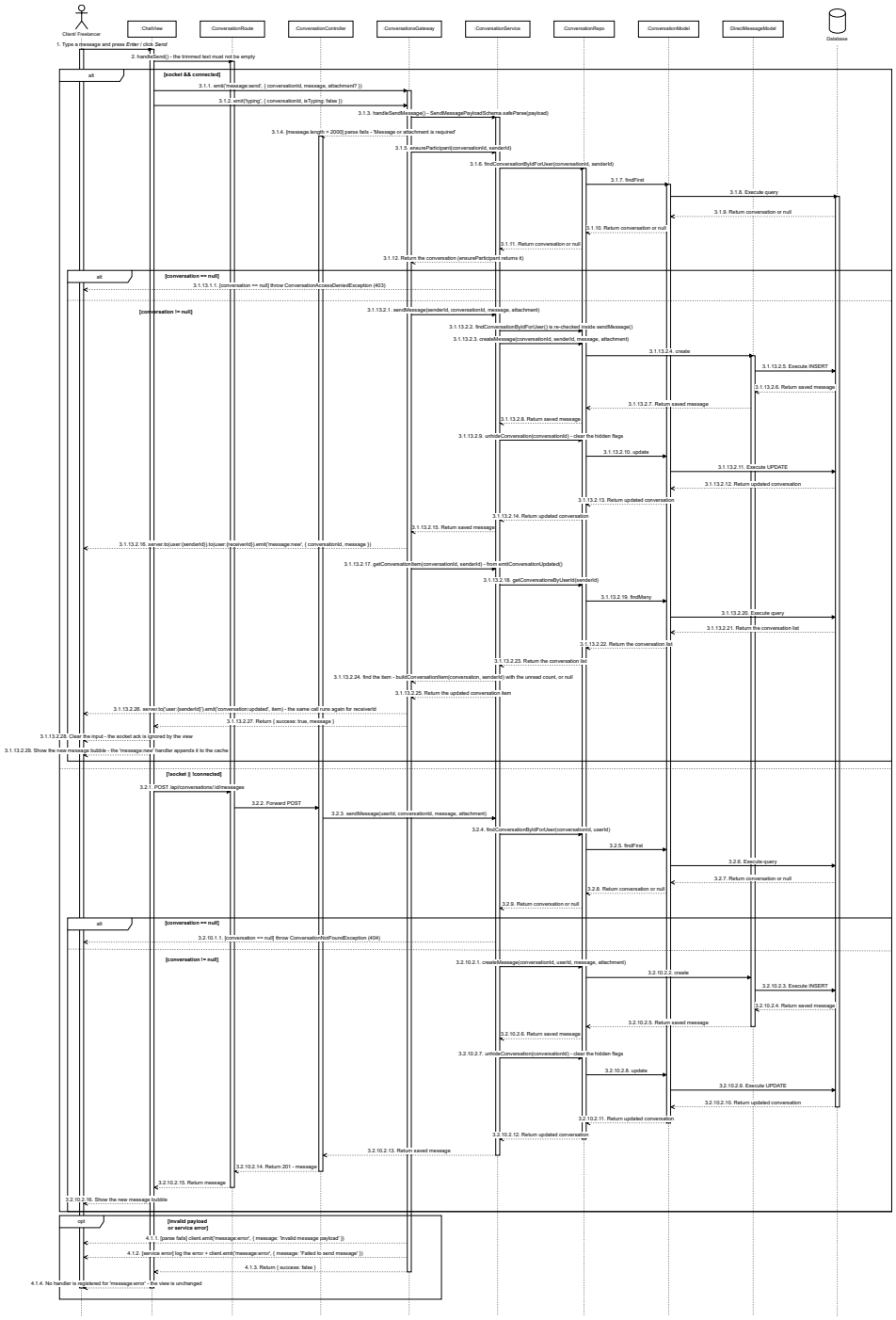


Figure 46: Sequence diagram of Send Message

3.22 Mark Message as Read

a. Class Diagram

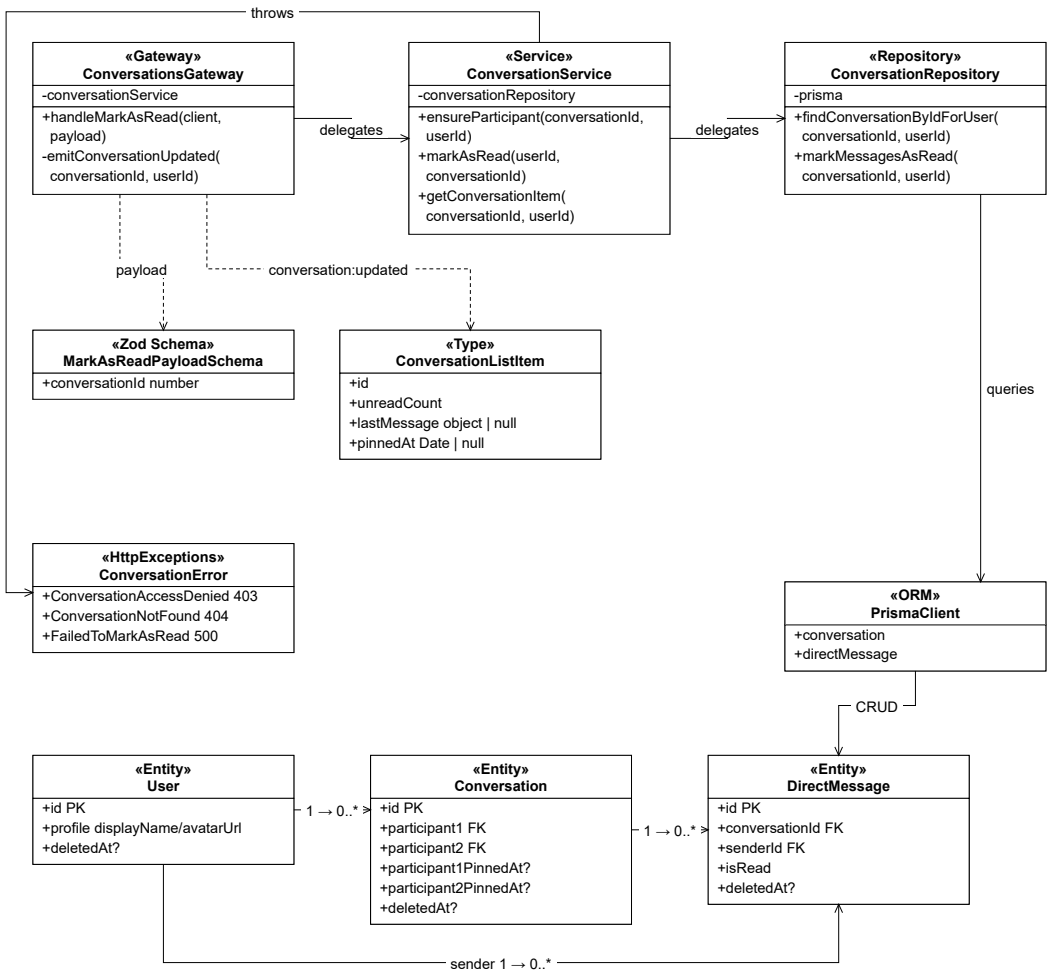


Figure 47: Class diagram of Mark Message as Read

b. Sequence Diagram(s)

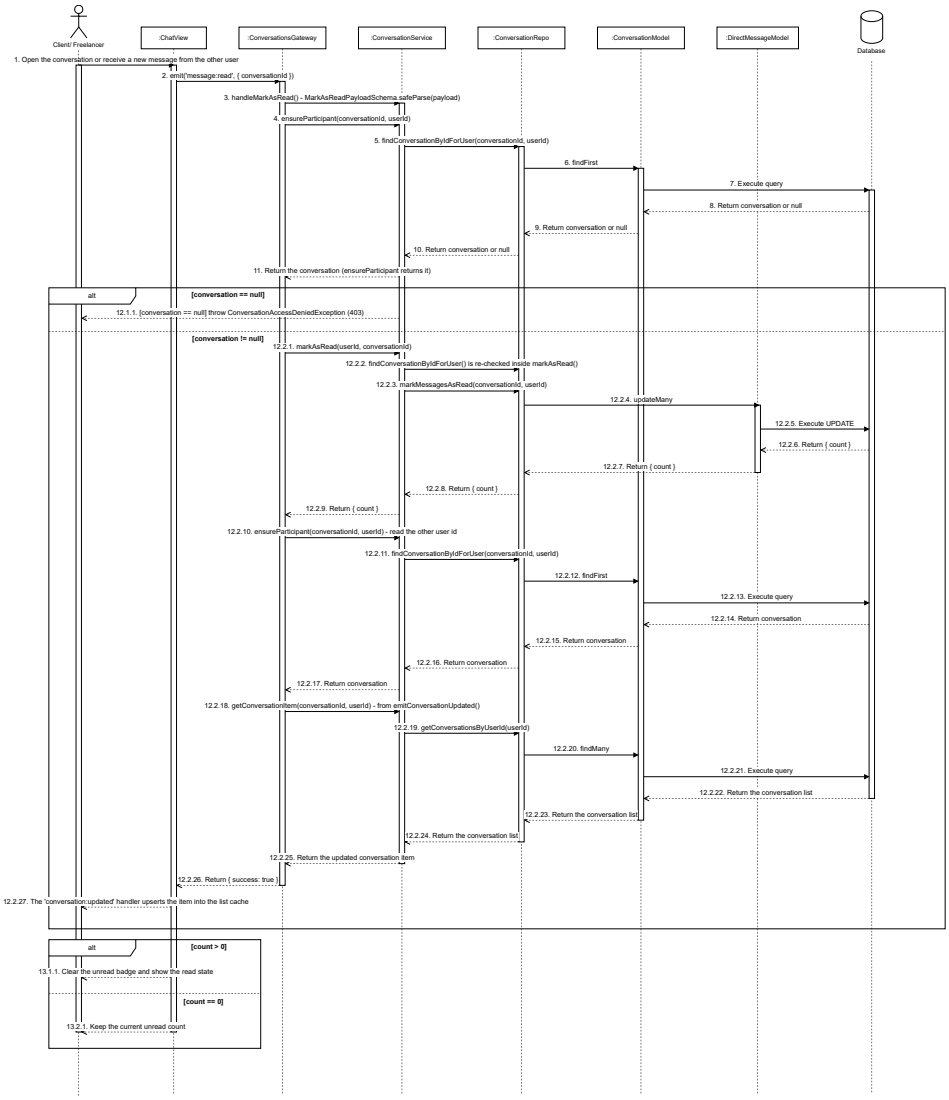


Figure 48: Sequence diagram of Mark Message as Read

3.23 Delete/Recall Message

a. Class Diagram

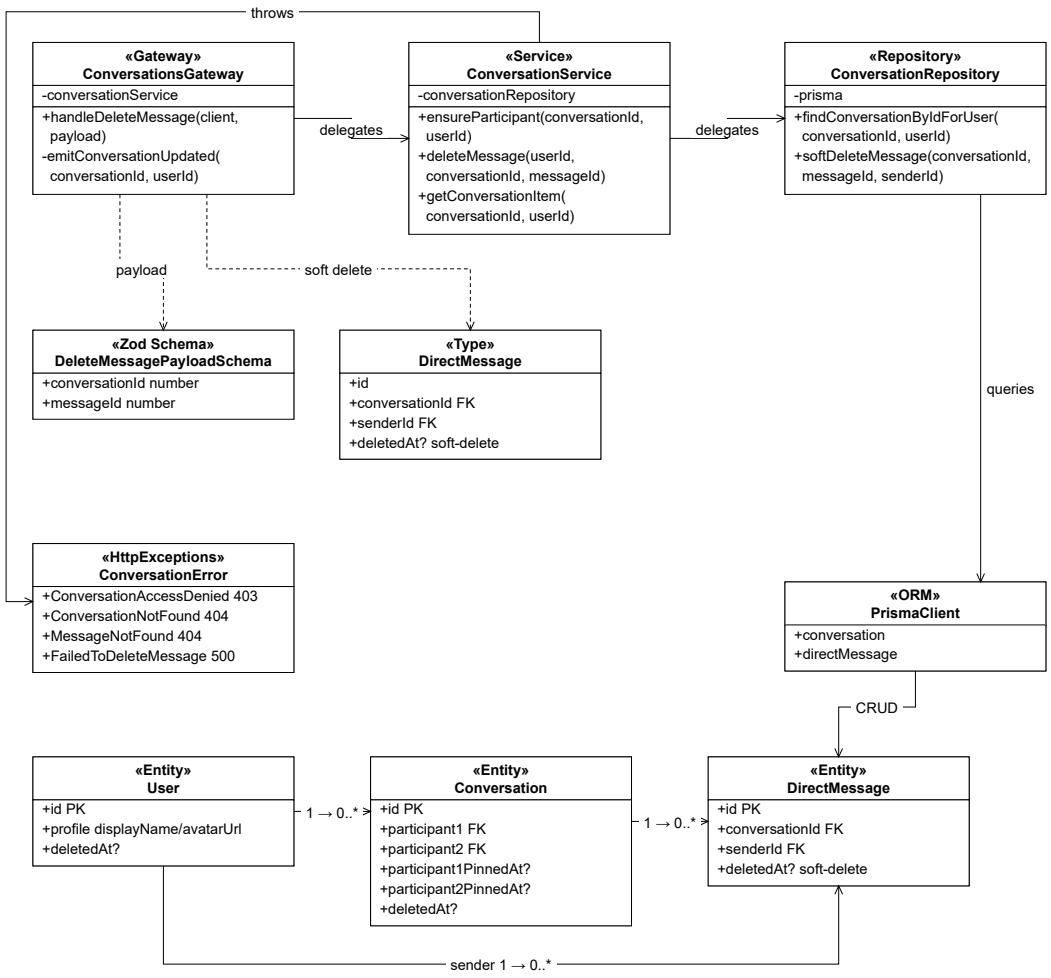


Figure 49: Class diagram of Delete/Recall Message

b. Sequence Diagram(s)

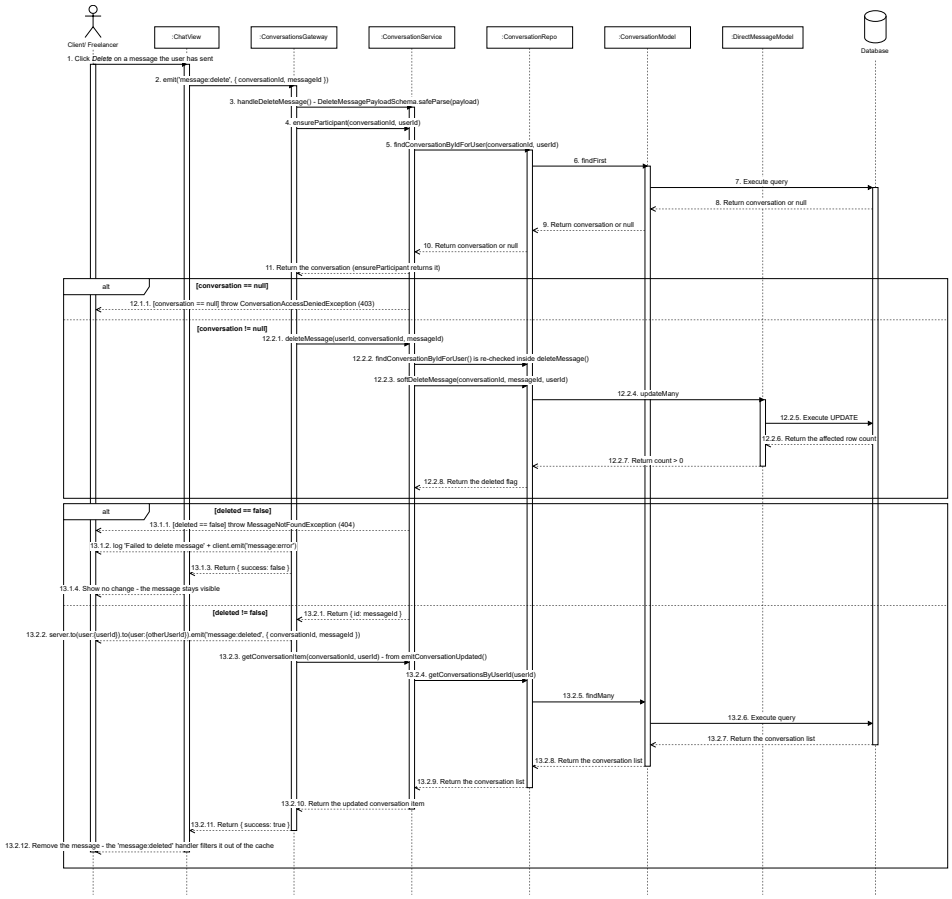


Figure 50: Sequence diagram of Delete/Recall Message

3.24 Pin/Unpin Conversation

a. Class Diagram

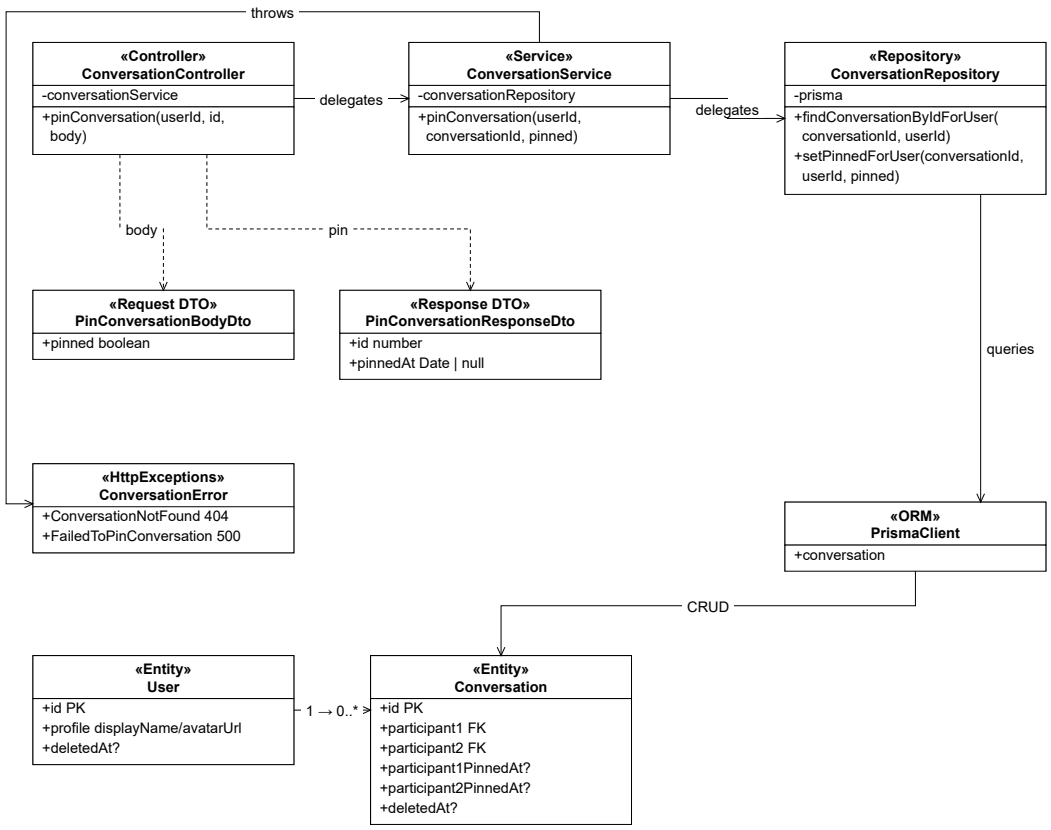


Figure 51: Class diagram of Pin/Unpin Conversation

b. Sequence Diagram(s)

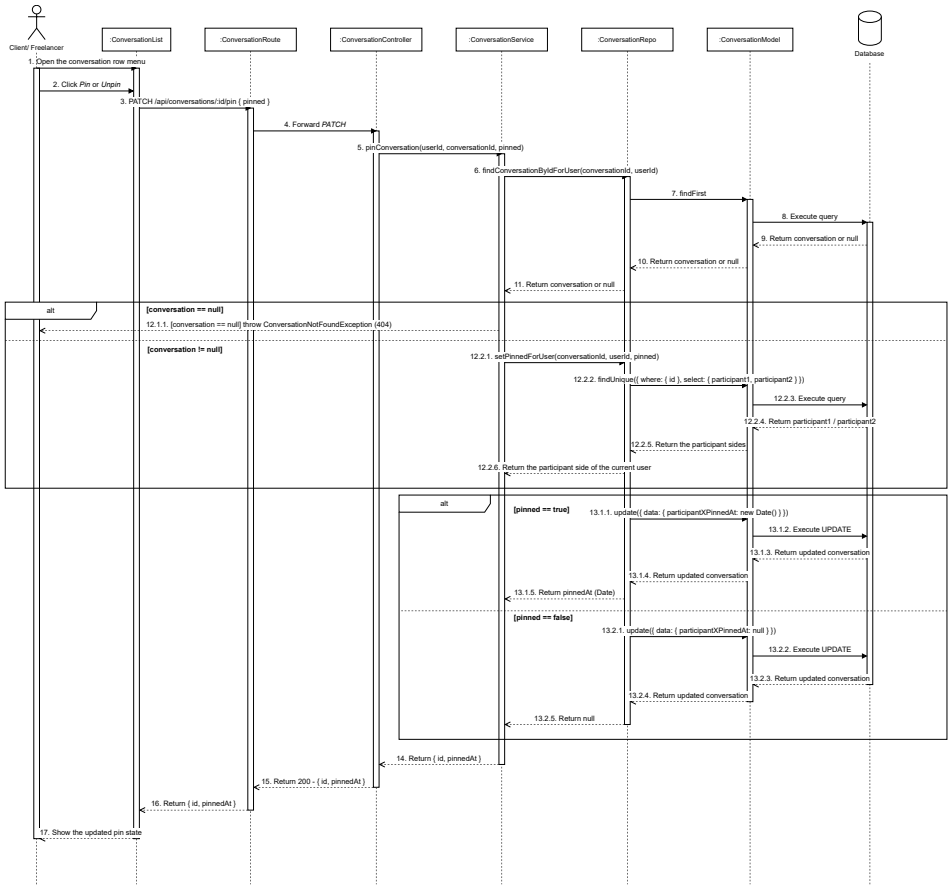


Figure 52: Sequence diagram of Pin/Unpin Conversation

3.25 Send File

a. Class Diagram

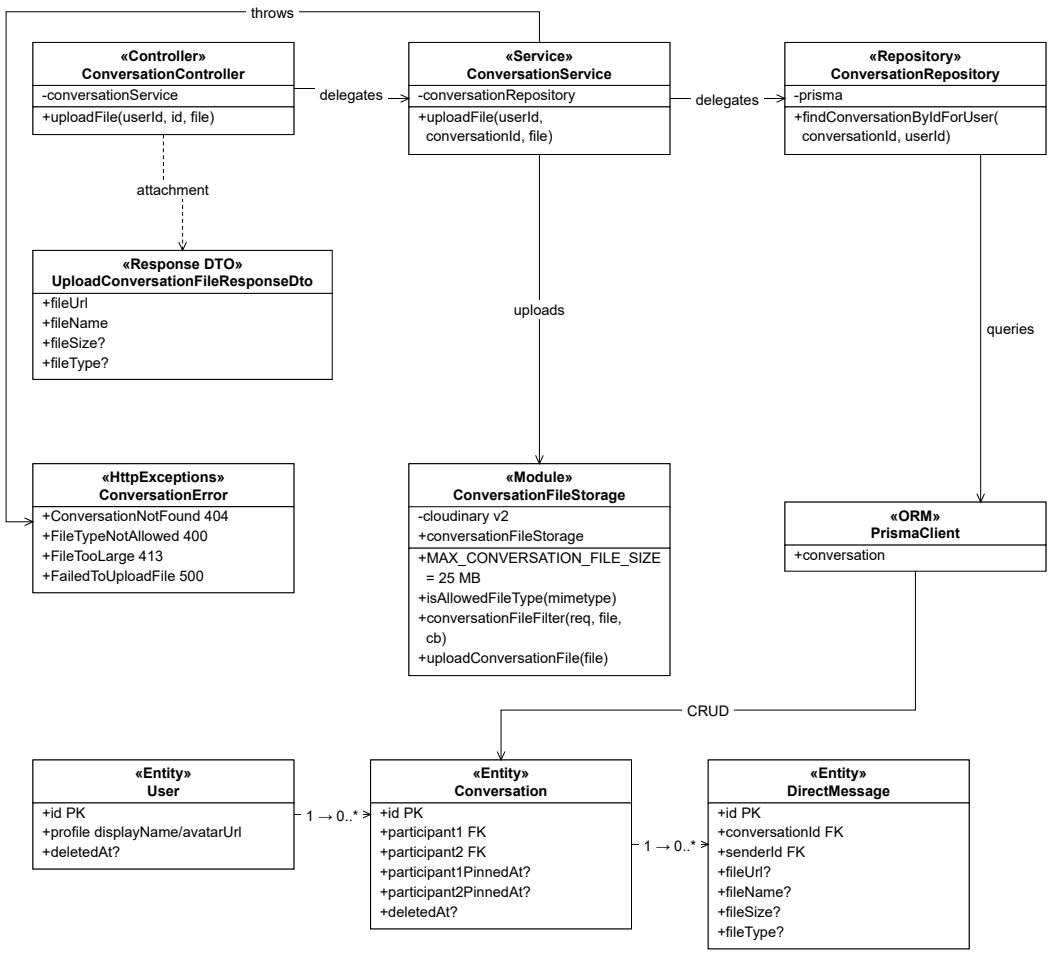


Figure 53: Class diagram of Send File

b. Sequence Diagram(s)

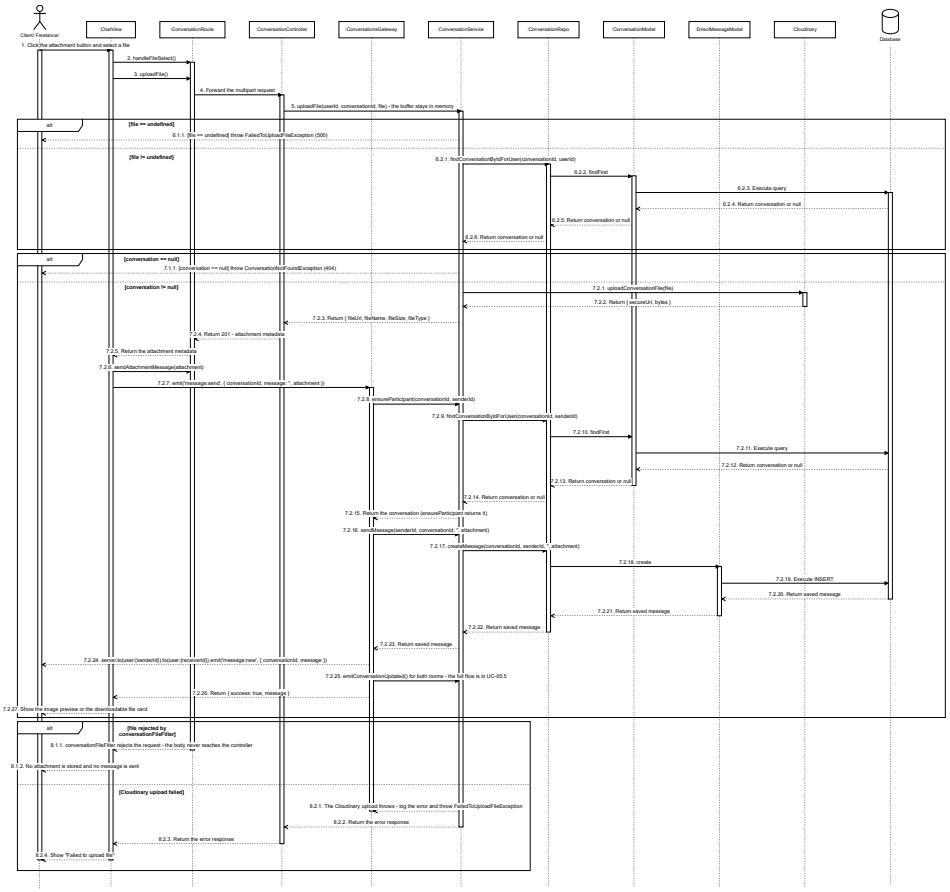


Figure 54: Sequence diagram of Send File

3.26 View Moderation Queue

a. Class Diagram

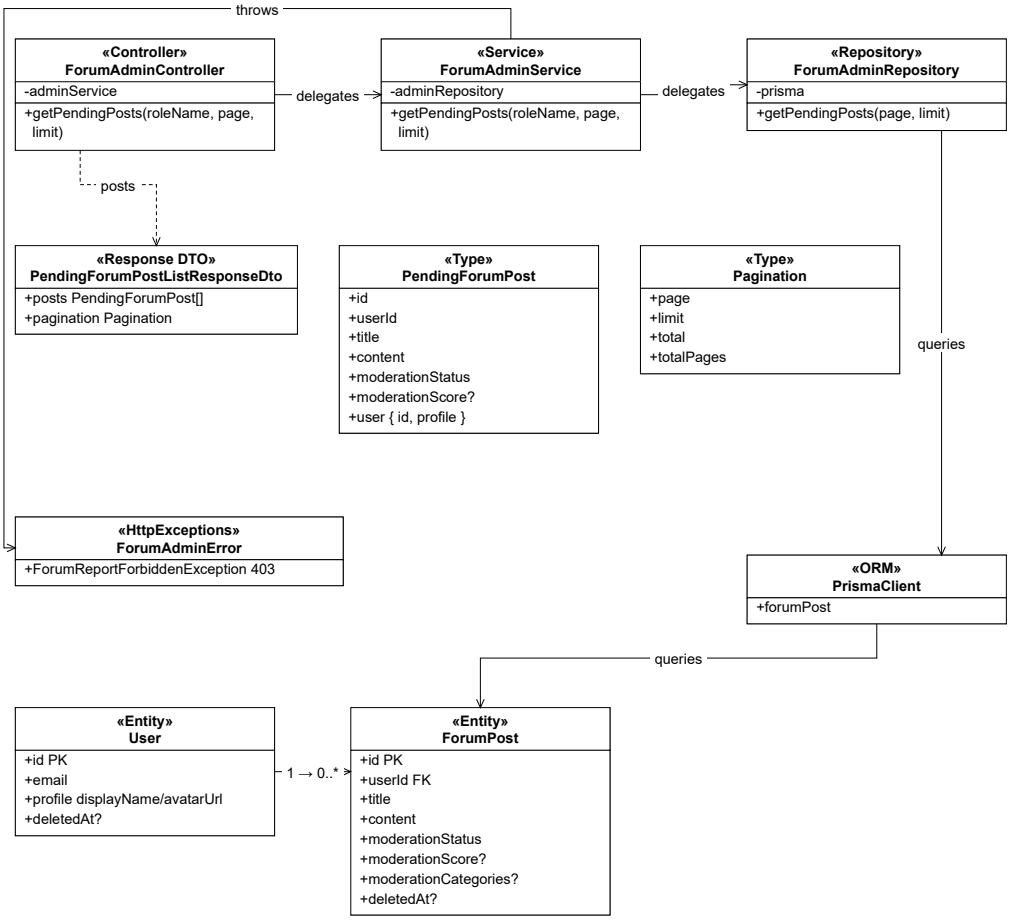


Figure 55: Class diagram of View Moderation Queue

b. Sequence Diagram(s)

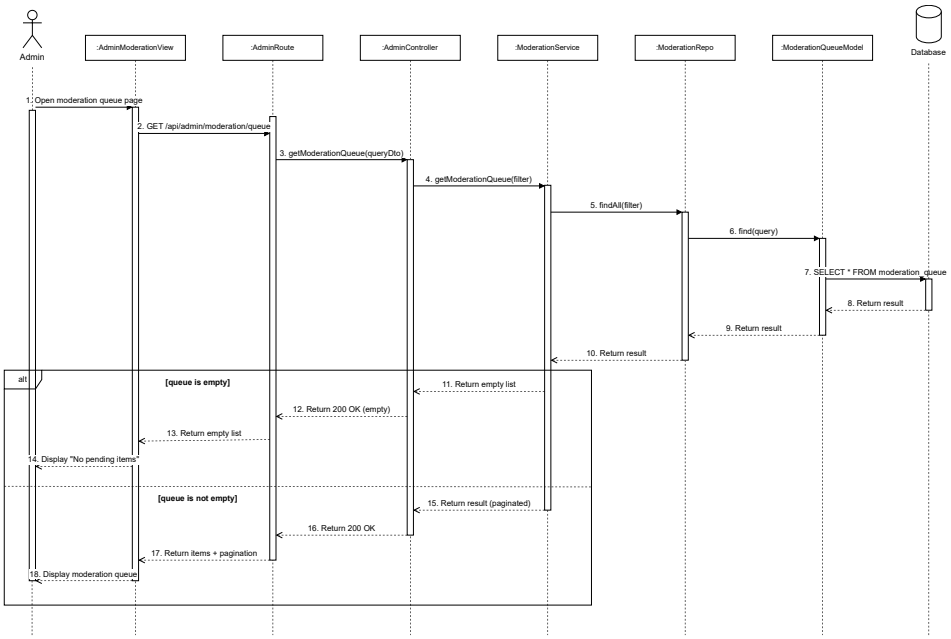


Figure 56: Sequence diagram of View Moderation Queue

3.27 View Moderation Item Detail

a. Class Diagram

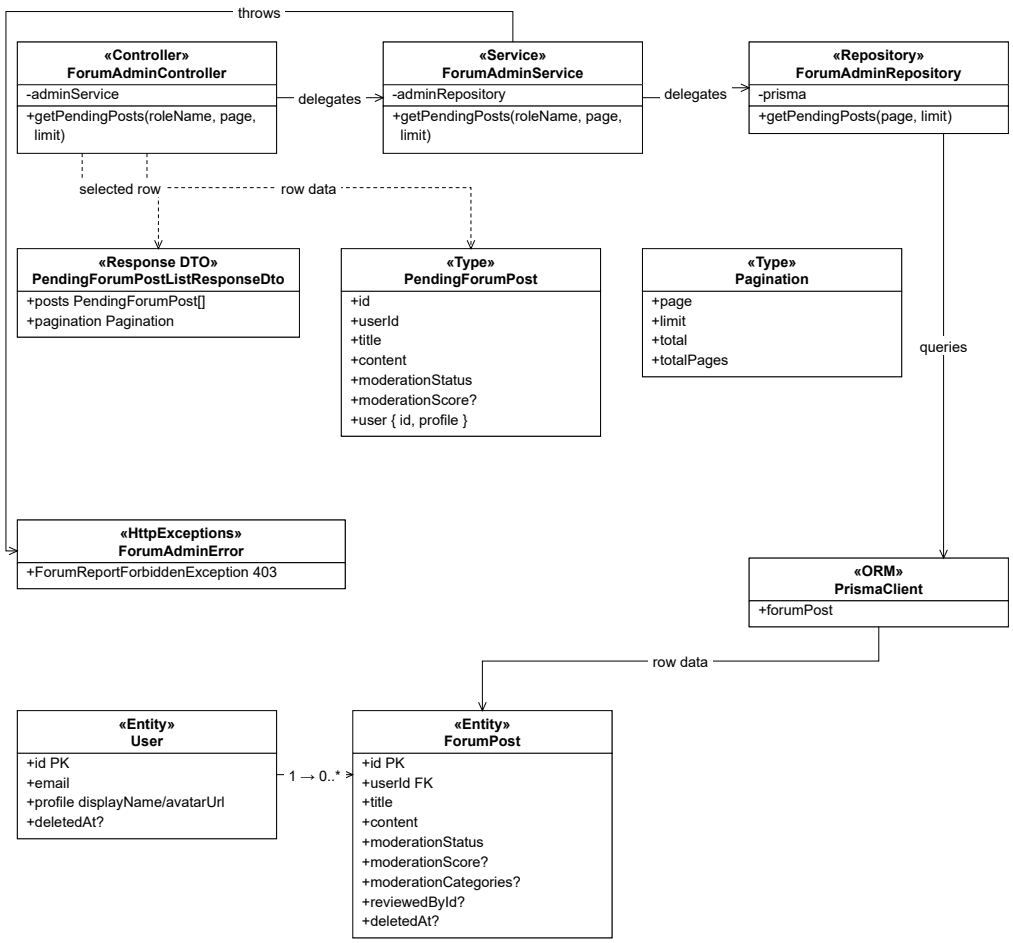


Figure 57: Class diagram of View Moderation Item Detail

b. Sequence Diagram(s)

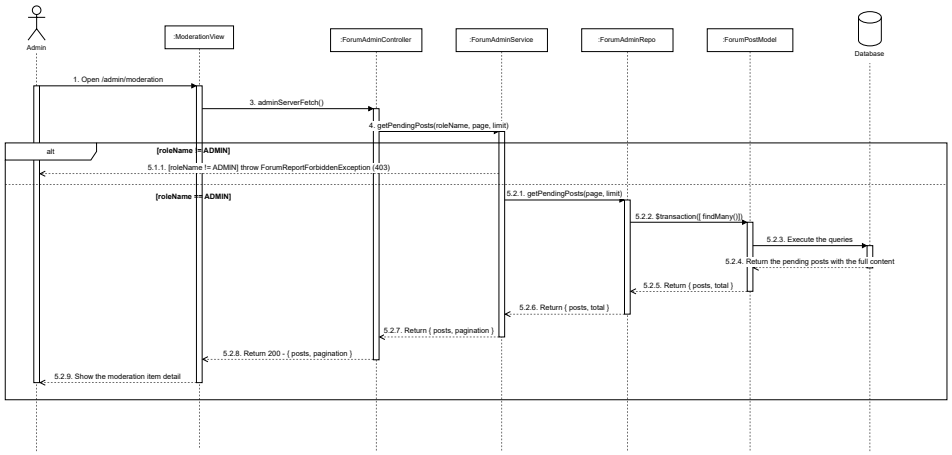


Figure 58: Sequence diagram of View Moderation Item Detail

3.28 Resolve Moderation Item

a. Class Diagram

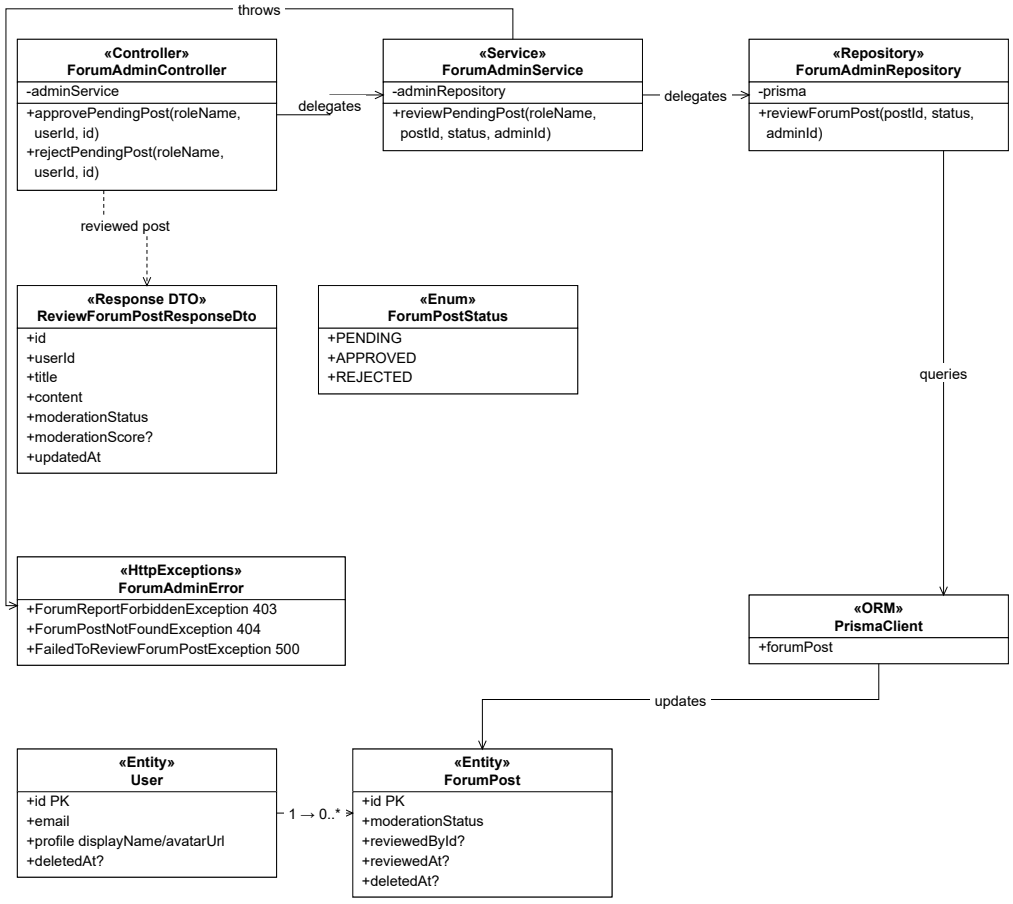


Figure 59: Class diagram of Resolve Moderation Item

b. Sequence Diagram(s)

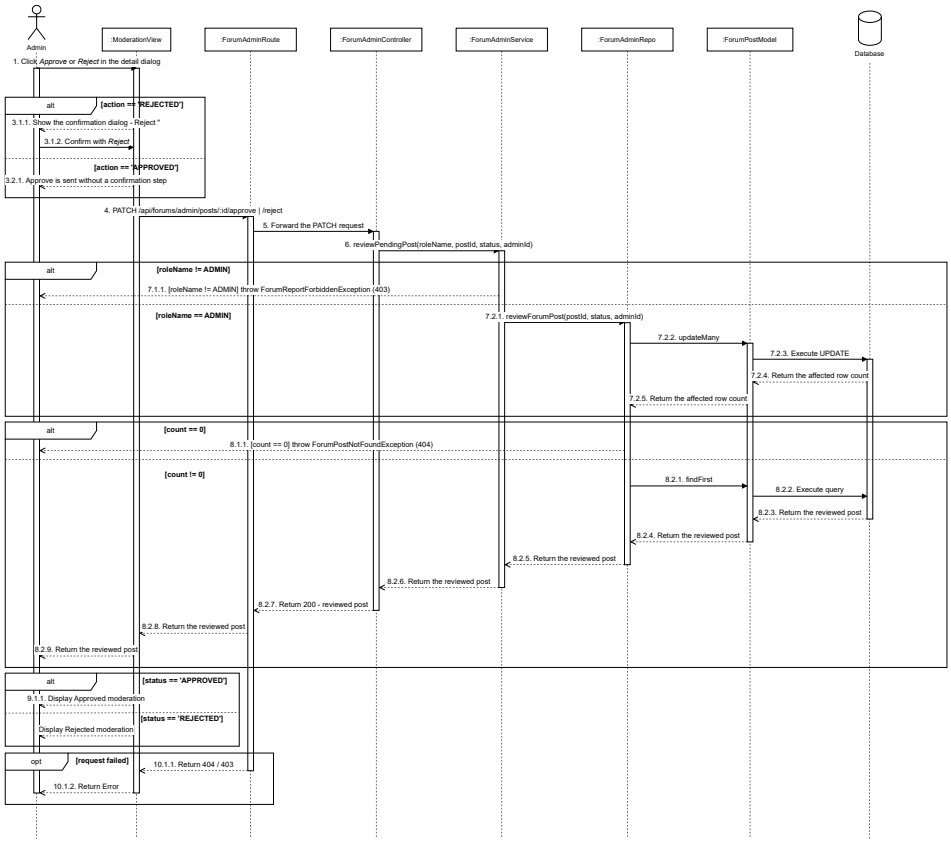


Figure 60: Sequence diagram of Resolve Moderation Item

3.29 View ID Verification Request List

a. Class Diagram

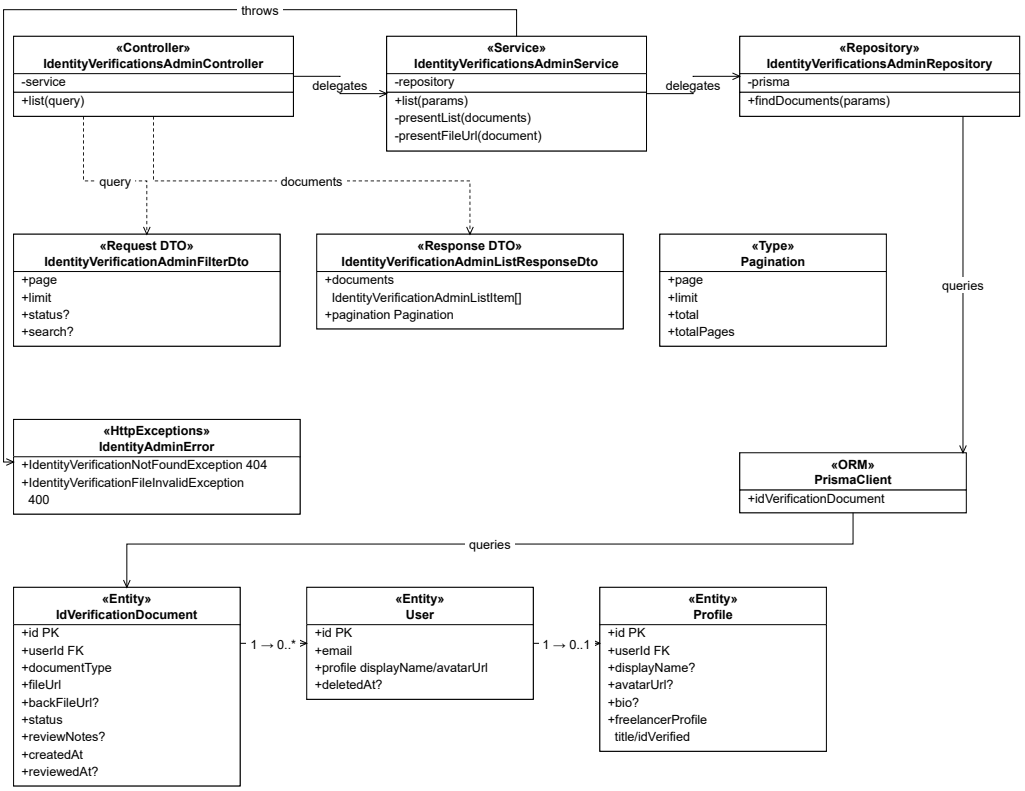


Figure 61: Class diagram of View ID Verification Request List

b. Sequence Diagram(s)

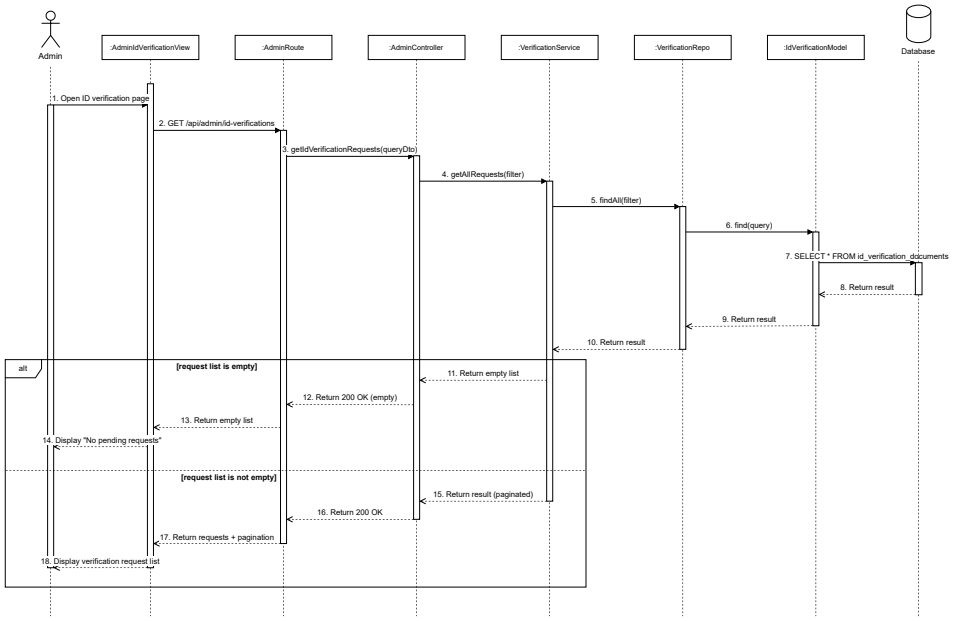


Figure 62: Sequence diagram of View ID Verification Request List

3.30 View ID Verification Request Detail

a. Class Diagram

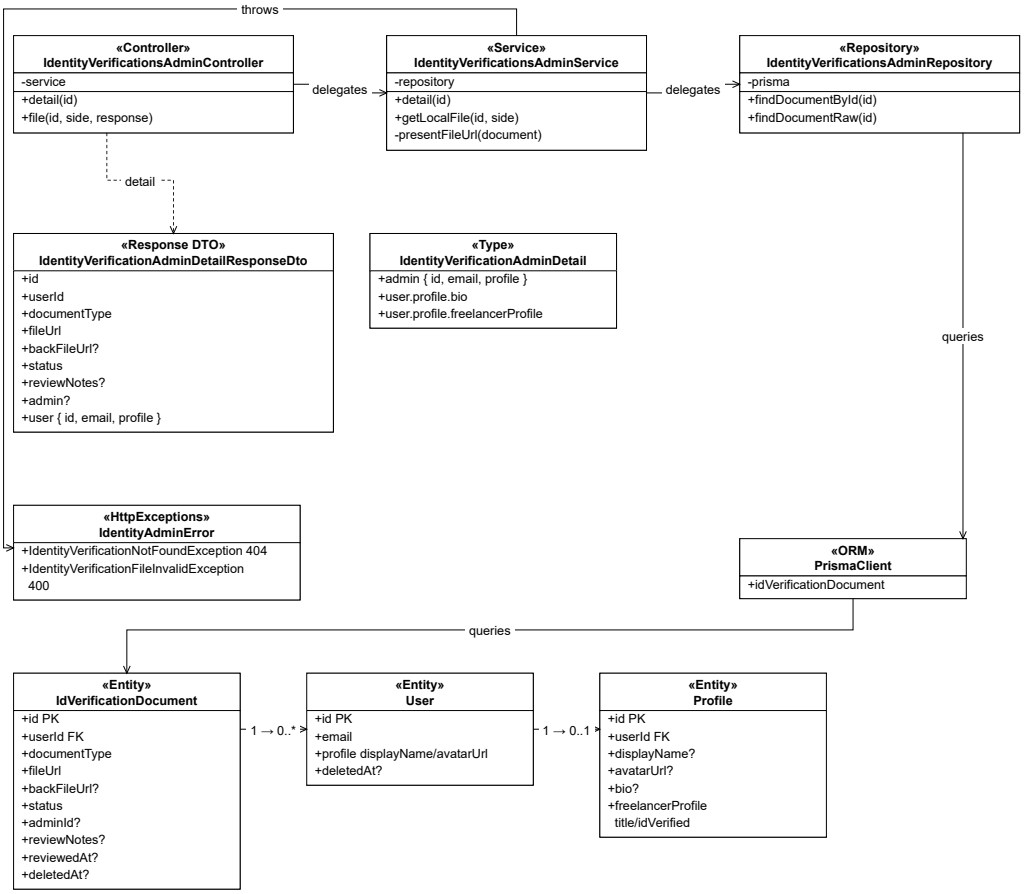


Figure 63: Class diagram of View ID Verification Request Detail

b. Sequence Diagram(s)

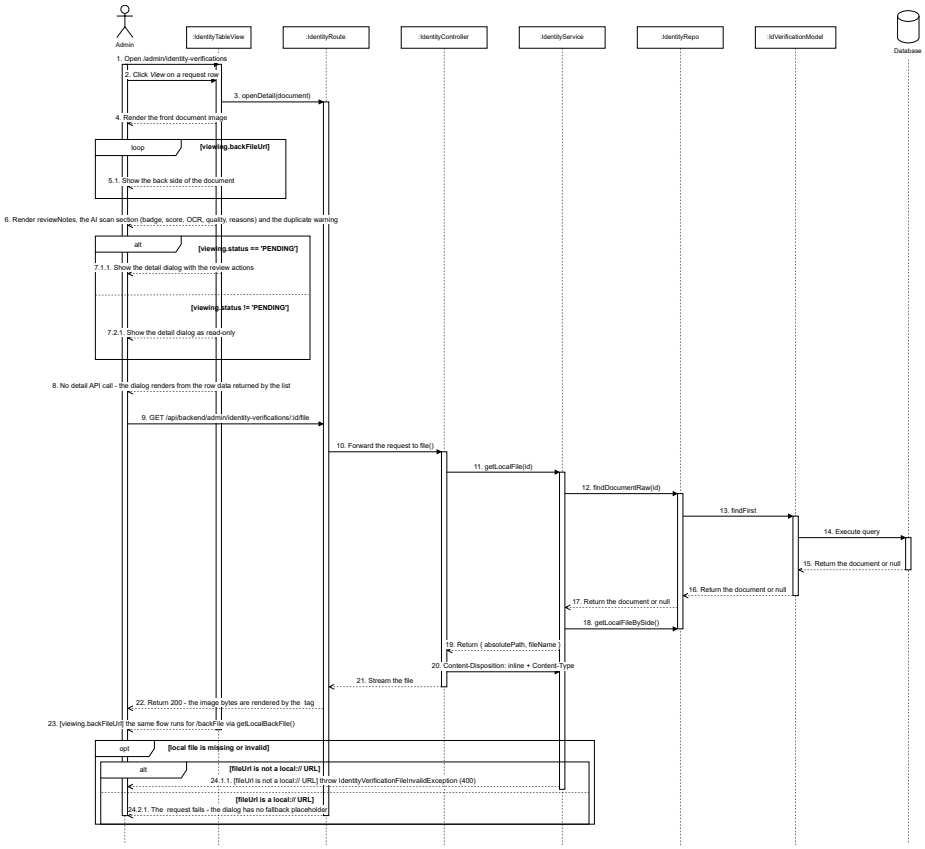


Figure 64: Sequence diagram of View ID Verification Request Detail

3.31 Approve ID Verification

a. Class Diagram

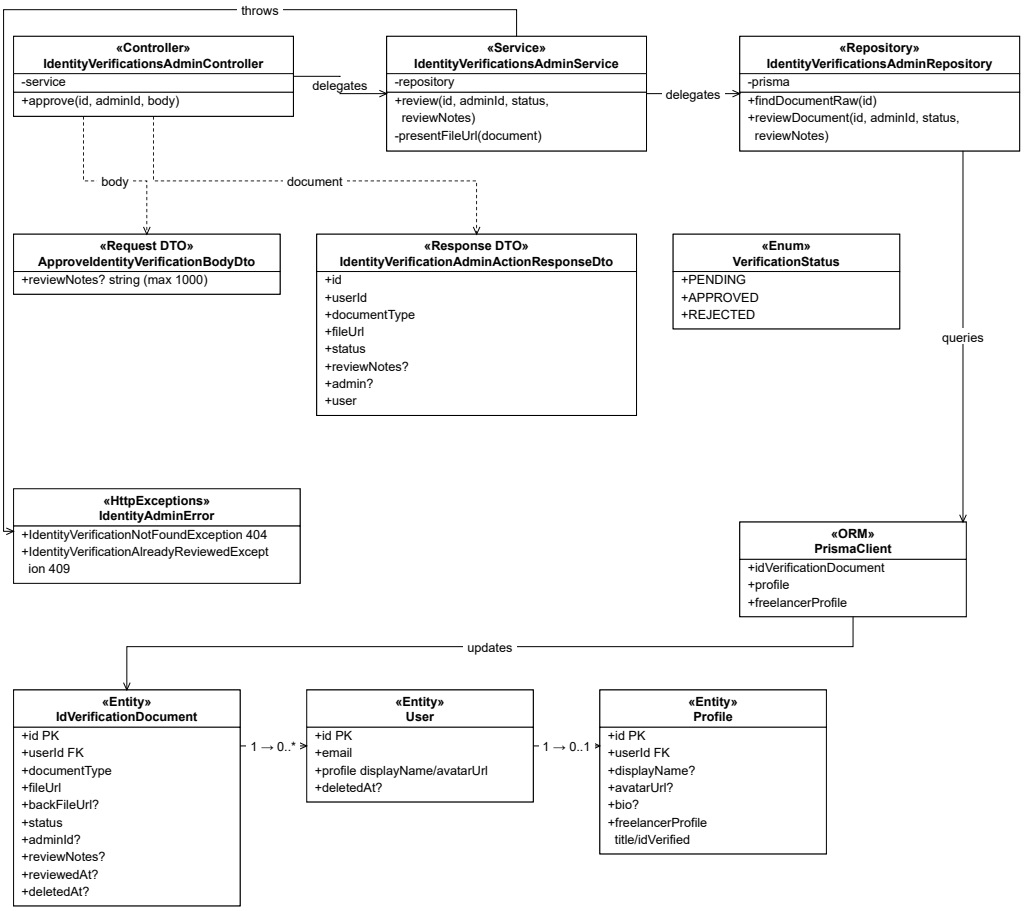


Figure 65: Class diagram of Approve ID Verification

b. Sequence Diagram(s)

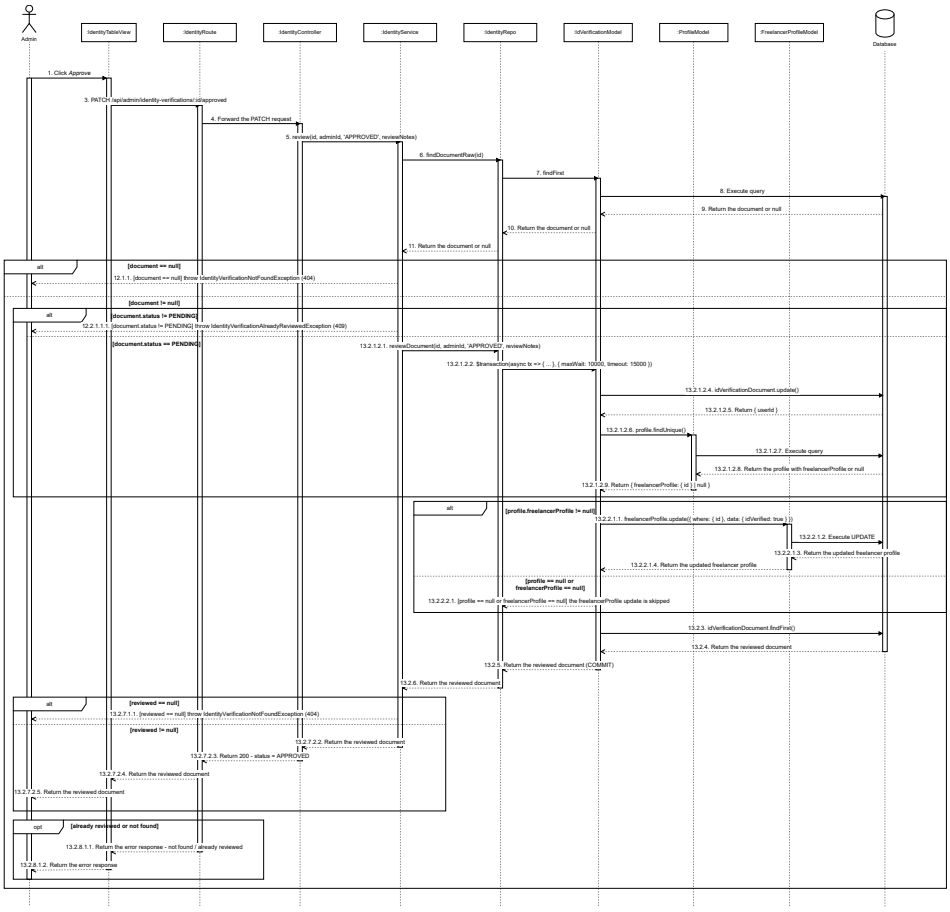


Figure 66: Sequence diagram of Approve ID Verification

3.32 Reject ID Verification

a. Class Diagram

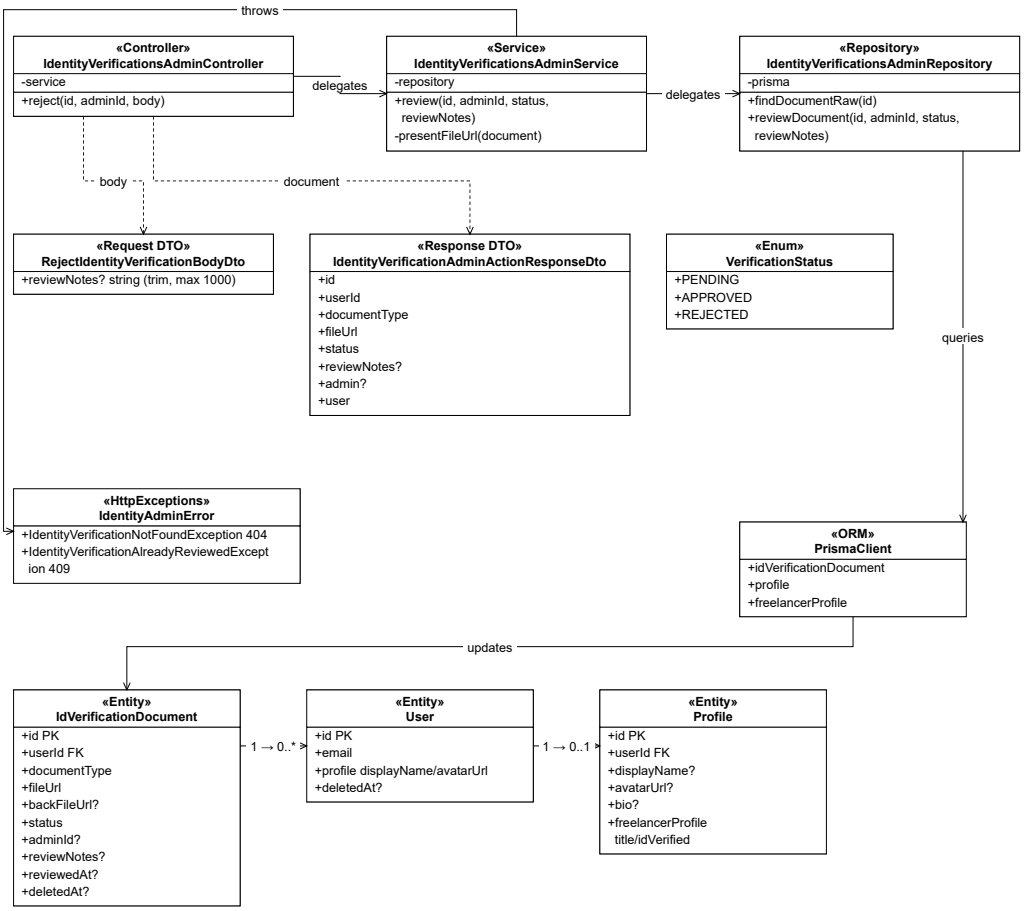


Figure 67: Class diagram of Reject ID Verification

b. Sequence Diagram(s)

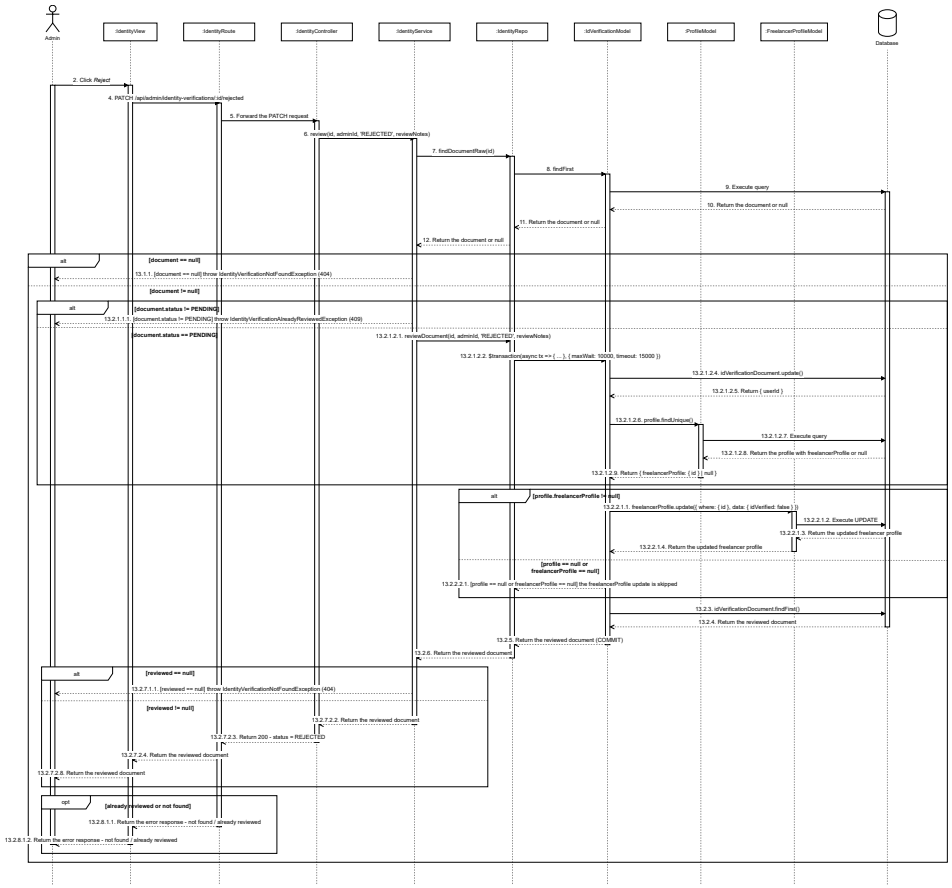


Figure 68: Sequence diagram of Reject ID Verification

3.33 View Deleted Posts/Comments List

a. Class Diagram

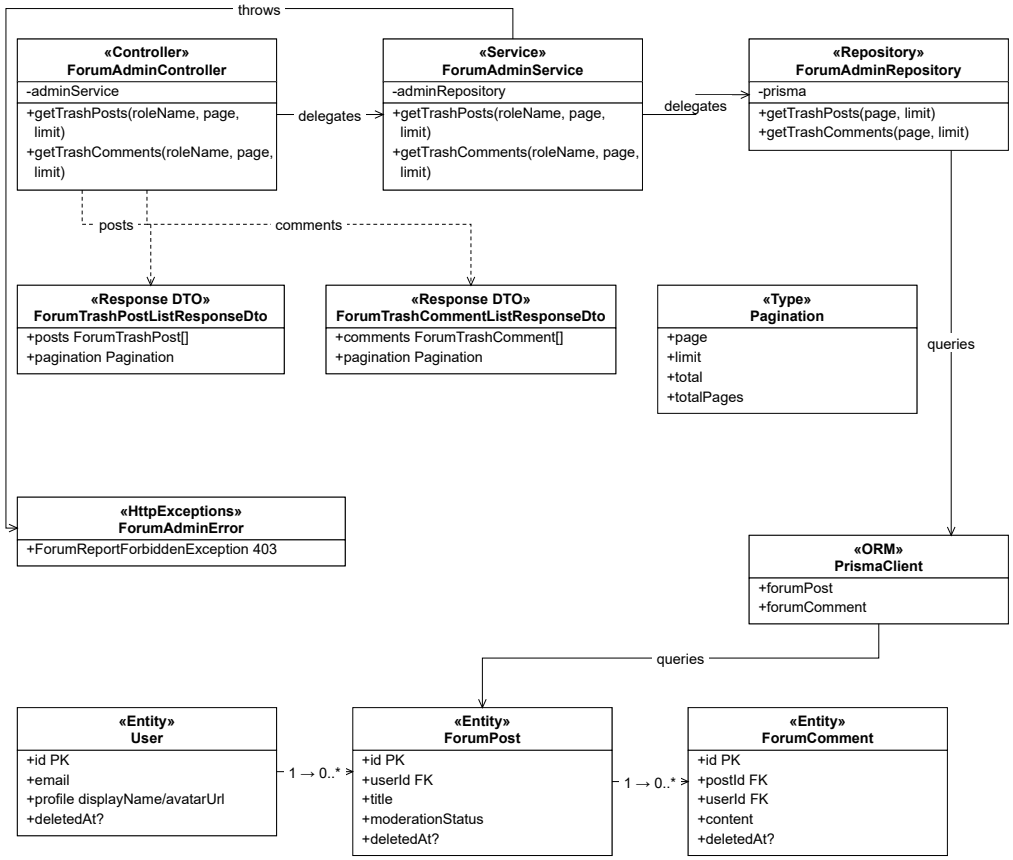


Figure 69: Class diagram of View Deleted Posts/Comments List

b. Sequence Diagram(s)

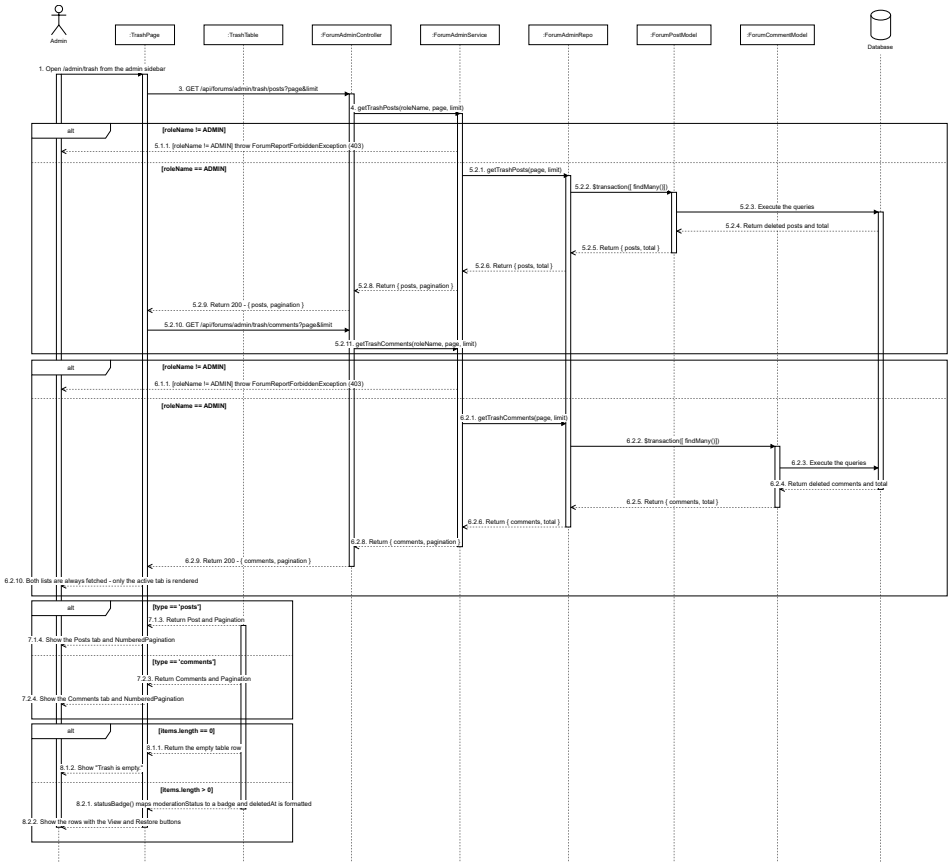


Figure 70: Sequence diagram of View Deleted Posts/Comments List

3.34 View Deleted Posts/Comments Detail

a. Class Diagram

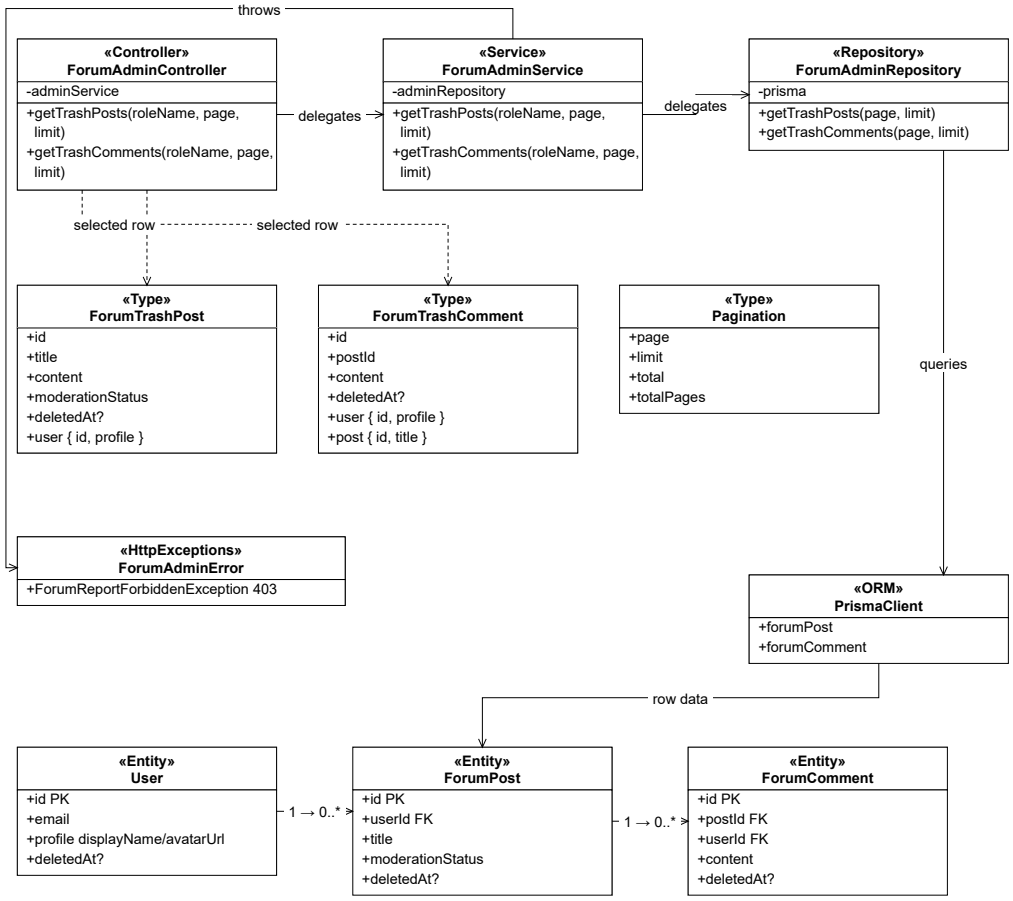


Figure 71: Class diagram of View Deleted Posts/Comments Detail

b. Sequence Diagram(s)

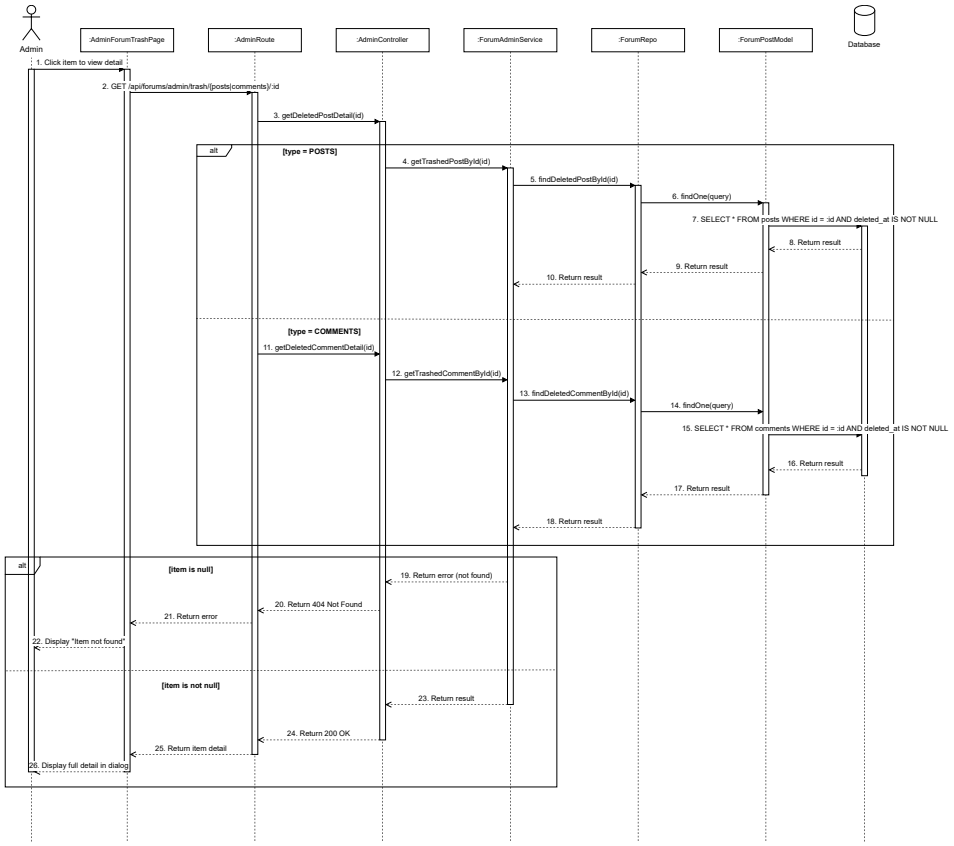


Figure 72: Sequence diagram of View Deleted Posts/Comments Detail

3.35 Restore Deleted Posts/Comments

a. Class Diagram

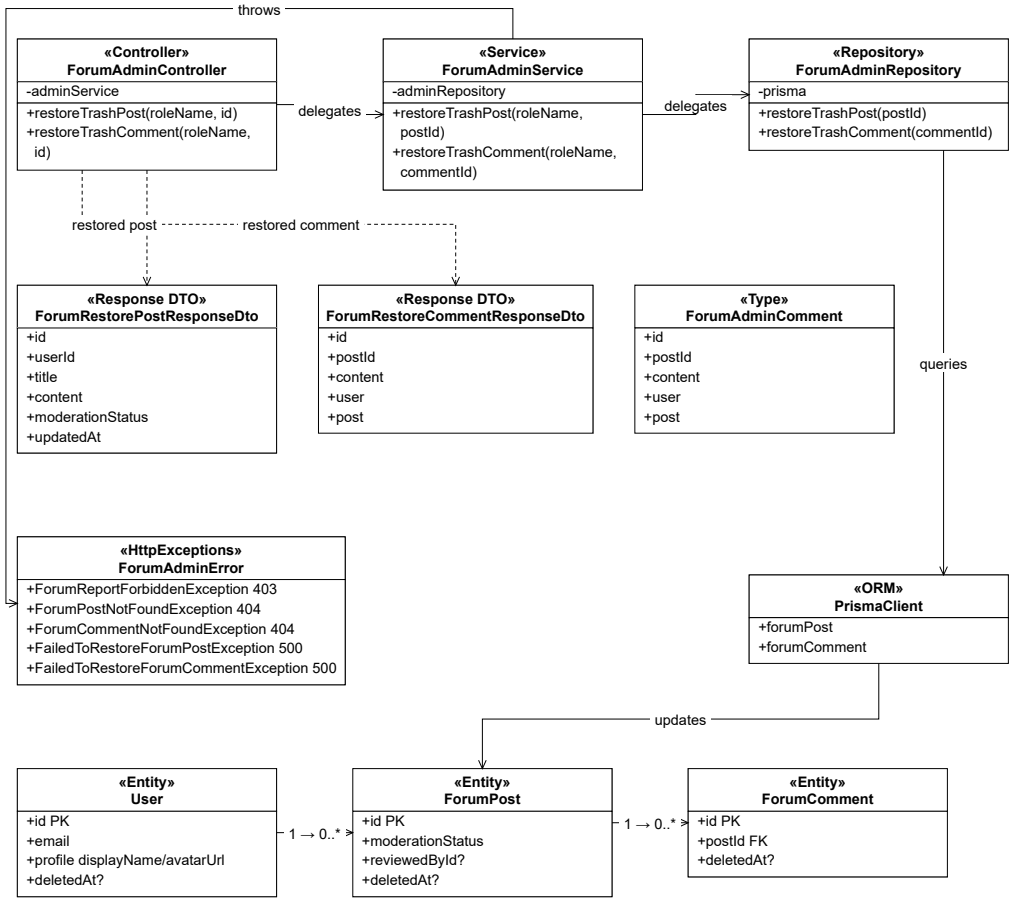


Figure 73: Class diagram of Restore Deleted Posts/Comments

b. Sequence Diagram(s)

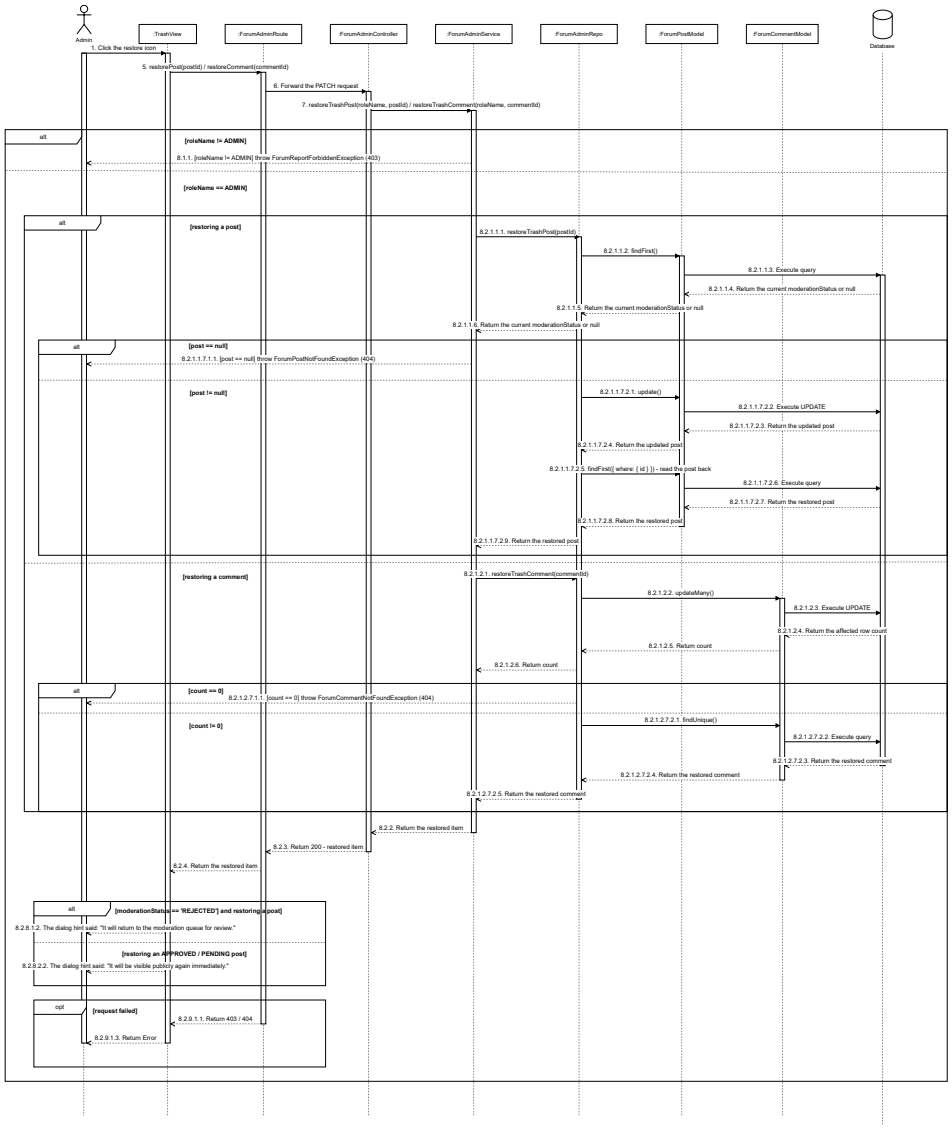


Figure 74: Sequence diagram of Restore Deleted Posts/Comments

4 Class Specification

4.1 ForumController Class

4.1.1 ClassMethod

No	Method	Description
01	`ForumController(forumService: ForumService)`	A method to initialize a new instance of the controller with the forum service. – `forumService`: `ForumService` – the service that owns category and post use cases. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to retrieve all active forum categories. – *Returns:* `Promise<ForumCategoryListResponseType>` – the category list produced by the service. – Route: `GET /forums/categories`; public endpoint.
03	`getTopForumCategories(query: { limit: number })`	A method to retrieve the categories with the largest number of public posts. – `query.limit`: `number` – maximum number of categories, validated by `TopLimitSchema` with default 3 and range 1–10. – *Returns:* `Promise<ForumCategoryTopListResponseType>` – ranked category list. – Route: `GET /forums/categories/top`; public endpoint.
04	`getTopActiveUsers(query: { limit: number })`	A method to retrieve the users with the most forum activity. – `query.limit`: `number` – maximum number of users, validated by `TopLimitSchema` with default 3 and range 1–10. – *Returns:* `Promise<ForumTopActiveUserListResponseType>` – users with post and comment counts. – Route: `GET /forums/users/top`; public endpoint.
05	`getForumCategoryById(id: number)`	A method to retrieve one forum category by its identifier. – `id`: `number` – category identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with post count. – Route: `GET /forums/categories/:id`; public endpoint.
06	`getForumCategoryBySlug(slug: string)`	A method to retrieve one forum category by its slug. – `slug`: `string` – unique category slug. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with post count. – Route: `GET /forums/categories/by-slug/:slug`; public endpoint.

No	Method	Description
07	`getForumPostLists( filter: ForumPostFilter Type)`	A method to retrieve a paginated, filterable list of public posts. – `filter`: `ForumPostFilterType` – page, limit, optional categoryId, optional userId, and optional search text, validated by `ForumPostFilterDto`. – *Returns:* `Promise<ForumPostListResponseType>` – posts plus pagination metadata. – Route: `GET /forums/posts`; public endpoint.
08	`getTopInteractedPosts( query: { limit: number; categoryId?: number })`	A method to retrieve the most interacted posts from the previous seven days. – `query.limit`: `number` – maximum number of posts, default 3. – `query.categoryId`: `number | undefined` – optional category filter. – *Returns:* `Promise<ForumTopPostListResponse Type>` – ranked posts with like and comment counts. – Route: `GET /forums/posts/top`; public endpoint.
09	`viewForumPostDetail(id: number)`	A method to retrieve the public detail of one post. – `id`: `number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponse Type>` – post, author summary, and category summary. – Route: `GET /forums/posts/:id`; public endpoint.
10	`getForumPostBySlug(slug: string)`	A method to retrieve the public detail of one post by slug. – `slug`: `string` – unique post slug. – *Returns:* `Promise<ViewForumPostDetailResponse Type>` – post detail. – Route: `GET /forums/posts/by-slug/:slug`; public endpoint.
11	`createForumPost( userId: number, body: CreateForumPostType)`	A method to create a forum post for the authenticated user. – `userId`: `number` – author identifier injected by `@User Active('userId')`. – `body`: `CreateForumPostType` – categoryId, title, content, and optional moderation result, validated by `CreateForumPostDto`. – *Returns:* `Promise<ForumPostType>` – the created post. – Route: `POST /forums/posts`; authenticated endpoint.

No	Method	Description
12	`deleteForumPost( userId: number, roleName: string, id: number)`	A method to soft-delete a post when the caller is its owner or an administrator. – `userId`: `number` – current user identifier. – `roleName`: `string` – current role name, used for the administrator override. – `id`: `number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post. – Route: `DELETE /forums/posts/:id`; authenticated endpoint.

4.1.2 ClassAttribute

No	Attribute	Type
01	`forumService`	`ForumService` – private readonly service dependency.

4.2 ForumService Class

4.2.1 ClassMethod

No	Method	Description
01	`ForumService(forumRepository: ForumRepository)`	A method to initialize the service with the forum repository. – `forumRepository`: `ForumRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getForumCategories()`	A method to load all active categories. – *Returns:* `Promise<ForumCategoryListResponseType>` – category list with public post counts. – Failure is mapped to `FailedToLoadForumCategoriesException`.

No	Method	Description
03	`getTopForumCategories( limit: number = 3)`	A method to load the categories with the most public posts. – `limit: number` – maximum number of categories; default 3. – *Returns:* `Promise<ForumCategoryTopListResponse Type>` – ranked categories. – Failure is mapped to `FailedToLoadForumCategories Exception`.
04	`getTopActiveUsers(limit: number = 5)`	A method to load the most active users by post and comment counts. – `limit: number` – maximum number of users; default 5. – *Returns:* `Promise<ForumTopActiveUserList ResponseType>` – user activity summaries. – The current implementation reuses `FailedToLoad ForumCategoriesException` for this failure path.
05	`getForumCategoryById(id: number)`	A method to load one category by identifier. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponse Type>` – category detail. – Prisma known errors are mapped to `FailedToLoad ForumCategoriesException`.
06	`getForumCategoryBySlug( slug: string)`	A method to resolve a category slug and return its detail. – `slug: string` – category slug. – *Returns:* `Promise<ForumCategoryDetailResponse Type>` – category detail. – Throws `ForumCategoryNotFoundException` when the slug does not exist.
07	`getForumPostBySlug(slug: string)`	A method to resolve a post slug and return its public detail. – `slug: string` – post slug. – *Returns:* `Promise<ViewForumPostDetailResponse Type>` – post detail. – Throws `ForumPostNotFoundException` when the slug does not exist.

No	Method	Description
08	`getForumPostLists(filter: ForumPostFilterType)`	A method to build a paginated post list from the repository result. – `filter`: `ForumPostFilterType` – page, limit, category, author, and search filters. – *Returns:* `Promise<ForumPostListResponseType>` – posts and pagination metadata, including `totalPages`. – Prisma known errors are mapped to `FailedToLoad ForumPostsException`.
09	`createForumPost( userId: number, body: CreateForumPostType)`	A method to validate the optional category and create a post. – `userId`: `number` – author identifier. – `body`: `CreateForumPostType` – post data and optional AI moderation result. – *Returns:* `Promise<ForumPostType>` – the created post. – Throws `ForumCategoryNotFoundException` for an unknown category and `FailedToCreateForumPost Exception` for a persistence failure.
10	`viewForumPostDetail(id: number)`	A method to load the public detail of a post. – `id`: `number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponse Type>` – post, author, and category summary. – Prisma known errors are mapped to `FailedToView ForumPostException`.
11	`updateForumPost( id: number, userId: number, body: UpdateForumPost Type)`	A method to update a post after checking existence and ownership. – `id`: `number` – post identifier. – `userId`: `number` – current user identifier. – `body`: `UpdateForumPostType` – updated category, title, and content. – *Returns:* `Promise<ForumPostType>` – the updated post. – Throws `ForumPostNotFoundException` or `Forum PostNotOwnedException`. The controller route is currently commented out.

No	Method	Description
12	`deleteForumPost( id: number, userId: number, roleName: string)`	A method to authorize and soft-delete a post. – `id: number` – post identifier. – `userId: number` – current user identifier. – `roleName: string` – current role name; `ADMIN` bypasses the owner check. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post. – Throws `ForumPostNotFoundException`, `ForumPostNotOwnedException`, or `FailedToDeleteForumPostException`.
13	`getTopInteractedPosts( limit: number = 3, categoryId?: number)`	A method to return the most interacted posts from the last seven days. – `limit: number` – maximum number of posts; default 3. – `categoryId: number | undefined` – optional category restriction. – *Returns:* `Promise<ForumTopPostListResponseType>` – ranked posts. – Failure is mapped to `FailedToLoadForumPostsException`.

4.2.2 ClassAttribute

No	Attribute	Type
01	`forumRepository`	`ForumRepository` – private readonly persistence dependency.

4.3 ForumRepository Class

4.3.1 ClassMethod

No	Method	Description
01	`ForumRepository(prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma: PrismaService` – pooled database client. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getForumCategories()`	A method to query active categories ordered by name. – *Returns:* `Promise<Array<Pick<ForumCategory Type, 'id' | 'name' | 'slug' | 'description' | 'createdAt' | 'updatedAt' | 'postCount'>>>` – category rows with public post counts. – Only non-deleted posts with moderation status `APPROVE D` are counted.
03	`getTopForumCategories( limit: number = 3)`	A method to query active categories ordered by public post count. – `limit: number` – maximum number of rows; default 3. – *Returns:* the same category projection as `getForum Categories()`, ranked by `posts._count` descending.
04	`getTopActiveUsers(limit: number = 5)`	A method to query non-deleted, non-banned users ordered by post count. – `limit: number` – maximum number of rows; default 5. – *Returns:* `Promise<ForumTopActiveUserType[]>` – user id, display name, avatar, post count, and comment count. – Comment counts include only non-deleted comments.
05	`findForumCategoryById(id: number)`	A method to check whether an active category exists. – `id: number` – category identifier. – *Returns:* `Promise<Pick<ForumCategoryType, 'id' > | null>` – the id when found, otherwise `null`.
06	`getForumCategoryById(id: number)`	A method to load one active category with its post count. – `id: number` – category identifier. – *Returns:* category projection with `postCount`. – Throws `ForumCategoryNotFoundException` when no active row exists.
07	`findForumCategoryBySlug( slug: string)`	A method to resolve an active category by slug. – `slug: string` – category slug. – *Returns:* `Promise<Pick<ForumCategoryType, 'id' > | null>` – the id when found, otherwise `null`.

No	Method	Description
08	`getForumPostLists(filter: ForumPostFilterType)`	A method to query a page of public posts and the matching total in one transaction. – `filter`: `ForumPostFilterType` – pagination, category, author, and case-insensitive title search. – *Returns:* `Promise<{ posts: ForumPostWithUserType[]; total: number }>` – mapped post rows and total count. – Visibility requires `deletedAt = null` and moderation status `APPROVED`.
09	`createForumPost( categoryId: number | null, userId: number, title: string, content: string, moderation?: ForumModerationType)`	A method to insert a post and generate its slug. – `categoryId`: `number | null` – optional category. – `userId`: `number` – author identifier. – `title`, `content`: `string` – post text. – `moderation`: `ForumModerationType | undefined` – optional AI result; status defaults to `PENDING`. – *Returns:* `Promise<ForumPostType>` – the inserted post.
10	`viewForumPostDetail(id: number)`	A method to query one public post with author and category summaries. – `id`: `number` – post identifier. – *Returns:* `Promise<ViewForumPostDetailResponseType>` – post detail. – Throws `ForumPostNotFoundException` when the post is missing, deleted, or not approved.
11	`findForumPostById(id: number)`	A method to load a non-deleted post for ownership checks. – `id`: `number` – post identifier. – *Returns:* `Promise<ForumPostType | null>` – the post or `null`.
12	`findForumPostBySlug(slug: string)`	A method to resolve a public post slug. – `slug`: `string` – post slug. – *Returns:* `Promise<Pick<ForumPostType, 'id'> | null>` – the id when found, otherwise `null`.

No	Method	Description
13	`updateForumPost( id: number, data: UpdateForumPostType)`	A method to update a non-deleted post and regenerate its slug from the title. – `id: number` – post identifier. – `data: UpdateForumPostType` – category, title, and content. – *Returns:* `Promise<ForumPostType>` – the updated post.
14	`softDeleteForumPost(id: number)`	A method to set `deletedAt` on a non-deleted post. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the soft-deleted post.
15	`getTopInteractedPosts( limit: number = 3, categoryId?: number)`	A method to query posts created during the previous seven days and rank them by interactions. – `limit: number` – maximum number of rows; default 3. – `categoryId: number | undefined` – optional category restriction. – *Returns:* `Promise<ForumTopPostType[]>` – posts with like count, comment count, and `interactionScore`. – Ordering is by likes, then comments, then creation time, all descending.

4.3.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for category and post management

Class / Element	Attributes or Signature	Responsibility
`ForumCategoryList ResponseDto`	Array of `ForumCategoryType`: `id`, `name`, `slug`, `description`, `createdAt`, `updated At`, `postCount`.	Response of `GET / forums/categories`.

Class / Element	Attributes or Signature	Responsibility
ForumCategoryDetail ResponseDto	id: number, name: string, slug: string, description: string | null, createdAt: Date, updatedAt: Date, postCount: number.	Detail response for category id and slug routes.
ForumCategoryTopList ResponseDto	Array of ForumCategoryType.	Response of the top-category route.
ForumTopActiveUser ListResponseDto	Array of ForumTopActiveUserType: id, displayName, avatarUrl, postCount, commentCount.	Response of the top-active-user route.
ForumPostList ResponseDto	posts: ForumPostWithUserType[], pagination: PaginationType.	Paginated post-list response.
ForumPostFilterDto	page: number, limit: number, categoryId?: number, userId?: number, search?: string.	Query validation for the post list.
CreateForumPostDto	categoryId?: number, title: string, content: string, moderation?: Forum ModerationType.	Request body for post creation.
CreateForumPost ResponseDto	ForumPostType: id, categoryId, userId, title, slug, content, moderationStatus, moderationScore, moderation Categories, createdAt, updatedAt.	Response of post creation.
ViewForumPostDetail ResponseDto	ForumPostType fields plus category: { id, name, slug } | null and user: { id, profile }.	Public post-detail response.
UpdateForumPostDto	categoryId?: number, title: string, content: string.	Request body for the implemented update-post method.
UpdateForumPost ResponseDto	ForumPostType.	Response of the implemented update-post method.
ForumTopPostList ResponseDto	Array of ForumTopPostType: post fields plus likeCount, commentCount, interaction Score, user, and category.	Response of the top-post route.

4.4 ForumLikeController Class

4.4.1 ClassMethod

No	Method	Description
01	`ForumLikeController(forumLikeService: ForumLikeService)`	A method to initialize the controller with the post-like service. – `forumLikeService`: `ForumLikeService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`toggleLikePost(userId: number, id: number)`	A method to toggle the current user's like on a post. – `userId`: `number` – current user identifier. – `id`: `number` – post identifier. – *Returns:* `Promise<ToggleLikeResponseType>` – `liked: true` after a like, `liked: false` after an unlike. – Route: `POST /forums/posts/:id/like`; authenticated endpoint.
03	`viewForumLikeDetail(postId: number)`	A method to list the users who liked a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – like rows with user summaries. – Route: `GET /forums/posts/:id/like`; public endpoint.

4.4.2 ClassAttribute

No	Attribute	Type
01	`forumLikeService`	`ForumLikeService` – private readonly service dependency.

4.5 ForumLikeService Class

4.5.1 ClassMethod

No	Method	Description
01	`ForumLikeService(forumLikeRepository: ForumLikeRepository)`	A method to initialize the service with the post-like repository. – `forumLikeRepository`: `ForumLikeRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`toggleLikePost(userId: number, postId: number)`	A method to decide whether a like must be created or removed. – `userId`: `number` – current user identifier. – `postId`: `number` – post identifier. – *Returns:* `Promise<ToggleLikeResponseType>` – the new like state. – Throws `ForumPostNotFoundException` when the post is missing; other failures become `FailedToLikeForumPostException`.
03	`viewForumLikeDetail(postId: number)`	A method to load the like rows of a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – like rows with user summaries. – Failure is mapped to `FailedToViewForumLikeDetailException`.

4.5.2 ClassAttribute

No	Attribute	Type
01	`forumLikeRepository`	`ForumLikeRepository` – private readonly persistence dependency.

4.6 ForumLikeRepository Class

4.6.1 ClassMethod

No	Method	Description
01	`ForumLikeRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findLikeByUserAndPost( userId: number, postId: number)`	A method to find a user's reaction row for a post. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number; commentId: number | null } | null>` – reaction id and optional comment link.
04	`findCommentOnlyLike(user Id: number)`	A method to find a reaction row that currently links only to a comment. – `userId`: `number` – user identifier. – *Returns:* `Promise<{ id: number } | null>` – the reusable reaction row, otherwise `null`.
05	`createPostLike(userId: number, postId: number)`	A method to create a post reaction, reusing a comment-only row when possible. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* the created or updated `ForumLike` row.
06	`deletePostLike(userId: number, postId: number)`	A method to remove a post reaction while preserving a comment reaction on the same row. – `userId`: `number` – user identifier. – `postId`: `number` – post identifier. – *Returns:* the updated or deleted `ForumLike` row, or `undefined` when no row exists.
07	`viewForumLikeDetail(post Id: number)`	A method to query all reaction rows for a post. – `postId`: `number` – post identifier. – *Returns:* `Promise<ForumLikeDetailResponseType>` – reaction rows with user id, profile display name, and avatar.

4.6.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for post reactions

Class / Element	Attributes or Signature	Responsibility
`ToggleLikeResponseDto`	`liked: boolean`.	Response of the toggle-like route.
`ViewForumLikeDetail ResponseDto`	Array of `ForumLikeType` with `user`: `id`, `user Id`, `postId`, `commentId`, `reactionType`, `createdAt`, and user profile summary.	Response of the post-like detail route.

4.7 ForumCommentController Class

4.7.1 ClassMethod

No	Method	Description
01	`ForumCommentController( forumCommentService: ForumCommentService)`	A method to initialize the controller with the comment service. – `forumCommentService`: `ForumCommentService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`getForumCommentLists( postId: number, filter: ForumComment FilterType, userId: number)`	A method to list the comments of a post with pagination. – `postId`: `number` – parent post identifier. – `filter`: `ForumCommentFilterType` – page and limit. – `userId`: `number` – current user, used to compute `liked ByMe`. – *Returns:* `Promise<ForumCommentListResponse Type>` – comments and pagination. – Route: `GET /forums/posts/:postId/comments`; authenticated endpoint.

No	Method	Description
03	`createForumComment(userId: number, postId: number, body: CreateForumCommentType)`	A method to create a comment on a post. – `userId: number` – author identifier. – `postId: number` – parent post identifier. – `body: CreateForumCommentType` – comment content. – *Returns:* `Promise<ForumCommentType>` – the created comment. – Route: `POST /forums/posts/:postId/comments`; authenticated endpoint.
04	`updateForumComment(userId: number, postId: number, id: number, body: EditForumCommentType)`	A method to edit a comment owned by the current user. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – `body: EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment. – Route: `PATCH /forums/posts/:postId/comments/:id`; authenticated endpoint.
05	`deleteForumComment(userId: number, roleName: string, postId: number, id: number)`	A method to soft-delete a comment owned by the user or by an administrator. – `userId: number` – current user identifier. – `roleName: string` – current role name. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment. – Route: `DELETE /forums/posts/:postId/comments/:id`; authenticated endpoint.
06	`toggleLikeComment(userId: number, postId: number, commentId: number)`	A method to toggle the current user's like on a comment. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<ToggleLikeCommentType>` – the new like state. – Route: `POST /forums/posts/:postId/comments/:commentId/like`; authenticated endpoint.

4.7.2 ClassAttribute

No	Attribute	Type
01	`forumCommentService`	`ForumCommentService` – private readonly service dependency.

4.8 ForumCommentService Class

4.8.1 ClassMethod

No	Method	Description
01	`ForumCommentService( forumCommentRepository: ForumCommentRepository)`	A method to initialize the service with the comment repository. – `forumCommentRepository: ForumComment Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`createForumComment( postId: number, userId: number, body: CreateForumComment Type)`	A method to verify the parent post and create a comment. – `postId: number` – parent post identifier. – `userId: number` – author identifier. – `body: CreateForumCommentType` – comment content. – *Returns:* `Promise<ForumCommentType>` – the created comment. – Throws `ForumPostNotFoundException` or `FailedTo CreateForumCommentException`.
03	`updateForumComment( postId: number, id: number, userId: number, body: EditForumCommentType)`	A method to verify comment existence, parent post, and ownership before editing. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – `userId: number` – current user identifier. – `body: EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment. – Throws `ForumCommentNotFoundException` or `Forum CommentNotOwnedException`.

No	Method	Description
04	`deleteForumComment( postId: number, id: number, userId: number, roleName: string)`	A method to authorize and soft-delete a comment. – `postId: number` – parent post identifier. – `id: number` – comment identifier. – `userId: number` – current user identifier. – `roleName: string` – `ADMIN` bypasses the owner check. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment. – Throws `ForumCommentNotFoundException`, `ForumCommentNotOwnedException`, or `FailedToDeleteForumCommentException`.
05	`toggleLikeComment( userId: number, postId: number, commentId: number)`	A method to create or remove a comment reaction. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<ToggleLikeCommentType>` – the new like state. – Throws `ForumCommentNotFoundException` or `FailedToLikeForumCommentException`.
06	`getForumCommentLists( filter: ForumCommentFilterType)`	A method to build a paginated comment list. – `filter: ForumCommentFilterType` – page, limit, postId, and optional userId. – *Returns:* `Promise<ForumCommentListResponseType>` – comments and pagination metadata. – Prisma known errors are mapped to `FailedToLoadForumCommentsException`.

4.8.2 ClassAttribute

No	Attribute	Type
01	`forumCommentRepository`	`ForumCommentRepository` – private readonly persistence dependency.

4.9 ForumCommentRepository Class

4.9.1 ClassMethod

No	Method	Description
01	`ForumCommentRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findCommentById(id: number)`	A method to load a non-deleted comment with its parent post and author summary. – `id`: `number` – comment identifier. – *Returns:* the comment projection or `null`.
04	`createForumComment( postId: number, userId: number, content: string)`	A method to insert a comment and return the response shape. – `postId`: `number` – parent post identifier. – `userId`: `number` – author identifier. – `content`: `string` – comment text. – *Returns:* `Promise<ForumCommentType>` – the comment with `likeCount = 0` and `likedByMe = false`.
05	`updateForumComment( id: number, data: EditForumCommentType)`	A method to update the content of a non-deleted comment. – `id`: `number` – comment identifier. – `data`: `EditForumCommentType` – new content. – *Returns:* `Promise<ForumCommentType>` – the updated comment.
06	`softDeleteForumComment( id: number)`	A method to set `deletedAt` on a non-deleted comment. – `id`: `number` – comment identifier. – *Returns:* `Promise<ForumCommentType>` – the soft-deleted comment.
07	`getForumCommentLists( filter: ForumCommentFilterType)`	A method to query a page of comments and the matching total in one transaction. – `filter`: `ForumCommentFilterType` – page, limit, postId, and optional current user. – *Returns:* `Promise<{ comments: ForumCommentType[]; total: number }>` – mapped comments and total. – `likedByMe` is computed from the optional user reaction.

No	Method	Description
08	`findLikeByUserAndComment( userId: number, commentId: number)`	A method to find a user's comment reaction. – `userId: number` – user identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<{ id: number; postId: number | null } | null>` – reaction id and optional post link.
09	`findLikeByUserAndPost( userId: number, postId: number)`	A method to find a user's post reaction for reuse by the comment-like flow. – `userId: number` – user identifier. – `postId: number` – post identifier. – *Returns:* `Promise<{ id: number; commentId: number | null } | null>` – reaction id and optional comment link.
10	`createCommentLike( userId: number, postId: number, commentId: number)`	A method to create a comment reaction, reusing a post-only row when possible. – `userId: number` – user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* the created or updated `ForumLike` row.
11	`deleteCommentLike( userId: number, commentId: number)`	A method to remove a comment reaction while preserving a post reaction on the same row. – `userId: number` – user identifier. – `commentId: number` – comment identifier. – *Returns:* the updated or deleted `ForumLike` row, or `undefined` when no row exists.

4.9.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for comments

Class / Element	Attributes or Signature	Responsibility
ForumCommentList ResponseDto	comments: ForumCommentType[], pagination: PaginationType.	Paginated comment-list response.
ForumCommentFilterDto	page: number, limit: number.	Query validation for comment listing.
CreateForumCommentDto	content: string.	Comment-creation body.
CreateForumComment ResponseDto	ForumCommentType: id, postId, user, content, createdAt, updatedAt, like Count, likedByMe.	Comment-creation response.
EditForumCommentDto	content: string.	Comment-update body.
EditForumComment ResponseDto	ForumCommentType.	Comment-update response.
DeleteForumComment ResponseDto	ForumCommentType.	Soft-delete response.
ToggleLikeComment ResponseDto	liked: boolean.	Comment-like toggle response.

4.10 ForumReportController Class

4.10.1 ClassMethod

No	Method	Description
01	ForumReportController(forumReportService: ForumReportService)	A method to initialize the controller with the report service. – forumReportService: ForumReportService – service dependency. – *Returns:* void – constructor, no return value.

No	Method	Description
02	`createPostReport( userId: number, postId: number, body: CreateForumReportType)`	A method to report a post. – `userId: number` – reporter identifier. – `postId: number` – reported post identifier. – `body: CreateForumReportType` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Route: `POST /forums/posts/:postId/reports`; authenticated endpoint.
03	`createCommentReport( userId: number, postId: number, commentId: number, body: CreateForumReportType)`	A method to report a comment. – `userId: number` – reporter identifier. – `postId: number` – parent post identifier. – `commentId: number` – reported comment identifier. – `body: CreateForumReportType` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Route: `POST /forums/posts/:postId/comments/:commentId/reports`; authenticated endpoint.
04	`checkPostReported( userId: number, postId: number)`	A method to check whether the current user has reported a post. – `userId: number` – current user identifier. – `postId: number` – post identifier. – *Returns:* `Promise<{ reported: boolean }>` – true when a matching report exists. – Route: `GET /forums/posts/:postId/is-reported`; authenticated endpoint.
05	`checkCommentReported( userId: number, postId: number, commentId: number)`	A method to check whether the current user has reported a comment. – `userId: number` – current user identifier. – `postId: number` – parent post identifier. – `commentId: number` – comment identifier. – *Returns:* `Promise<{ reported: boolean }>` – true when a matching report exists. – Route: `GET /forums/posts/:postId/comments/:commentId/is-reported`; authenticated endpoint.

No	Method	Description
06	`getForumReportLists( userId: number, roleName: string, page: number, limit: number, status: string, search: string)`	A method to list the report queue for administrators. – `userId: number` – current user identifier. – `roleName: string` – role checked by the service. – `page, limit: number` – pagination, defaulted to 1 and 10. – `status, search: string` – optional status and reason filters. – *Returns:* `Promise<ForumReportListResponseType>` – reports and pagination. – Route: `GET /forums/reports`; administrator endpoint.
07	`updateReportStatus( userId: number, roleName: string, reportId: number, body: UpdateReportStatusType)`	A method to change the status of a report. – `userId: number` – administrator identifier recorded on the report. – `roleName: string` – role checked by the service. – `reportId: number` – report identifier. – `body: UpdateReportStatusType` – target status. – *Returns:* `Promise<UpdateReportStatusResponseType>` – the updated report. – Route: `PATCH /forums/reports/:reportId/status`; administrator endpoint.

4.10.2 ClassAttribute

No	Attribute	Type
01	`forumReportService`	`ForumReportService` – private readonly service dependency.

4.11 ForumReportService Class

4.11.1 ClassMethod

No	Method	Description
01	`ForumReportService( forumReportRepository: ForumReportRepository)`	A method to initialize the service with the report repository. – `forumReportRepository: ForumReportRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`createPostReport( reporterId: number, postId: number, reason: string)`	A method to validate a post and create its first report by the user. – `reporterId: number` – reporter identifier. – `postId: number` – reported post identifier. – `reason: string` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Throws `ForumPostNotFoundException`, `ForumReportAlreadyExistsException`, or `FailedToCreateForumReportException`.
03	`createCommentReport( reporterId: number, postId: number, commentId: number, reason: string)`	A method to validate a comment and create its first report by the user. – `reporterId: number` – reporter identifier. – `postId: number` – parent post identifier. – `commentId: number` – reported comment identifier. – `reason: string` – report reason. – *Returns:* `Promise<CreateForumReportResponseType>` – the created report. – Throws `ForumCommentNotFoundException`, `ForumReportAlreadyExistsException`, or `FailedToCreateForumReportException`.
04	`checkReportedByUser( reporterId: number, postId: number | null, commentId: number | null)`	A method to find an existing report for the current user and target. – `reporterId: number` – reporter identifier. – `postId: number | null` – post target, or `null`. – `commentId: number | null` – comment target, or `null`. – *Returns:* `Promise<{ id: number } | null>` – the report id when one exists.

No	Method	Description
05	`getForumReportLists( roleName: string, page: number, limit: number, status?: string, search?: string)`	A method to authorize an administrator and return the report queue. – `roleName: string` – must equal `ADMIN`. – `page, limit: number` – pagination. – `status, search: string | undefined` – optional filters. – *Returns:* `Promise<ForumReportListResponseType>` – reports and pagination. – Throws `ForumReportForbiddenException` or `FailedToLoadForumReportsException`.
06	`updateReportStatus( reportId: number, status: string, adminId: number, roleName: string)`	A method to authorize an administrator and update a post report status. – `reportId: number` – report identifier. – `status: string` – target status. – `adminId: number` – administrator identifier. – `roleName: string` – must equal `ADMIN`. – *Returns:* `Promise<UpdateReportStatusResponseType>` – the updated report. – Throws `ForumReportForbiddenException`, `ForumReportNotFoundException`, or `FailedToUpdateReportStatusException`.

4.11.2 ClassAttribute

No	Attribute	Type
01	`forumReportRepository`	`ForumReportRepository` – private readonly persistence dependency.

4.12 ForumReportRepository Class

4.12.1 ClassMethod

No	Method	Description
01	`ForumReportRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findPostById(postId: number)`	A method to check that a non-deleted post exists. – `postId`: `number` – post identifier. – *Returns:* `Promise<{ id: number } | null>` – the id when found, otherwise `null`.
03	`findCommentById(comment Id: number)`	A method to check that a non-deleted comment exists and belongs to a post. – `commentId`: `number` – comment identifier. – *Returns:* `Promise<{ id: number; postId: number } | null>` – comment id and parent post id.
04	`findExistingReport( reporterId: number, post Id: number | null, commentId: number | null)`	A method to find a duplicate report by the same user and target. – `reporterId`: `number` – reporter identifier. – `postId`: `number | null` – post target, or `null`. – `commentId`: `number | null` – comment target, or `null`. – *Returns:* `Promise<{ id: number } | null>` – the report id when one exists.
05	`createReport( reporterId: number, post Id: number | null, commentId: number | null, reason: string)`	A method to insert a report. – `reporterId`: `number` – reporter identifier. – `postId`, `commentId`: `number | null` – exactly one target is populated by the service. – `reason`: `string` – report reason. – *Returns:* `Promise<ForumReportType>` – the created report.
06	`findReportById(id: number)`	A method to load a report by identifier. – `id`: `number` – report identifier. – *Returns:* `Promise<ForumReportType | null>` – the report or `null`.

No	Method	Description
07	`getReportList( page: number, limit: number, status?: string, search?: string)`	A method to query a page of reports and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `status`: `string | undefined` – optional status filter. – `search`: `string | undefined` – optional case-insensitive reason search. – *Returns:* `Promise<{ reports: ForumReportListResponseType['reports']; total: number }>` – report rows and total.
08	`updateReportStatus( id: number, status: string, adminId: number | null)`	A method to update a report status and record the reviewing administrator. – `id`: `number` – report identifier. – `status`: `string` – target status, converted to upper case. – `adminId`: `number | null` – reviewing administrator. – *Returns:* `Promise<ForumReportType>` – the updated report.

4.12.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

Supporting DTO classes for reports

Class / Element	Attributes or Signature	Responsibility
`CreateForumReportDto`	`reason: string.`	Report-creation body.
`CreateForumReportResponseDto`	`ForumReportType`: `id`, `reporterId`, `postId`, `commentId`, `reason`, `status`, `adminId`, `createdAt`.	Report-creation response.
`UpdateReportStatusDto`	`status: PENDING | REVIEWED | RESOLVED | DISMISSED.`	Report-status body.
`UpdateReportStatusResponseDto`	`ForumReportType`.	Report-status response.

Class / Element	Attributes or Signature	Responsibility
ForumReportList ResponseDto	reports: ForumReportType[] with reporter, post, and comment summaries, plus pagination.	Administrator report queue.

4.13 ConversationController Class

4.13.1 ClassMethod

No	Method	Description
01	ConversationController(conversationService: ConversationService)	A method to initialize the controller with the conversation service. – conversationService: ConversationService – service that owns all conversation use cases. – *Returns:* void – constructor, no return value.
02	getConversations(userId: number)	A method to return the current user's conversation list. – userId: number – current user identifier injected by @ UserActive('userId'). – *Returns:* Promise<GetConversationsResponse Type> – conversation items with other user, last message, unread count, and pin time. – Route: GET /conversations; authenticated endpoint.
03	createConversation(userId: number, body: CreateConversationBody Type)	A method to create or reopen a conversation with another user. – userId: number – current user identifier. – body.participantId: number – the other participant, validated by CreateConversationBodyDto. – *Returns:* Promise<CreateConversationResponse Type> – id, participant1, participant2, and createdAt. – Route: POST /conversations; authenticated endpoint.

No	Method	Description
04	`getMessages( userId: number, id: number)`	A method to return the message history of one conversation. – `userId: number` – current user identifier. – `id: number` – conversation identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<DirectMessageType[]>` – non-deleted messages in ascending creation order. – Route: `GET /conversations/:id/messages`; authenticated endpoint.
05	`sendMessage( userId: number, id: number, body: SendMessageBodyType)`	A method to send a message through the REST fallback. – `userId: number` – sender identifier. – `id: number` – conversation identifier. – `body: SendMessageBodyType` – message text and optional attachment, validated by `SendMessageBodyDto`. – *Returns:* `Promise<DirectMessageType>` – the saved message. – Route: `POST /conversations/:id/messages`; authenticated endpoint.
06	`uploadFile( userId: number, id: number, file?: Express.Multer.File)`	A method to upload a message attachment for a conversation participant. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – `file: Express.Multer.File | undefined` – multipart file read into memory by `FileInterceptor`. – *Returns:* `Promise<MessageAttachmentType>` – Cloudinary URL, original name, byte size, and MIME type. – Route: `POST /conversations/:id/files`; authenticated endpoint.
07	`deleteMessage( userId: number, id: number, messageId: number)`	A method to soft-delete a message sent by the current user. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – `messageId: number` – message identifier. – *Returns:* `Promise<DeleteMessageResponseType>` – the deleted message id. – Route: `DELETE /conversations/:id/messages/:messageId`; authenticated endpoint.

No	Method	Description
08	`markAsRead( userId: number, id: number)`	A method to mark all messages from the other participant as read. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – *Returns:* `Promise<{ count: number }>` – number of updated messages. This route has no response DTO decorator. – Route: `PATCH /conversations/:id/read`; authenticated endpoint.
09	`hideConversation( userId: number, id: number)`	A method to hide a conversation only for the current user. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – *Returns:* `Promise<HideConversationResponse Type>` – id and hidden flag. – Route: `PATCH /conversations/:id/hide`; authenticated endpoint.
10	`pinConversation( userId: number, id: number, body: PinConversation BodyType)`	A method to pin or unpin a conversation for the current user. – `userId: number` – current user identifier. – `id: number` – conversation identifier. – `body.pinned: boolean` – target pin state, validated by `PinConversationBodyDto`. – *Returns:* `Promise<PinConversationResponseType>` – id and the new `pinnedAt` value. – Route: `PATCH /conversations/:id/pin`; authenticated endpoint.

4.13.2 ClassAttribute

No	Attribute	Type
01	`conversationService`	`ConversationService` – private readonly service dependency.

4.14 ConversationService Class

4.14.1 ClassMethod

No	Method	Description
01	`ConversationService( conversationRepository: ConversationRepository)`	A method to initialize the service with the conversation repository. – `conversationRepository`: `Conversation Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`buildConversationItem( conversation: RawConversationItem, userId: number)`	A private method to map a repository row to the public conversation item. – `conversation`: `RawConversationItem` – row with both participants, last message, and unread count. – `userId`: `number` – current user, used to select the other participant and pin time. – *Returns:* `ConversationListItemType` – normalized item with `otherUser`, `lastMessage`, `unreadCount`, and `pinnedAt`.
03	`getConversations(userId: number)`	A method to load and sort all visible conversations of a user. – `userId`: `number` – current user identifier. – *Returns:* `Promise<GetConversationsResponse Type>` – pinned conversations first, then by last-message time or creation time. – Any failure is mapped to `FailedToLoad ConversationsException`.
04	`getConversationItem( conversationId: number, userId: number)`	A method to rebuild one conversation item for a realtime update. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – recipient of the realtime item. – *Returns:* `Promise<ConversationListItemType | null>` – the item, or `null` when the conversation is not visible to that user.
05	`getMessages( userId: number, conversationId: number)`	A method to authorize a participant and return the message history. – `userId`: `number` – current user identifier. – `conversationId`: `number` – conversation identifier. – *Returns:* `Promise<DirectMessageType[]>` – non-deleted messages. – Throws `ConversationNotFoundException` when the user is not a participant; other failures become `FailedTo LoadConversationsException`.

No	Method	Description
06	`createConversation( userId: number, participantId: number)`	A method to create a conversation or return the existing pair. – `userId: number` – current user identifier. – `participantId: number` – the other user. – *Returns:* `Promise<CreateConversationResponse Type>` – id, participant1, participant2, and createdAt. – Rejects self-conversation with `CannotStart ConversationWithSelfException`; an existing hidden conversation is unhidden.
07	`sendMessage( userId: number, conversationId: number, message: string, attachment?: Message AttachmentType | null)`	A method to save a message and unhide the conversation for both participants. – `userId: number` – sender identifier. – `conversationId: number` – conversation identifier. – `message: string` – text; may be empty when an attachment exists. – `attachment: MessageAttachmentType | null | undefined` – optional file metadata. – *Returns:* `Promise<DirectMessageType>` – the saved message. – Throws `ConversationNotFoundException` or `FailedToSendMessageException`.
08	`uploadFile( userId: number, conversationId: number, file?: Express.Multer. File)`	A method to validate participation and upload a file to Cloudinary. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – `file: Express.Multer.File | undefined` – in-memory multipart file. – *Returns:* `Promise<MessageAttachmentType>` – secure URL, original name, byte size, and MIME type. – Throws `FailedToUploadFileException`, `ConversationNotFoundException`, or passes through a Nest `HttpException`.

No	Method	Description
09	`deleteMessage( userId: number, conversationId: number, messageId: number)`	A method to authorize a participant and soft-delete the sender's message. – `userId: number` – current user identifier; must be the sender. – `conversationId: number` – conversation identifier. – `messageId: number` – message identifier. – *Returns:* `Promise<{ id: number }>` – the deleted message id. – Throws `ConversationNotFoundException`, `MessageNotFoundException`, or `FailedToDeleteMessageException`.
10	`markAsRead( userId: number, conversationId: number)`	A method to mark messages from the other participant as read. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<{ count: number }>` – number of updated messages. – Throws `ConversationNotFoundException` or `FailedToMarkAsReadException`.
11	`hideConversation( userId: number, conversationId: number)`	A method to set the current participant's hidden timestamp. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<HideConversationResponseType>` – id and hidden flag. – Throws `ConversationNotFoundException` or `FailedToHideConversationException`.
12	`pinConversation( userId: number, conversationId: number, pinned: boolean)`	A method to set or clear the current participant's pin timestamp. – `userId: number` – current user identifier. – `conversationId: number` – conversation identifier. – `pinned: boolean` – true to pin, false to unpin. – *Returns:* `Promise<PinConversationResponseType>` – id and `pinnedAt`. – Throws `ConversationNotFoundException` or `FailedToPinConversationException`.

No	Method	Description
13	`ensureParticipant( conversationId: number, userId: number)`	A method used by the gateway to assert that a socket user belongs to a conversation. – `conversationId: number` – conversation identifier. – `userId: number` – socket user identifier. – *Returns:* the non-deleted `Conversation` row. – Throws `ConversationAccessDeniedException` when the user is not a participant.

4.14.2 ClassAttribute

No	Attribute	Type
01	`conversationRepository`	`ConversationRepository` – private readonly persistence dependency.
02	`logger`	`Logger` – private readonly NestJS logger; records attachment-upload failures.

4.15 ConversationRepository Class

4.15.1 ClassMethod

No	Method	Description
01	`ConversationRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma: PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findConversationBetween( userId1: number, user Id2: number)`	A method to find the non-deleted conversation for a user pair in either participant order. – `userId1, userId2: number` – the two users. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.

No	Method	Description
03	`getConversationsByUserId( userId: number)`	A method to query all conversations visible to a user with previews and unread counts. – `userId: number` – current user identifier. – *Returns:* conversation rows with both participant profiles, the latest non-deleted message, and a filtered message count. – A conversation is included only when the current participant's hidden timestamp is `null`.
04	`findConversationByIdFor User( conversationId: number, userId: number)`	A method to load a non-deleted conversation only when the user is a participant. – `conversationId: number` – conversation identifier. – `userId: number` – current user identifier. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.
05	`findConversationById( conversationId: number)`	A method to load a non-deleted conversation without a participant check. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<Conversation | null>` – the conversation or `null`.
06	`hideConversationForUser( conversationId: number, userId: number)`	A method to set the hidden timestamp of the participant who requested the hide. – `conversationId: number` – conversation identifier. – `userId: number` – participant identifier. – *Returns:* `Promise<boolean>` – true when the row was updated, false when the conversation or participant does not match.
07	`unhideConversation( conversationId: number)`	A method to clear both participants' hidden timestamps after new activity. – `conversationId: number` – conversation identifier. – *Returns:* the updated `Conversation` row.
08	`setPinnedForUser( conversationId: number, userId: number, pinned: boolean)`	A method to set or clear the participant's pin timestamp. – `conversationId: number` – conversation identifier. – `userId: number` – participant identifier. – `pinned: boolean` – true sets the current time, false clears it. – *Returns:* `Promise<Date | null>` – the new timestamp, or `null` when unpinned or not a participant.

No	Method	Description
09	`getMessages(conversationId: number)`	A method to query all non-deleted messages of a conversation in ascending time order. – `conversationId: number` – conversation identifier. – *Returns:* `Promise<DirectMessage[]>` – Prisma message rows.
10	`createConversation(participant1: number, participant2: number)`	A method to insert a conversation pair. – `participant1: number` – first participant (the creator in the service flow). – `participant2: number` – second participant. – *Returns:* `Promise<Conversation>` – the inserted row.
11	`createMessage(conversationId: number, senderId: number, message: string, attachment?: MessageAttachmentType)`	A method to insert a message and copy optional attachment metadata. – `conversationId: number` – conversation identifier. – `senderId: number` – sender identifier. – `message: string` – message text. – `attachment: MessageAttachmentType | undefined` – optional file metadata. – *Returns:* `Promise<DirectMessage>` – the inserted message.
12	`softDeleteMessage(conversationId: number, messageId: number, senderId: number)`	A method to set `deletedAt` only when the message belongs to the conversation and sender. – `conversationId: number` – conversation identifier. – `messageId: number` – message identifier. – `senderId: number` – sender identifier. – *Returns:* `Promise<boolean>` – true when one non-deleted row was updated.
13	`markMessagesAsRead(conversationId: number, userId: number)`	A method to mark messages from the other participant as read. – `conversationId: number` – conversation identifier. – `userId: number` – current user; messages sent by this user are excluded. – *Returns:* the Prisma batch payload with the updated row count.
14	`countUnread(conversationId: number, userId: number)`	A method to count unread messages from the other participant. – `conversationId: number` – conversation identifier. – `userId: number` – current user. – *Returns:* `Promise<number>` – count of non-deleted, unread messages not sent by the user.

4.15.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.16 ConversationsGateway Class

4.16.1 ClassMethod

No	Method	Description
01	`ConversationsGateway( conversationService: ConversationService, tokenService: TokenService)`	A method to initialize the gateway dependencies. – `conversationService`: `ConversationService` – shared use-case service. – `tokenService`: `TokenService` – verifies the access token during the socket handshake. – *Returns:* `void` – constructor, no return value.
02	`handleConnection(client: Socket)`	A method to authenticate a new socket and join its user room. – `client`: `Socket` – connecting Socket.IO client. – Reads `client.handshake.auth.token`, falling back to the bearer token in the authorization header; verifies it with `TokenService.verifyAccessToken`. – *Returns:* `Promise<void>` – stores `userId` in `client.data` and joins `user:<userId>`; disconnects on failure.
03	`handleDisconnect(client: Socket)`	A method to log a disconnected socket. – `client`: `Socket` – disconnected client. – *Returns:* `void` – no return value.
04	`handleSendMessage( client: Socket, payload: unknown)`	A method to handle the `message:send` event. – `client`: `Socket` – sender socket. – `payload`: `unknown` – validated by `SendMessage PayloadSchema`. – Ensures participation, saves the message, emits `message:new` to both user rooms, and refreshes both conversation lists. – *Returns:* `Promise<{ success: boolean; message?: DirectMessageType }>` – success flag and saved message; emits `message:error` on failure.

No	Method	Description
05	handleMarkAsRead(client: Socket, payload: unknown)	A method to handle the message:read event. – client: Socket – reader socket. – payload: unknown – validated by MarkAsRead PayloadSchema. – Calls markAsRead and emits conversation:updated to both participants. – *Returns:* Promise<{ success: boolean }> – success flag.
06	handleDeleteMessage(client: Socket, payload: unknown)	A method to handle the message:delete event. – client: Socket – sender socket. – payload: unknown – validated by DeleteMessage PayloadSchema. – Calls deleteMessage, emits message:deleted to both user rooms, and refreshes both conversation lists. – *Returns:* Promise<{ success: boolean }> – success flag; emits message:error on failure.
07	handleHideConversation(client: Socket, payload: unknown)	A method to handle the conversation:hide event. – client: Socket – current user's socket. – payload: unknown – validated by HideConversation PayloadSchema. – Calls hideConversation and emits conversation: hidden only to the current user's room. – *Returns:* Promise<{ success: boolean }> – success flag.
08	handleTyping(client: Socket, payload: unknown)	A method to handle the typing event. – client: Socket – sender socket. – payload: unknown – validated by TypingPayload Schema. – Ensures participation and forwards typing with conversationId and isTyping only to the other user's room. – *Returns:* Promise<{ success: boolean }> – success flag.

No	Method	Description
09	`emitConversationUpdated( conversationId: number, userId: number)`	A private method to push the latest conversation item to one user. – `conversationId`: `number` – conversation identifier. – `userId`: `number` – target user. – *Returns:* `Promise<void>` – emits `conversation: updated` when `getConversationItem` returns a visible item.

4.16.2 ClassAttribute

No	Attribute	Type
01	`server`	`Server` – Socket.IO server injected by `@WebSocketServer()`; used to address user rooms.
02	`logger`	`Logger` – private readonly NestJS logger for connection and event failures.
03	`conversationService`	`ConversationService` – private readonly service dependency.
04	`tokenService`	`TokenService` – private readonly access-token verifier.

4.17 ForumAdminController Class

4.17.1 ClassMethod

No	Method	Description
01	`ForumAdminController( adminService: ForumAdmin Service)`	A method to initialize the controller with the administration service. – `adminService`: `ForumAdminService` – service dependency. – *Returns:* `void` – constructor, no return value.

No	Method	Description
02	`getAdminStats(roleName: string)`	A method to return the forum dashboard counters. – `roleName: string` – current role name from `@User Active('roleName')`. – *Returns:* `Promise<ForumAdminStatsType>` – category, post, comment, report, pending-report, and user totals plus the five newest posts. – Route: `GET /forums/admin/stats`; administrator endpoint.
03	`getAdminCategoryLists( roleName: string, page: number, limit: number, search: string, deleted: string, sortBy: 'id' | 'name' | 'createdAt', sortOrder: 'asc' | 'desc')`	A method to list categories with search, deleted-state filtering, and sorting. – `roleName: string` – current role name. – `page, limit: number` – coerced with `Number()` and defaulted to 1 and 10. – `search: string` – optional case-insensitive name search. – `deleted: string` – optional `'true'` or `'false'` filter. – `sortBy, sortOrder: 'id' | 'name' | 'createdAt'` and `'asc' | 'desc'` – optional sort controls. – *Returns:* `Promise<ForumAdminCategoryListResponseType>` – categories and pagination. – Route: `GET /forums/admin/categories`; administrator endpoint.
04	`getAdminCategoryById( roleName: string, id: number)`	A method to return one active category. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponseType>` – category detail with public post count. – Route: `GET /forums/admin/categories/:id`; administrator endpoint.
05	`createAdminCategory( roleName: string, body: CreateForumCategoryBodyDto)`	A method to create a category. – `roleName: string` – current role name. – `body: CreateForumCategoryBodyDto` – name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category. – Route: `POST /forums/admin/categories`; administrator endpoint.

No	Method	Description
06	`updateAdminCategory( roleName: string, id: number, body: UpdateForum CategoryBodyDto)`	A method to rename or re-describe a category. – `roleName: string` – current role name. – `id: number` – category identifier. – `body: UpdateForumCategoryBodyDto` – optional name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Route: `PATCH /forums/admin/categories/:id`; administrator endpoint.
07	`deleteAdminCategory( roleName: string, id: number)`	A method to soft-delete a category and detach its posts. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Route: `DELETE /forums/admin/categories/:id`; administrator endpoint.
08	`restoreAdminCategory( roleName: string, id: number)`	A method to restore a soft-deleted category. – `roleName: string` – current role name. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Route: `PATCH /forums/admin/categories/:id/ restore`; administrator endpoint.
09	`getAdminCommentLists( roleName: string, page: number, limit: number, search: string)`	A method to list active comments for administration. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – `search: string` – optional case-insensitive content search. – *Returns:* `Promise<ForumAdminCommentList ResponseType>` – comments with author and parent-post summaries, plus pagination. – Route: `GET /forums/admin/comments`; administrator endpoint.

No	Method	Description
10	`getPendingPosts( roleName: string, page: number, limit: number)`	A method to list posts waiting for moderation. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<PendingForumPostListResponse Type>` – non-deleted `PENDING` posts, oldest first, with pagination. – Route: `GET /forums/admin/pending-posts`; administrator endpoint.
11	`getTrashPosts( roleName: string, page: number, limit: number)`	A method to list deleted or rejected posts. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<ForumTrashPostListResponse Type>` – posts with `deletedAt` set or moderation status `R EJECTED`. – Route: `GET /forums/admin/trash/posts`; administrator endpoint.
12	`getTrashComments( roleName: string, page: number, limit: number)`	A method to list deleted comments. – `roleName: string` – current role name. – `page, limit: number` – pagination, defaulted to 1 and 10. – *Returns:* `Promise<ForumTrashCommentList ResponseType>` – soft-deleted comments with author and parent-post summaries. – Route: `GET /forums/admin/trash/comments`; administrator endpoint.
13	`restoreTrashPost( roleName: string, id: number)`	A method to restore a post from the trash. – `roleName: string` – current role name. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – Route: `PATCH /forums/admin/trash/posts/:id/ restore`; administrator endpoint.

No	Method	Description
14	`restoreTrashComment(roleName: string, id: number)`	A method to restore a comment from the trash. – `roleName: string` – current role name. – `id: number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment. – Route: `PATCH /forums/admin/trash/comments/:id/restore`; administrator endpoint.
15	`approvePendingPost(roleName: string, userId: number, id: number)`	A method to approve a pending post. – `roleName: string` – current role name. – `userId: number` – reviewing administrator recorded in `reviewedById`. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the approved post. – Route: `PATCH /forums/admin/posts/:id/approve`; administrator endpoint.
16	`rejectPendingPost(roleName: string, userId: number, id: number)`	A method to reject a pending post and move it to the trash. – `roleName: string` – current role name. – `userId: number` – reviewing administrator. – `id: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the rejected post, now soft-deleted. – Route: `PATCH /forums/admin/posts/:id/reject`; administrator endpoint.

4.17.2 ClassAttribute

No	Attribute	Type
01	`adminService`	`ForumAdminService` – private readonly service dependency.

4.18 ForumAdminService Class

4.18.1 ClassMethod

No	Method	Description
01	`ForumAdminService( adminRepository: ForumAdminRepository)`	A method to initialize the service with the administration repository. – `adminRepository`: `ForumAdminRepository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`getAdminStats(roleName: string)`	A method to authorize an administrator and load dashboard counters. – `roleName`: `string` – must equal `RoleName.ADMIN`. – *Returns:* `Promise<ForumAdminStatsType>` – counters and the five newest posts. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
03	`getAdminCommentLists( roleName: string, page: number, limit: number, search?: string)`	A method to authorize an administrator and return a page of active comments. – `roleName`: `string` – must equal `RoleName.ADMIN`. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional content filter. – *Returns:* `Promise<ForumAdminCommentList ResponseType>` – comments and pagination with `total Pages`.
04	`getAdminCategoryLists( roleName: string, page: number, limit: number, search?: string, sortBy?: 'id' | 'name' | 'createdAt', sortOrder?: 'asc' | 'desc', deleted?: string)`	A method to authorize an administrator and return a page of categories. – `roleName`: `string` – must equal `RoleName.ADMIN`. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional name filter. – `sortBy`, `sortOrder`: optional sort controls; repository defaults are `id` and `desc`. – `deleted`: `string | undefined` – `'true'` lists only deleted, `'false'` only active. – *Returns:* `Promise<ForumAdminCategoryList ResponseType>` – categories and pagination.

No	Method	Description
05	`getAdminCategoryById( roleName: string, categoryId: number)`	A method to authorize an administrator and load one active category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<ForumCategoryDetailResponse Type>` – category detail. – Throws `ForumCategoryNotFoundException` when absent, or `ForumReportForbiddenException` for other failures.
06	`createAdminCategory( roleName: string, body: CreateForum CategoryBodyType)`	A method to authorize an administrator and create a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `body: CreateForumCategoryBodyType` – name and optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category. – Unknown repository errors are rethrown unchanged.
07	`updateAdminCategory( roleName: string, categoryId: number, body: UpdateForum CategoryBodyType)`	A method to authorize an administrator and update a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – `body: UpdateForumCategoryBodyType` – optional name and description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Unknown repository errors are rethrown unchanged.
08	`deleteAdminCategory( roleName: string, categoryId: number)`	A method to authorize an administrator and soft-delete a category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Throws `FailedToDeleteForumCategoryException` for an unexpected failure.

No	Method	Description
09	`restoreAdminCategory( roleName: string, categoryId: number)`	A method to authorize an administrator and restore a deleted category. – `roleName: string` – must equal `RoleName.ADMIN`. – `categoryId: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Throws `FailedToRestoreForumCategoryException` for an unexpected failure.
10	`getPendingPosts( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the moderation queue. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<PendingForumPostListResponseType>` – pending posts and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
11	`reviewPendingPost( roleName: string, postId: number, status: 'APPROVED' | 'REJECTED', adminId: number)`	A method to authorize an administrator and apply a moderation decision. – `roleName: string` – must equal `RoleName.ADMIN`. – `postId: number` – post identifier. – `status: 'APPROVED' | 'REJECTED'` – decision passed by the approve/reject routes. – `adminId: number` – reviewing administrator. – *Returns:* `Promise<ForumPostType>` – the reviewed post. – A Prisma `P2025` error is mapped to `ForumPostNotFoundException`; other Prisma errors become `FailedToReviewForumPostException`.
12	`getTrashPosts( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the post trash. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<ForumTrashPostListResponseType>` – deleted or rejected posts and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.

No	Method	Description
13	`getTrashComments( roleName: string, page: number, limit: number)`	A method to authorize an administrator and load the comment trash. – `roleName: string` – must equal `RoleName.ADMIN`. – `page, limit: number` – pagination. – *Returns:* `Promise<ForumTrashCommentList ResponseType>` – deleted comments and pagination. – Throws `ForumReportForbiddenException` for a non-admin or a repository failure.
14	`restoreTrashPost( roleName: string, post Id: number)`	A method to authorize an administrator and restore a post. – `roleName: string` – must equal `RoleName.ADMIN`. – `postId: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – Throws `FailedToRestoreForumPostException` for an unexpected failure.
15	`restoreTrashComment( roleName: string, commentId: number)`	A method to authorize an administrator and restore a comment. – `roleName: string` – must equal `RoleName.ADMIN`. – `commentId: number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment. – A Prisma `P2025` error is mapped to `ForumCommentNot FoundException`; other Prisma errors become `FailedTo RestoreForumCommentException`.

4.18.2 ClassAttribute

No	Attribute	Type
01	`adminRepository`	`ForumAdminRepository` – private readonly persistence dependency.

4.19 ForumAdminRepository Class

4.19.1 ClassMethod

No	Method	Description
01	`ForumAdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`getPendingPosts(page: number, limit: number)`	A method to query the moderation queue and its total in one transaction. – `page, limit`: `number` – pagination. – *Returns:* `Promise<{ posts: PendingForumPostType[]; total: number }>` – non-deleted `PENDING` posts, oldest first, with author summary. – `moderationCategories` is normalized to `string[] | null`.
03	`reviewForumPost( postId: number, status: 'APPROVED' | 'REJECTED', adminId: number)`	A method to apply a moderation decision only to a non-deleted `PENDING` post. – `postId`: `number` – post identifier. – `status`: `'APPROVED' | 'REJECTED'` – target moderation status. – `adminId`: `number` – reviewer recorded in `reviewedById`. – *Returns:* `Promise<ForumPostType>` – the reviewed post. – `REJECTED` also sets `deletedAt`; a zero-row update throws `ForumPostNotFoundException`.
04	`getAdminStats()`	A method to count forum entities and load the five newest posts in one transaction. – *Returns:* `Promise<ForumAdminStatsType>` – active category, post, comment, and user counts; total and pending reports; five recent posts. – Posts, comments, categories, and users exclude soft-deleted rows; report totals include all rows.
05	`getAdminCommentLists( page: number, limit: number, search?: string)`	A method to query a page of active comments and the matching total in one transaction. – `page, limit`: `number` – pagination. – `search`: `string | undefined` – optional case-insensitive content filter. – *Returns:* `Promise<{ comments: ForumAdminCommentType[]; total: number }>` – comments with author and parent-post summaries, newest first.

No	Method	Description
06	`getAdminCategoryLists( page: number, limit: number, search?: string, sortBy?: 'id' | 'name' | 'createdAt', sortOrder?: 'asc' | 'desc', deleted?: string)`	A method to query a page of categories and the matching total in one transaction. – `page`, `limit`: `number` – pagination. – `search`: `string | undefined` – optional case-insensitive name filter. – `sortBy`, `sortOrder`: optional sort controls; defaults are `'id'` and `'desc'`. – `deleted`: `string | undefined` – `'true'` selects only deleted rows, `'false'` only active rows, omitted selects both. – *Returns:* `Promise<{ categories: ForumAdminCategoryType[]; total: number }>` – categories with `postCount` and pagination total.
07	`getAdminCategoryById(id: number)`	A method to load one active category with its public post count. – `id`: `number` – category identifier. – *Returns:* `Promise<ForumCategoryType | null>` – the category detail or `null`.
08	`createAdminCategory(data: { name: string; description?: string | null })`	A method to create a category with a unique name and generated slug. – `data.name`: `string` – category name. – `data.description`: `string | null | undefined` – optional description. – *Returns:* `Promise<ForumCategoryType>` – the created category with `postCount = 0`. – Throws `ForumCategoryAlreadyExistsException` for a case-insensitive active-name conflict; slug collisions receive a numeric suffix.
09	`updateAdminCategory( id: number, data: UpdateForumCategoryBodyType)`	A method to update a category and regenerate its slug when the name changes. – `id`: `number` – category identifier. – `data`: `UpdateForumCategoryBodyType` – optional name and description. – *Returns:* `Promise<ForumCategoryType>` – the updated category. – Throws `ForumCategoryNotFoundException` or `ForumCategoryAlreadyExistsException`.

No	Method	Description
10	`deleteAdminCategory(id: number)`	A method to detach posts and soft-delete a category in one transaction. – `id: number` – category identifier. – *Returns:* `Promise<{ message: string }>` – deletion confirmation. – Non-deleted posts in the category are reassigned to `categoryId = null` before the category receives `deletedAt`.
11	`restoreAdminCategory(id: number)`	A method to restore a soft-deleted category after a name-conflict check. – `id: number` – category identifier. – *Returns:* `Promise<ForumCategoryType>` – the restored category. – Throws `ForumCategoryNotFoundException` or `ForumCategoryAlreadyExistsException`.
12	`getTrashPosts(page: number, limit: number)`	A method to query deleted or rejected posts and the matching total in one transaction. – `page, limit: number` – pagination. – *Returns:* `Promise<{ posts: ForumTrashPostType[]; total: number }>` – trash posts with `deletedAt` and author summary. – Ordering is by `deletedAt` descending with null values last.
13	`getTrashComments(page: number, limit: number)`	A method to query soft-deleted comments and the matching total in one transaction. – `page, limit: number` – pagination. – *Returns:* `Promise<{ comments: ForumTrashCommentType[]; total: number }>` – trash comments with `deletedAt` and parent-post summary, newest deletion first.
14	`restoreTrashPost(postId: number)`	A method to restore a post from the trash. – `postId: number` – post identifier. – *Returns:* `Promise<ForumPostType>` – the restored post. – A deleted `APPROVED` post becomes visible again; a `REJECTED` post returns to `PENDING` and its reviewer fields are cleared.

No	Method	Description
15	`restoreTrashComment( commentId: number)`	A method to clear `deletedAt` on a trashed comment. – `commentId`: `number` – comment identifier. – *Returns:* `Promise<ForumAdminCommentType>` – the restored comment with author and parent-post summaries. – Throws `ForumCommentNotFoundException` when no deleted row matches.

4.19.2 ClassAttribute

No	Attribute	Type
01	`prisma`	`PrismaService` – private readonly database client.

4.20 IdentityVerificationsAdminController Class

4.20.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminController( service: Identity VerificationsAdmin Service)`	A method to initialize the controller with the administration service. – `service`: `IdentityVerificationsAdminService` – service dependency. – *Returns:* `void` – constructor, no return value.
02	`list(query: Identity VerificationAdminFilter Dto)`	A method to return a filtered, paginated document list. – `query`: `IdentityVerificationAdminFilterDto` – page, limit, optional status, and optional search. – *Returns:* `Promise<IdentityVerificationAdmin ListResponseType>` – documents and pagination. – Route: `GET /admin/identity-verifications`.

No	Method	Description
03	`detail(id: number)`	A method to return one document with user and reviewer detail. – `id: number` – document identifier parsed by `ParseIntPipe`. – *Returns:* `Promise<IdentityVerificationAdminDetailType>` – document, user profile, freelancer verification state, and reviewing administrator. – Route: `GET /admin/identity-verifications/:id`.
04	`file(id: number, side: string | undefined, response: Response)`	A method to stream a legacy `local://` document image. – `id: number` – document identifier. – `side: string | undefined` – `'back'` selects the back image; any other value selects the front. – `response: Response` – Express response used to set headers. – *Returns:* `Promise<StreamableFile>` – file stream; sets inline `Content-Disposition` and `application/octet-stream`. – Route: `GET /admin/identity-verifications/:id/file`.
05	`approve(id: number, adminId: number, body: ApproveIdentityVerificationBodyDto)`	A method to approve a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator from `@UserActive('userId')`. – `body: ApproveIdentityVerificationBodyDto` – optional `reviewNotes` up to 1000 characters. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Route: `PATCH /admin/identity-verifications/:id/approve`.
06	`reject(id: number, adminId: number, body: RejectIdentityVerificationBodyDto)`	A method to reject a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator. – `body: RejectIdentityVerificationBodyDto` – optional trimmed `reviewNotes` up to 1000 characters. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Route: `PATCH /admin/identity-verifications/:id/reject`.

4.20.2 ClassAttribute

No	Attribute	Type
01	`service`	`IdentityVerificationsAdmin Service` – private readonly service dependency.

4.21 IdentityVerificationsAdminService Class

4.21.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminService( repository: IdentityVerificationsAdmin Repository)`	A method to initialize the service with the identity-verification repository. – `repository`: `IdentityVerificationsAdmin Repository` – persistence dependency. – *Returns:* `void` – constructor, no return value.
02	`presentFileUrl(document: T)`	A private generic method to replace legacy `local://` file paths with API URLs. – `document`: `T` – object with `id`, `fileUrl`, and optional `backFileUrl`. – *Returns:* the same object with `fileUrl` set to the file route and, when needed, `backFileUrl` set to the same route with `?side=back`. – Non-local URLs are returned unchanged.
03	`presentList(documents: T[])`	A private method to map `presentFileUrl` over a document list. – `documents`: `T[]` – list of document objects. – *Returns:* the mapped list.
04	`list(params: { page: number; limit: number; status?: string; search?: string })`	A method to load a filtered page of documents. – `params`: object – page, limit, optional status string, and optional search. – *Returns:* `Promise<IdentityVerificationAdmin ListResponseType>` – documents with presentation URLs and pagination. – `status` is cast to the Prisma `VerificationStatus` union before the repository call.

No	Method	Description
05	`detail(id: number)`	A method to load one document and present its file URLs. – `id: number` – document identifier. – *Returns:* `Promise<IdentityVerificationAdminDetailType>` – the document detail. – Throws `IdentityVerificationNotFoundException`.
06	`review( id: number, adminId: number, status: ReviewableStatus, reviewNotes: string | null)`	A method to approve or reject a pending document. – `id: number` – document identifier. – `adminId: number` – reviewing administrator. – `status: ReviewableStatus` – `APPROVED` or `REJECTED`. – `reviewNotes: string | null` – optional reviewer note. – *Returns:* `Promise<IdentityVerificationAdminActionResponseType>` – the reviewed document detail. – Throws `IdentityVerificationNotFoundException` or `IdentityVerificationAlreadyReviewedException`.
07	`getLocalFile( id: number, side: 'front' | 'back' = 'front' )`	A method to resolve and validate a legacy local document file. – `id: number` – document identifier. – `side: 'front' | 'back'` – which stored URL to use; default front. – *Returns:* `Promise<{ absolutePath: string; fileName: string }>` – resolved path and base filename. – Throws `IdentityVerificationNotFoundException` or `IdentityVerificationFileInvalidException`; rejects paths escaping the uploads root.

4.21.2 ClassAttribute

No	Attribute	Type
01	`repository`	`IdentityVerificationsAdminRepository` – private readonly persistence dependency.

4.22 IdentityVerificationsAdminRepository Class

4.22.1 ClassMethod

No	Method	Description
01	`IdentityVerifications AdminRepository( prisma: PrismaService)`	A method to initialize the repository with the shared Prisma service. – `prisma`: `PrismaService` – database client. – *Returns:* `void` – constructor, no return value.
02	`findDocuments(params: IdentityVerificationAdmin ListParams)`	A method to query a filtered page of active documents and the matching total. – `params`: `IdentityVerificationAdminListParams` – page, limit, optional status, and optional search. – *Returns:* `Promise<{ documents: IdentityVerificationAdminListItemType[]; total: number }>` – documents with user summary, newest first. – Search matches user email or profile display name case-insensitively; `deletedAt` must be `null`.
03	`findDocumentById(id: number)`	A method to load the full detail projection of an active document. – `id`: `number` – document identifier. – *Returns:* `Promise<IdentityVerificationAdmin DetailType | null>` – document, reviewer, user, and freelancer summary, or `null`.
04	`findDocumentRaw(id: number)`	A method to load the minimal fields needed for review and file streaming. – `id`: `number` – document identifier. – *Returns:* `Promise<{ id: number; userId: number; fileUrl: string; backFileUrl: string | null; status: VerificationStatus } | null>` – raw document fields, or `null`.

No	Method	Description
05	reviewDocument(id: number, adminId: number, status: VerificationStatus, reviewNotes: string | null)	A method to persist a review and synchronize the freelancer verification flag in one transaction. – id: number – document identifier. – adminId: number – reviewing administrator. – status: VerificationStatus – APPROVED or REJECTED. – reviewNotes: string | null – reviewer note. – *Returns:* Promise<IdentityVerificationAdminDetailType | null> – the updated detail, or null. – Updates the document status, reviewer, notes, and reviewedAt; sets FreelancerProfile.idVerified to true only for APPROVED. Transaction limits are 10s wait and 15s timeout.

4.22.2 ClassAttribute

No	Attribute	Type
01	prisma	PrismaService – private readonly database client.